

长广溪智能制造（无锡）有限公司

Changguangxi Intelligent Manufacturing (Wuxi) Co., Ltd.

协作机器人二次开发接口文档 （Python）

文档版本：v2.20e

发布日期：2025 年 10 月 27 日

文档编号：CGXi-QM-CR-0013



长广溪智能制造（无锡）有限公司

Changguangxi Intelligent Manufacturing(Wuxi) Co., Ltd.



长广溪智能制造（无锡）有限公司（简称长广溪智造）保留对此文件修改之权利且不另行通知。长广溪智能制造（无锡）有限公司所提供之信息相信为正确且可靠之信息，但并不保证本文件中绝无错误。请于向长广溪智能制造（无锡）有限公司提出订单前，自行确定所使用之相关技术文件及规格为最新之版本。若因贵公司使用本公司之文件或产品，而涉及第三人之专利或著作权等智能财产权之应用及配合时，则应由贵公司负责取得同意及授权，本公司仅单纯贩售产品，上述关于同意及授权，非属本公司应为保证之责任。又未经长广溪智能制造（无锡）有限公司之正式书面许可，本公司之所有产品不得使用于医疗器材，维持生命系统及飞航等相关设备。

注意事项

本手册仅适用于长广溪智能制造（无锡）有限公司 G 系列协作机器人。

目录

一、SDK 介绍	12
二、数据结构	12
2.1 机械臂句柄	12
2.2 接口调用返回值	12
2.3 坐标系选择	13
2.4 点转换类型	13
2.5 位姿变换	13
2.6 运动触发模式	14
2.7 TCP 偏移量	14
2.8 运动参数	14
2.9 简化版运动参数	15
2.10 圆弧运动参数	15
2.11 运动模式	15
2.12 关节状态	16
2.13 机械臂状态	16
2.14 安全模式	17
2.15 程序状态	17
2.16 弹窗信息	17
2.17 版本信息	18
2.18 Lua 变量	18
2.19 变量元素	18
2.20 变量信息	18
2.21 消息来源	19
2.22 警告等级	19
2.23 世界时间	19
2.24 日志信息	20
2.25 版本信息	20
2.26 负载	20
2.27 校验码检查	20
2.28 串口通信	21
2.29 波特率	21
2.30 串口配置	21
2.31 IO 类型	22
2.32 安装角度	22
2.33 系统功能输入	22
2.34 系统功能输出	22
2.35 IO 选项卡输出	23
2.36 输出信号状态 (自动复位)	23
2.37 输入信号	23

2.38 输出信号	23
2.39 工具 IO 类型	24
2.40 模拟量类型	24
2.41 模拟量输入	24
2.42 模拟量输出	24
2.43 工具电压	24
2.44 编码器配置	24
2.45 BOOL 寄存器	25
2.46 寄存器	25
2.47 工具端信息	25
2.48 点坐标系	25
2.49 线坐标系	25
2.50 面坐标系	26
2.51 机器人限值等级	26
2.52 碰撞后处理模式	26
2.53 关节限值模式	26
2.54 安全限值模式	27
2.55 安全限值	27
2.56 关节限值	27
2.57 安全平面	27
2.58 安全 IO 输入信号	28
2.59 安全 IO 输出信号	28
2.60 控制柜网络配置	28
2.61 MODBUS 地址映射功能	28
2.62 MODBUS 地址映射	29
2.63 MODBUS 连接状态	29
2.64 MODBUS 参数配置	29
2.65 通讯变量选择	29
2.66 脚本程序状态	29
2.67 串口类型	30
2.68 轨迹点数据	30
2.69 轨迹参数	30
2.70 轨迹数据	30
2.71 下载轨迹数据	30
2.72 轨迹记录参数	30
2.73 轨迹记录状态	31
2.74 轨迹运行状态	31
2.75 控制器模式	31
2.76 力控设置	31
2.77 力控传感器 tcp	31
2.78 力控传感器负载	32

2.79 力控传感器信息	32
2.80 通讯配置	32
2.81 力控传感器工具设置	32
2.82 力控配置	32
2.83 力控控制参数	33
2.84 力控数据	33
2.85 机器人状态数据	33
2.86 机械臂和扩展轴数据	33
2.87 扩展轴关节移动参数	33
2.88 扩展轴点动运动参数	34
2.89 扩展轴运动参数	34
2.90 扩展轴类型	35
2.91 扩展轴配置	35
2.92 常用宏定义	35
三、接口函数	36
3.1 基础	36
3.1.1 基础接口	36
3.1.1.1 机器人创建连接	36
3.1.1.2 机器人断开连接	37
3.1.1.3 机器人关机	38
3.1.1.4 机械臂上电	38
3.1.1.5 机械臂断电	39
3.1.1.6 机械臂使能	40
3.1.1.7 机械臂关使能	40
3.1.1.8 机械臂故障复位	41
3.1.1.9 获取控制柜状态	42
3.1.1.10 写机械臂速度百分比	43
3.1.1.11 读机械臂速度百分比	43
3.1.1.12 获取 SDK 版本号	44
3.1.1.13 获取机械臂状态数据	45
3.1.2 程序控制	46
3.1.2.1 下载脚本程序	46
3.1.2.2 下载脚本程序 (加密)	47
3.1.2.3 上传脚本程序	48
3.1.2.4 运行程序	49
3.1.2.5 停止程序运行	50
3.1.2.6 暂停程序运行	51
3.1.2.7 读取脚本当前运行行号	52
3.1.2.8 读脚本运行状态	53
3.1.2.9 上传工程文件	53
3.1.2.10 下载工程文件	54

3.1.2.11 是否存在弹窗	55
3.1.2.12 读取弹窗信息	56
3.1.2.13 设置输入弹窗信息	57
3.1.2.14 关闭弹窗	58
3.2 运动模块	59
3.2.1 运动接口	59
3.2.1.1 轴空间运动 moveJ	59
3.2.1.2 轴空间运动 movejoint	60
3.2.1.3 直线运动 moveL	61
3.2.1.4 直线运动 moveline	62
3.2.1.5 点动运动	64
3.2.1.6 圆弧运动	65
3.2.1.7 运动控制	67
3.2.1.8 设置阻塞移动中参数阈值	68
3.2.1.9 PointControlParaSimple 数据转换	69
3.2.1.10 设置软件自由驱动启动	70
3.2.1.11 读机械臂运动状态	71
3.2.1.12 读取移动缓存区中移动指令的数量	71
3.2.2 运动状态信息	72
3.2.2.1 读实际 TCP 位置	72
3.2.2.2 读目标 TCP 位置	73
3.2.2.3 读实际 TCP 速度	74
3.2.2.4 读目标 TCP 速度	75
3.2.2.5 读实际 TCP 加速度	76
3.2.2.6 读当前使用的 TCP 偏移信息	76
3.2.2.7 读取当前使用负载信息	77
3.2.3 运动配置	78
3.2.3.1 读所有 TCP 偏移信息列表	78
3.2.3.2 读取所有负载信息	79
3.2.3.3 获取指定 TCPoffset 的位姿	81
3.2.4 关节状态	81
3.2.4.1 读实际关节位置	82
3.2.4.2 读目标关节位置	82
3.2.4.3 读实际关节速度	83
3.2.4.4 读目标关节速度	84
3.2.4.5 读实际关节加速度	85
3.2.4.6 读目标关节加速度	86
3.2.4.7 读实际关节电机电流	86
3.2.4.8 读目标关节电机电流	87
3.2.4.9 读实际关节转矩	88
3.2.4.10 读实际关节采集的母线电压	89

3.2.4.11 读关节温度	90
3.2.4.12 读关节模式	91
3.2.5 轨迹复现	91
3.2.5.1 读取控制柜当前存在的轨迹索引	91
3.2.5.2 上传轨迹	92
3.2.5.3 下载轨迹	93
3.2.5.4 轨迹控制	100
3.2.5.5 轨迹数据转换为轨迹文件	101
3.2.5.6 轨迹文件转换为轨迹数据	102
3.2.5.7 设置轨迹记录参数	103
3.2.5.8 获取当前轨迹记录状态	104
3.2.5.9 获取当前轨迹运行状态	105
3.2.6 扩展轴运动	106
3.2.6.1 扩展轴关节移动	106
3.2.6.2 扩展轴直线移动	107
3.2.6.3 扩展轴点动移动	108
3.2.6.4 扩展轴运动	109
3.3 系统属性	110
3.3.1 硬件信息	110
3.3.1.1 读取控制柜硬件序列号	110
3.3.1.2 读取整臂序列号	111
3.3.1.3 获取版本信息	112
3.3.2 本体参数	113
3.3.2.1 读机械臂 DH 参数	113
3.3.2.2 读机械臂标准 DH 参数	114
3.3.2.3 读机械臂动力学参数	114
3.3.2.4 读机械臂零位补偿值	115
3.3.2.5 读机械臂减速比	116
3.3.3 系统信息	117
3.3.3.1 读机械臂当前状态	117
3.3.3.2 获取机械臂日志数量	118
3.3.3.3 获取机械臂实时日志	119
3.3.3.4 获取机械臂历史所有日志	120
3.3.3.5 日志功能是否开启	121
3.3.3.6 日志大小设置	121
3.3.3.7 读系统时间	122
3.3.3.8 读取机型型号	123
3.3.3.9 读取控制器型号	123
3.3.3.10 触发黑匣子数据	124
3.3.3.11 获取配置改变标志	125
3.3.3.12 触发系统更新	126

3.4 变量	127
3.4.1 数字量	127
3.4.1.1 读控制柜标准数字输出	127
3.4.1.2 写控制柜标准数字输出	128
3.4.1.3 读控制柜可配置数字输出	129
3.4.1.4 读控制柜所有可配置数字输出	130
3.4.1.5 写控制柜可配置数字输出	130
3.4.1.6 读工具端数字输出	131
3.4.1.7 写工具端数字输出	132
3.4.1.8 读控制柜标准数字输入	133
3.4.1.9 读控制柜可配置数字输入	134
3.4.1.10 读控制柜所有可配置数字输入	135
3.4.1.11 读工具端数字输入	136
3.4.2 模拟量	136
3.4.2.1 读控制柜模拟量输出	137
3.4.2.2 写控制柜模拟量输出	137
3.4.2.3 读工具端模拟量输入	138
3.4.2.4 读控制柜模拟量输入	139
3.4.2.5 读机械臂控制柜和工具端上所有输入数字量和模拟量	140
3.4.2.6 读机械臂控制柜和工具端上所有输出数字量和模拟量	141
3.4.3 通讯变量	142
3.4.3.1 读 bool 寄存器	143
3.4.3.2 写 bool 寄存器	144
3.4.3.3 读 int16 寄存器	145
3.4.3.4 写 int16 寄存器	146
3.4.3.5 读 int32 寄存器	147
3.4.3.6 写 int32 寄存器	148
3.4.3.7 读 float 寄存器	149
3.4.3.8 写 float 寄存器	150
3.4.4 脚本变量	150
3.4.4.1 读安装变量	151
3.4.4.2 修改安装变量	151
3.4.4.3 读取程序变量数据	152
3.4.4.4 修改程序变量数据	153
3.4.5 其它 IO	154
3.4.5.1 获取编码器计数数量	154
3.4.5.2 设置监控界面的工具输出电压	155
3.4.5.3 读取监控界面的工具输出电压	156
3.5 数学模块	157
3.5.1 位姿变换	157
3.5.1.1 位姿转换	157

3.5.1.2 位姿求逆	158
3.5.1.3 轴角转欧拉角	158
3.5.1.4 欧拉角转轴角	159
3.5.1.5 TCP 位姿转法兰位姿	160
3.5.1.6 基坐标系下位姿转移到用户坐标系下	160
3.5.1.7 目标点旋转变换	161
3.5.2 运动学	162
3.5.2.1 正解	162
3.5.2.2 逆解	163
3.6 配置	164
3.6.1 安装	164
3.6.1.1 设置安装角度	164
3.6.1.2 读取安装角度	165
3.6.2 TCP	166
3.6.2.1 读取 TCP 个数	166
3.6.2.2 读取 TCP 数据	167
3.6.2.3 读取 TCP 数据(通过名称)	168
3.6.2.4 增加 TCP	169
3.6.2.5 删除 TCP	170
3.6.2.6 编辑 TCP 数据	170
3.6.2.7 读取当前激活的 TCP 索引	171
3.6.2.8 激活 TCP	172
3.6.2.9 计算 TCP 位置	173
3.6.2.10 计算 TCP 角度	174
3.6.3 负载	175
3.6.3.1 读取负载个数	175
3.6.3.2 读取负载数据	176
3.6.3.3 读取负载数据(通过名称)	176
3.6.3.4 增加负载	177
3.6.3.5 删除负载	178
3.6.3.6 编辑负载	179
3.6.3.7 读取当前激活的负载索引	180
3.6.3.8 激活负载	181
3.6.4 初始位/包装位	182
3.6.4.1 设置初始位	182
3.6.4.2 读取初始位	183
3.6.4.3 删除初始位	184
3.6.4.4 读取包装位	184
3.6.5 启动	185
3.6.5.1 设置机器人是否自动上电使能	185
3.6.5.2 读取机器人自动上电状态	186

3.6.6 控制参数	187
3.6.6.1 设置机器人拖动阻尼	187
3.6.6.2 读取机器人拖动阻尼	188
3.6.6.3 设置转矩异常门限倍率	189
3.6.6.4 读转矩异常门限倍率	190
3.6.6.5 设置扩展轴步长及速度倍率	191
3.6.6.6 读扩展轴步长及速度倍率	191
3.6.7 机器人限值	192
3.6.7.1 设置限制模式	192
3.6.7.2 读取限制模式	193
3.6.7.3 设置自定义模式下参数	194
3.6.7.4 读取自定义模式下参数	195
3.6.7.5 设置碰撞后处理方式	196
3.6.7.6 读取碰撞后处理方式	197
3.6.8 关节限值	197
3.6.8.1 设置关节限值	198
3.6.8.2 读取关节限值	199
3.6.9 安全平面	199
3.6.9.1 读取安全平面个数	200
3.6.9.2 读取安全平面信息	200
3.6.9.3 增加安全平面	201
3.6.9.4 删除安全平面	202
3.6.9.5 修改安全平面数据	203
3.6.10 安全 I/O	204
3.6.10.1 设置可配置输入信号	204
3.6.10.2 读取可配置输入信号	205
3.6.10.3 设置可配置输出信号	206
3.6.10.4 读取可配置输出信号	207
3.6.11 示教器控制	207
3.6.11.1 设置是否启用示教器	208
3.6.11.2 读取当前是否启用示教器	208
3.6.12 IO 变量	209
3.6.12.1 设置数字输入信号配置	209
3.6.12.2 读取数字输入信号配置	210
3.6.12.3 设置数字输出信号配置	211
3.6.12.4 读取数字输出信号配置	212
3.6.12.5 设置工具 IO 类型	213
3.6.12.6 读取工具 IO 类型	214
3.6.12.7 设置模拟量输入信号类型	215
3.6.12.8 读取模拟量输入信号类型	216
3.6.12.9 设置模拟量输出信号类型	217

3.6.12.10 读取模拟量输出信号类型	218
3.6.12.11 设置工具电压输出值	219
3.6.12.12 读取工具电压输出值	220
3.6.12.13 设置编码器配置	220
3.6.12.14 读取编码器配置	221
3.6.13 通讯变量	222
3.6.13.1 设置位寄存器配置	222
3.6.13.2 读取位寄存器配置	224
3.6.13.3 读取所有位寄存器配置	225
3.6.13.4 设置 16 位整数寄存器变量名	225
3.6.13.5 读取 16 位整数寄存器变量名	226
3.6.13.6 读取所有 16 位整数寄存器变量名	227
3.6.13.7 设置 32 位整数寄存器变量名	228
3.6.13.8 读取 32 位整数寄存器变量名	229
3.6.13.9 读取所有 32 位整数寄存器变量名	230
3.6.13.10 设置浮点寄存器变量名	231
3.6.13.11 读取浮点寄存器变量名	232
3.6.13.12 读取所有浮点寄存器变量名	233
3.6.14 安装变量	234
3.6.14.1 读取安装变量个数	234
3.6.14.2 读取安装变量数据	235
3.6.14.3 增加安装变量	235
3.6.14.4 删除安装变量	236
3.6.14.5 修改安装变量	237
3.6.15 坐标系	238
3.6.15.1 读取点坐标系个数	238
3.6.15.2 读取点坐标系数据	239
3.6.15.3 读取点坐标系数据(通过名称)	240
3.6.15.4 增加点坐标系	241
3.6.15.5 删除点坐标系	242
3.6.15.6 修改点坐标系数据	243
3.6.15.7 读取线坐标系个数	244
3.6.15.8 读取线坐标系数据	244
3.6.15.9 读取线坐标系数据(通过名称)	245
3.6.15.10 增加线坐标系	246
3.6.15.11 删除线坐标系	248
3.6.15.12 修改线坐标系数据	249
3.6.15.13 读取面坐标系个数	250
3.6.15.14 读取面坐标系数据	251
3.6.15.15 读取面坐标系数据(通过名称)	252
3.6.15.16 增加面坐标系	253

3.6.15.17 删除面坐标系	255
3.6.15.18 修改面坐标系数据	255
3.6.15.19 读取基坐标系数据	257
3.6.15.20 读取工具坐标系数据	258
3.6.15.21 计算点坐标系	259
3.6.15.22 计算线坐标系	260
3.6.15.23 计算面坐标系	261
3.6.16 串口通讯	262
3.6.16.1 设置串口配置	262
3.6.16.2 读取串口配置	263
3.6.17 网络	264
3.6.17.1 设置控制柜网络配置	264
3.6.17.2 读取控制柜网络配置	265
3.6.17.3 设置控制柜从站配置	266
3.6.17.4 读取控制柜从站配置	267
3.6.18 modbus 主站	268
3.6.18.1 读取从站个数	268
3.6.18.2 读取从站数据	269
3.6.18.3 添加从站	270
3.6.18.4 删除从站	271
3.6.18.5 修改从站数据	272
3.6.18.6 从站连接/断开	273
3.6.18.7 获取从站连接状态	274
3.6.18.8 读取地址映射数量	275
3.6.18.9 读取地址映射数据	275
3.6.18.10 添加地址映射	276
3.6.18.11 删除地址映射	278
3.6.18.12 修改地址映射数据	278
3.6.19 力控	280
3.6.19.1 设置力控配置	280
3.6.19.2 读力控配置	281
3.6.19.3 开启力控并下发相关设置	282
3.6.19.4 关闭力控	283
3.6.19.5 设置力控控制参数	284
3.6.19.6 读力控控制参数	285
3.6.19.7 读力控数据	286
3.6.20 扩展轴	287
3.6.20.1 设置扩展轴使能状态	287
3.6.20.2 读取扩展轴使能状态	287
3.6.20.3 读取机械臂和扩展轴数据	288
3.6.20.4 获取扩展轴配置	289

3.6.20.5 设置扩展轴配置	290
3.6.20.6 增加扩展轴	291
3.6.20.7 删除扩展轴	292
四、综合示例	294
4.1 机械臂使能	294
4.2 工程控制	295
4.3 移动控制	296
4.4 轨迹再现	300
4.5 安装变量操作	303
4.6 点坐标系操作	305
4.7 输出弹窗	307
4.8 数学变量	309
4.9 TCP 操作	310
附录 A 速度和加速度范围	313

一、SDK 介绍

CGXi 的 CR SDK 是基于网络通信协议 TCP/IP 实现的，提供了用于操作机械臂的接口，支持 python，C/C++，C#三种语言。

本文是基于 Python3.7 开发，暂不支持 Python3.7 以外的版本，提供操作机械臂的接口。

二、数据结构

方便接口函数的理解，特将一些常用的数据结构做详细的解释：接口中的长度单位统一为毫米（mm），角度单位统一为度（°）。

2.1 机械臂句柄

```
crRobotHandle = NewType("crRobotHandle",c_int)
```

2.2 接口调用返回值

```
class CRresult(c_int):  
    sucess = 0          #成功；分别有以下 3 种：连接、获取参数、写入参数  
    success = 0         #成功；分别有以下 3 种：连接、获取参数、写入参数  
    error = 1           #一般错误  
    thread_running = 2  #通讯连接重复建立  
    operate_timeout = 3 #操作超时，无返回值；注：处理超时导致无结果输出  
    result_invalid = 4  #返回结果无效；原因：rpc 获取结果为空  
    out_of_range = 5    #输入数值越界；需输入在范围内的值  
    mutex_invalid = 6   #内部错误  
    para_error = 7      #参数无效；需检查输入参数是否为空  
    no_result = 8        #无对应数值  
    no_assignTCPindex = 9 #未找到指定索引号 TCP 偏移量  
    no_handle = 10       #未创建相关句柄  
    handle_repeat = 11   #重复建立相同的句柄  
    repeat_name = 12     #重命名  
    delete_invalid = 13 #删除唯一的 TCP 或者负载  
    set_bit_reg_invalid = 14 #同时设置位寄存器输入与输出  
    repeat_id = 15       #id 重复  
    file_encryption = 16 #文件加密  
    robotmode_error = 17 #当前状态不能执行该操作  
    move_error = 18      #移动阻塞运动时，返回的移动状态错误
```

```

forcectrl_wrongpara = 19
#力控 SDK 参数设置错误：包括参数长度错误、payload 为负数
forcectrl_invaildcmd = 20
#力控 SDK 无效的指令：包括 forceEna 等于 1,0 以外的值；forcebasetype 除 0 以外的值；forcectrltype 除 1,0 以外的值；auto/manual 1,0 以外的值
forcectrl_parablocked = 21    #力控 SDK freedrive 模式下部分参数失效被屏蔽
forcectrl_cmdblocked = 22
#力控 SDK 机器人运动时或已在力控模式下时禁止再打开力控
forcectrl_configblocked = 23
#力控 SDK 机器人运动时或已在力控模式下时禁止下发配置
forcectrl_notenable = 24      #力控 SDK 机器人使能之前不能打开传感器
forcectrl_wrongsensor = 25    #力控 SDK 传感器型号选择错误
outputrule_error = 26         #未设置系统输出，禁止启用自复位
analoginputputtype_error = 27
#模拟量输入类型错误：注：工具端模拟量输入类型只有电压

```

2.3 坐标系选择

```

class CoordinateType(c_int):
    baseCoordinate = 0          #基坐标系
    jointCoordinate = 1         #关节坐标系
    PointCoordinate = 2         #点坐标系
    LineCoordinate = 3          #线坐标系
    PlaneCoordinate = 4         #面坐标系
    ToolBaseCoordinate = 5      #工具坐标系
    visionCoordinate = 6        #视角坐标系

```

2.4 点转换类型

```

class PointTransType(c_int):
    pointTransStop = 0          #过渡点停止
    pointTransConstantSpeed = 1 #匀速过渡
    pointTransSmooth = 2        #柔顺过渡（不匀速，不停止）

```

2.5 位姿变换

```

class PoseTranType(c_int):          #预留参数
    poseTranMoveToTargetPose = 0
    poseTranKeepStartPose = 1

```


2.6 运动触发模式

```
class MotiontriggerMode(c_int):
    MovetriggerbySafetySignal = 0          #安全同步信号触发
    MovetriggerbyOnlyRpc = 1              #指令直接触发
    MovetriggerbyMoveCache = 2            #移动指令会进入缓存区
```

2.7 TCP 偏移量

```
class TCPMsg(Structure):
    _fields_ = [('tcpName', c_char*20),          #tcp 名称
                ('tcpOffset', c_double*ROB_AXIS_NUM)]
    #tcp 偏移量: x,y,z (单位: mm); Rx,Ry,Rz (单位: °)
```

2.8 运动参数

```
class PointControlPara(Structure):
    _fields_ = [('pose', c_double*ROB_AXIS_NUM),
                #坐标点向量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
                ('jointpos', c_double*ROB_AXIS_NUM),
                #关节角度: 关节 1~6 (单位: °)
                ('tcpOffset', c_double*ROB_AXIS_NUM),
                #tcp 偏移量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
                ('tcpID', c_int),
                #tcp 的 ID 号,当 tcpID 为非负数时, 使用 tcpID 对应的 TCP 偏移量作为输入; 当 tcpID 为
                #负数时, 使用 tcpoffset 数据作为 TCP 偏移量输入;
                ('coordinateType', CoordinateType),          #坐标系类型
                ('coordinatePose', c_double*ROB_AXIS_NUM),
                #参考坐标系位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
                #speed 可以是 tcp 速度或者是关节速度,当 coordinateType 设置为 jointCoordinate 时, 该速
                #度为关节速度, 其它情况是 tcp 速度
                #作为 tcp 速度时, 单位为: mm/s; 作为关节速度时, 单位为: °/s
                ('speed', c_double*ROB_AXIS_NUM),          #速度
                #acc 可以是 tcp 加速度或者是关节加速度,当 coordinateType 设置为 jointCoordinate 时,
                #该速度为关节加速度, 其它情况是 tcp 加速度
                #作为 tcp 加速度时, 单位为: mm/s²; 作为关节加速度时, 单位为: °/s²
                ('acc', c_double*ROB_AXIS_NUM),          #加速度
                ('jerk', c_double*ROB_AXIS_NUM),          #加加速度, 预留参数
                ('pointTransType', PointTransType),          #位置过渡方式
                ('pointTransRadius', c_double),          #过渡半径, 预留参数
                ('poseTranType', PoseTranType),          #姿态变换方式, 预留参数
```

(`'motiontriggerMode', MotiontriggerMode`) #运动轨迹触发方式

2.9 简化版运动参数

```
class PointControlParaSimple(Structure):
    _fields_ = [('pose', c_double*ROB_AXIS_NUM),
        #坐标点向量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
        ('jointpos', c_double*ROB_AXIS_NUM),
        #关节角度: 关节 1~6 (单位: °)
        ('tcpOffset', c_double*ROB_AXIS_NUM),
        #tcp 偏移量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
        ('coordinateType', CoordinateType), #坐标系类型
        ('coordinatePose', c_double*ROB_AXIS_NUM),
        #参考坐标系位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
        #speed 可以是 tcp 速度或者是关节速度,当 coordinateType 设置为 jointCoordinate 时, 该
        速度为关节速度, 其它情况是 tcp 速度
        #作为 tcp 速度时, 单位为: mm/s; 作为关节速度时, 单位为: ° /s
        ('speed', c_double*ROB_AXIS_NUM), #速度
        #acc 可以是 tcp 加速度或者是关节加速度,当 coordinateType 设置为 jointCoordinate 时,
        该速度为关节加速度, 其它情况是 tcp 加速度
        #作为 tcp 加速度时, 单位为: mm/s²; 作为关节加速度时, 单位为: ° /s²
        ('acc', c_double*ROB_AXIS_NUM), #加速度
        ('pointTransType', PointTransType), #位置过渡方式
        ('motiontriggerMode', MotiontriggerMode) #运动轨迹触发方式
```

2.10 圆弧运动参数

```
class PointControlParaList(Structure):
    _fields_ = [('pointcontrolpara', PointControlPara*MOVECPOINT_NUM), #过渡点和终点
        ('fixedrot', c_int),
        #方向模式 0-无约束 1-固定
        ('centralangle', c_double)] #圆弧角,
        #单位: °
```

2.11 运动模式

```
class MoveType(c_int):
    DecStop = 0 #当前运动减速停止
    ImdStop = 1 #当前运动立即停止
    Run = 2 #运动
```

2.12 关节状态

```
class JointModes(c_int):
    JOINT_MODE_POWER_OFF = 1          #断电
    JOINT_MODE_BOOTING = 2            #机械臂上电至空闲状态下
    JOINT_MODE_IDLE = 3
    #上电完成,空闲状态 (ethercat——>op)
    JOINT_MODE_BACK_DRIVE = 4         #反向驱动状态
    JOINT_MODE_RELEASE_BRAKE = 5      #机械臂使能, 松抱闸过程中
    JOINT_MODE_CSP_STOP = 7           #关节使能静止(位置模式)
    JOINT_MODE_CSP_MOVING = 8         #关节使能运动(位置模式)
    JOINT_MODE_CST_STOP = 9           #关节使能静止(转矩模式)
    JOINT_MODE_CST_MOVING = 10        #关节使能运动(转矩模式)
    JOINT_MODE_FAULT = 11             #错误状态
    JOINT_MODE_READY_FOR_POWER_OFF = 12 #准备断电
```

2.13 机械臂状态

```
class RobotModes(c_int):
    Closed = 0          #关闭
    Disconnect = 1      #断开连接状态
    ConfirmSafty = 2    #安全确认状态
    Booting = 3         #控制柜正在启动
    ControlerIdle = 4   #控制柜启动完成, 空闲状态
    ControlerUpdataFirmWare = 5 #固件更新
    JointPowerOff = 6   #控制柜启动完成, 本体未上电
    JointPowerOn = 7    #机器人上电过程
    JointIdle = 8       #机器人上电完成, 空闲
    BackDrive = 9       #反向驱动
    ReleaseBrake = 10   #机器人使能、松抱闸中
    Enable = 11         #机器人使能完成, 并已完全松开抱闸
    CloseBrake = 12     #机器人断使能、关抱闸中
    Jog = 100           #点动
    Teach = 101         #拖动示教
    ForceControlTest = 102 #力控测试
    ProgramStop = 103   #使能完成, 程序停止
    ProgramPause = 104  #使能完成, 程序暂停
    ProgramStopping = 105 #使能完成, 程序和运动正在停止
    ProgramPauseing = 106 #使能完成, 程序和运动正在暂停
    ProgramRun_MotionStop = 107 #程序运行, 运动停止
    ProgramRun_MotionReducing = 108 #程序运行, 运动减速中
```

```
ProgramRun_MotionMoving = 109    #程序运行
ProgramRun_MotionCanBlend = 110  #程序运行，机器人进入到可交融区域
Imdstop = 111                    #紧急停止
ProtectiveStop = 112             #防护停止
```

2.14 安全模式

```
class SafetyModes(c_int):
    SAFETY_MODE_UNDEFINED_SAFETY_MODE = 1    #未定义
    SAFETY_MODE_VALIDATE_JOINT_ID = 2        #验证关节的 ID
    SAFETY_MODE_FAULT = 3
    #安全错误，其他错误出现一段时间后，最终保持的错误状态
    SAFETY_MODE_VIOLATION = 4                #在安全平面处越界
    SAFETY_MODE_ROBOT_EMERGENCY_STOP = 5     #急停按钮按下
    SAFETY_MODE_SYSTEM_EMERGENCY_STOP = 6
    #可配置 IO 输入任意两路设置为紧急停机，断开后出现的错误状态
    SAFETY_MODE_SAFEGUARD_STOP = 7          #安全 IO 断开
    SAFETY_MODE_RECOVERY = 8
    #在违反安全限制后，机器人停止，重启后，进入恢复模式
    SAFETY_MODE_PROTECTIVE_STOP = 9
    #保护性停止，运动过程中，外部强力推动机械手
    SAFETY_MODE_REDUCED = 10                #限制模式
    SAFETY_MODE_NORMAL = 11                #正常运行模式
```

2.15 程序状态

```
class ProgramState(c_int):
    PROGRAM_STATE_STOP = 0                #程序停止
    PROGRAM_STATE_PAUSE = 1               #程序暂停
    PROGRAM_STATE_RUN = 2                  #程序运行
```

2.16 弹窗信息

```
class PopUpMsg(Structure):
    _fields_ = [('popupType',c_int),        #弹窗类型，0-输出弹窗，1-输入弹窗
                ('var_data',c_char*1024),  #弹窗显示的字符串，字符串类型
                ('var_title',c_char*128),   #弹窗标题，字符串类型
                ('iswarning',c_bool),       #是否是警告信息，bool 类型
                ('iserror',c_bool),        #是否是报警信息，bool 类型
                ('isblocking',c_bool),     #是否暂停程序运行，bool 类型
                ('inVarType',VariableType), #输入弹窗时，变量类型
```

('inVarMinValue',c_double),	#输入弹窗时，变量数值范围最小值
('inVarMaxValue',c_double)]	#输入弹窗时，变量数值范围最大值

2.17 版本信息

```
class Version(Structure):
    _fields_ = [('deviceType',c_ubyte*20),    #机器人类型
                ('versionNo',c_ubyte*20),    #版本信息
                ('bugfix',c_int),            #bug 修正
                ('buildDate',c_longlong),    #编译日期
                ('hardwareID',c_ubyte*20)    #硬件 id
                ('bootVersionNo',c_ubyte*20)] #boot 版本号
```

2.18 Lua 变量

```
class VariableType(c_int):
    LUA_TNIL = 0          #空
    LUA_TBOOLEAN = 1     #布尔
    LUA_TLIGHTUSERDATA = 2 #预留
    LUA_TNUMBER = 3      #数字
    LUA_TSTRING = 4      #字符串
    LUA_TTABLE = 5       #表
    LUA_TFUNCTION = 6    #函数
    LUA_TUSERDATA = 7    #自定义类型
    LUA_TTHREAD = 8      #线程
    LUA_NUMTAGS = 9      #预留
```

2.19 变量元素

```
class VarItemData(Structure):
    _fields_ = [('varItemType',VariableType), #数据类型
                ('boolValue',c_bool),        #bool 类型数据值
                ('numberValue',c_double),    #number 类型数据值
                ('stringValue',c_char*(CR6_INSTALL_VAR_MAX_LENGTH))]
    #char 类型字符串数据值
```

2.20 变量信息

```
class VariableMsg(Structure):
    _fields_ = [('variableName',c_char*CR6_INSTALL_VAR_NAME_LENGTH), #变量名
                ('variableType',VariableType),                       #数据类型
```

```
(‘variableID’,c_int), #变量 id
(‘boolValue’,c_bool), #bool 类型数据值
(‘numberValue’,c_double), #number 类型数据值
(‘stringValue’,c_char*CR6_INSTALL_VAR_NAME_LENGTH), #char 类型字符串数据值
(‘tableValueCount’,c_int), #table 类型元素个数
(‘tableValue’,VarItemData*Script_MAX_TableElement) #table 类型元素值
(‘tableValueStr’,c_char*CR6_INSTALL_VAE_TABLE_MAX_LENGTH)] #table 值
```

2.21 消息来源

```
class CommMessageSource(c_int):
    Base = 0 #基座关节（关节 1）
    Shoulder = 1 #肩部关节（关节 2）
    Elbow = 2 #肘部关节（关节 3）
    Wrist1 = 3 #腕关节 1（关节 4）
    Wrist2 = 4 #腕关节 2（关节 5）
    Wrist3 = 5 #腕关节 3（关节 6）
    Tool = 6 #工具模块
    Controller = 100 #主控制柜
    SafetyProcessorB = 101 #协处理器
    TeachPendant = 200 #示教器
```

2.22 警告等级

```
class WarningLevel(c_int):
    MESSAGE_WARNING_LEVEL_INFO = 1 #正常消息
    MESSAGE_WARNING_LEVEL_WARNING = 2 #警告不停机
    MESSAGE_WARNING_LEVEL_VIOLATION = 3 #违反限制停机
    MESSAGE_WARNING_LEVEL_FAULT = 4 #系统故障，紧急停止
    MESSAGE_WARNING_LEVEL_USER = 5 #用户打印输出
```

2.23 世界时间

```
class WorldTime(Structure):
    _fields_=[(‘year’,c_uint), #年
(‘month’, c_uint), #月
(‘day’, c_uint), #日
(‘week’, c_uint), #周
(‘hour’, c_uint), #时
(‘minute’, c_uint), #分
(‘second’, c_uint), #秒]
```

```
(‘milliSecond’, c_uint) ] #毫秒
```

2.24 日志信息

```
class RobotLogMsg(Structure):
    _fields_ = [(‘robotMessageCode’, c_uint), #消息代码
                (‘robotMessageType’, c_uint), #消息类型，预留
                (‘source’, CommMessageSource), #消息来源
                (‘textMessage’, c_char * 100), #消息信息
                (‘worldTime’, WorldTime), #时间戳
                (‘warningLevel’, WarningLevel), #警告等级
                (‘robotMode’, RobotModes), #机器人状态
                (‘timeStamp_us’, c_ulonglong), #精确时间戳
                (‘messageIndex’, c_ulonglong)] #消息序号
```

2.25 版本信息

```
class SysVersion(Structure):
    _fields_ = [(‘sjqVersion’, Version), #示教器版本
                (‘kzqVersion’, Version), #控制柜版本
                (‘jpmVersion’, Version), #本体放电板版本
                (‘saftyCtrlVersion’, Version), #安全板版本
                (‘jointVersion’, Version * ROB_AXIS_NUM), #关节版本
                (‘etfVersion’, Version)] #工具法兰版本
```

2.26 负载

```
class PayLoad(Structure):
    _fields_ = [(‘payloadName’, c_char*20), #负载名称
                (‘toolPayload’, c_double), #负载重量：kg
                (‘centerOfGravity’, c_double*3)] #负载中心（CX，CY，CZ）（单位：mm）
```

2.27 校验码检查

```
class ParityCheck(c_int):
    no = 0 #无校验
    odd = 1 #奇校验
    even = 2 #偶校验
```

2.28 串口通信

```
class SerialCommuType(c_int):
    FreeComm = 0          #自由格式
    Modbus_RTU = 1        #modbus RTU 格式
    Modbus_ASCII = 2      #modbus ASCII 格式
```

2.29 波特率

```
class BaudRate(c_int):
    BaudRate300 = 0      #波特率为 300Baud
    BaudRate600 = 1      #波特率为 600Baud
    BaudRate1200 = 2     #波特率为 1200Baud
    BaudRate2400 = 3     #波特率为 2400Baud
    BaudRate4800 = 4     #波特率为 4800Baud
    BaudRate9600 = 5     #波特率为 9600Baud
    BaudRate19200 = 6    #波特率为 19200Baud
    BaudRate28800 = 7    #波特率为 28800Baud
    BaudRate38400 = 8    #波特率为 38400Baud
    BaudRate57600 = 9    #波特率为 57600Baud
    BaudRate115200 = 10   #波特率为 115200Baud
    BaudRate192000 = 11   #波特率为 192000Baud
    BaudRate256000 = 12   #波特率为 256000Baud
    BaudRate288000 = 13   #波特率为 288000Baud
    BaudRate384000 = 14   #波特率为 384000Baud
    BaudRate512000 = 15   ///波特率为 512000Baud
    BaudRate576000 = 16   #波特率为 576000Baud
    BaudRate768000 = 17   #波特率为 768000Baud
    BaudRate1000000 = 18  #波特率为 1000000Baud
    BaudRate1200000 = 19  #波特率为 1200000Baud
    BaudRate1500000 = 20  #波特率为 1500000Baud
    BaudRate2400000 = 21  #波特率为 2400000Baud
    BaudRate3000000 = 22  #波特率为 3000000Baud
```

2.30 串口配置

```
class SerialCommSettings(Structure):
    _fields_ = [('baudRate', BaudRate),    #波特率
                ('parity', ParityCheck),   #奇偶校验
                ('dataBits', c_int),       #数据位
                ('stopBits', c_double),    #停止位
```


<code>('retryNumber', c_int),</code>	<code>#重试次数</code>
<code>('timeout', c_int),</code>	<code>#超时时间，单位：ms</code>
<code>('serialCommType', SerialCommuType),</code>	<code>#串口通信类型</code>
<code>('modbusSlaveNo', c_int)]</code>	<code>#modbus 从站站号</code>

2.31 IO 类型

```
class VarDigitalIOType(c_int):
    Invaile = 0          #无效
    Digital = 1          #标准数字量
    ConfigurableDigital = 2 #可配置数字量
    ToolDigital = 3      #工具
    Analog = 4           #模拟量
    ToolAnalog = 5       #工具模拟量
```

2.32 安装角度

```
class InstallAngle(Structure):
    _fields_ = [('baseAngle', c_double), #基座旋转角度，单位：°
                ('tiltAngle', c_double)] #基座倾斜角，单位：°
```

2.33 系统功能输入

```
class InputAction(c_int):
    InputAction_NONE = 0          #无动作
    InputAction_RobotON = 1       #本体上电使能
    InputAction_MovetoHome = 2    #回初始点
    InputAction_RunProgram = 3    #运行程序
    InputAction_SuspendedProgram = 4 #暂停程序
    InputAction_StopProgram = 5   #停止程序
    InputAction_FreeDrive = 6     #自由拖动
    InputAction_DownEnable = 7    #下使能
    InputAction_ClearFault = 8    #清除故障
```

2.34 系统功能输出

```
class OutputAction(c_int):
    OutputAction_NONE = 0          #无动作
    OutputAction_RobotON_HIGH = 1  #机器人使能时为高
    OutputAction_RobotON_LOW = 2   #机器人使能时为低
    OutputAction_RobotPowerOn_HIGH = 3 #已上电时为高
```

```

OutputAction_RobotPowerOn_LOW = 4      #已上电时为低
OutputAction_PowerOn_HIGH = 5          #已开机时为高
OutputAction_PowerOn_LOW = 6           #已开机时为低
OutputAction_WarnErr_HIGH = 7          #有系统错误时为高
OutputAction_WarnErr_LOW = 8           #有系统错误时为低
OutputAction_ProgramStatus_Stop_HIGH = 10 #程序停止时为高
OutputAction_ProgramStatus_Stop_LOW = 11 #程序停止时为低
OutputAction_Fault_Stop_HIGH = 12      #异常停止时为高
OutputAction_Fault_Stop_LOW = 13       #异常停止时为低
OutputAction_ProgramStatus_Pause_HIGH = 14 #程序暂停输出为高
OutputAction_ProgramStatus_Pause_LOW = 15 #程序暂停输出为低
OutputAction_MotionStatus_Moving_HIGH = 20 #运动时为高
OutputAction_MotionStatus_Moving_LOW = 21 #运动时为低
OutputAction_MovetoHome_HIGH = 22      #回初始位到位信号，到位时为高

```

2.35 IO 选项卡输出

```

class OutputOptions(c_int):          #IO 选项卡输出操作管理
    OutputOptions_Enable = 0         #启用
    OutputOptions_Manual = 1         #仅手动
    OutputOptions_Disable = 2        #禁用

```

2.36 输出信号状态（自动复位）

```

class OutputRule(c_int):
    OutputRule_Disable = 0           #禁用
    OutputRule_Enable = 1            #启用

```

2.37 输入信号

```

class IOInputConfiguration(Structure):
    _fields_ = [('inputFilteringTime',c_int),    #输入信号滤波时间，单位:ms
                ('inputName',c_char * NAMELENGTH), #输入信号名称
                ('inputActions',InputAction)]    #输入信号操作

```

2.38 输出信号

```

class IOOutputConfiguration(Structure):
    _fields_ = [('outputName',c_char * NAMELENGTH), #输出信号名称
                ('outputActions',OutputAction),     #输出信号条件
                ('outputOptions',OutputOptions),    #输出信号操作控制选项

```

(‘outputRule’, OutputRule)]

#输出信号自复位控制选项

2.39 工具 IO 类型

```
class ToolIoType(c_int):
    IO_OUTPUT = 0          #输出
    IO_INPUT = 1           #输入
```

2.40 模拟量类型

```
class AnalogType(c_int):
    Current = 0            #电流
    Voltage = 1            #电压
```

2.41 模拟量输入

```
class AnalogInputConfiguration(Structure):
    _fields_ = [('analogInputName', c_char * NAMELENGTH), #模拟量输入通道名称
                ('analogInputType', AnalogType)]          #模拟量输入类型
```

2.42 模拟量输出

```
class AnalogOutputConfiguration(Structure):
    _fields_ = [('analogOutputName', c_char * NAMELENGTH), #模拟量输出通道名称
                ('analogOutputType', AnalogType)]          #模拟量输出类型
```

2.43 工具电压

```
class ToolPower(c_int):
    Power_invald = 0       #无效值
    Power_on = 1           #24V
    Power_off = 2          #0V
```

2.44 编码器配置

```
class EncoderConfiguration(Structure):
    _fields_ = [('eqepType', c_int),          #类型
                ('eqepCountMode', c_int),     #模式
                ('strobeInputMode', c_int),    #探针
                ('eqepEnabled', c_int),        #控制
```

2.45 BOOL 寄存器

```
class BitRegisterConfiguration(Structure):
    _fields_ = [('GeneralPurposeBOOLeanRegisterName', c_char * NAMELENGTH),
                #BOOL 型寄存器名称
                ('GeneralPurposeBOOLeanRegisterInputActions', InputAction),
                #BOOL 型寄存器系统功能输入
                ('GeneralPurposeBOOLeanRegisterOutputActions', OutputAction),
                #BOOL 型寄存器系统功能输出
                ('GeneralPurposeBOOLeanRegisterOutputRule', OutputRule)]
                #BOOL 型寄存器自复位输出
```

2.46 寄存器

```
class RegisterConfiguration(Structure):
    _fields_ = [('GeneralPurposeRegisterName', c_char * NAMELENGTH)]
    #寄存器名称
```

2.47 工具端信息

```
class PoseMessage(Structure):
    _fields_ = [('toolPosition', c_double * ROB_AXIS_NUM),
                #位姿：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
                ('toolAxisAngle', c_double * ROB_AXIS_NUM)]    #关节角度：关节 1~6（单位：°）
```

2.48 点坐标系

```
class PointCSNode(Structure):
    _fields_ = [('isValid', c_int),          #是否有效
                ('id', c_int),              #id 号
                ('name', c_char * NAMELENGTH), #点坐标系名字
                ('point', PoseMessage)]      #点坐标系数据
```

2.49 线坐标系

```
class LineCSNode(Structure):
    _fields_ = [('isValid', c_int),          #是否有效
                ('id', c_int),              #id 号
                ('name', c_char * NAMELENGTH), #线坐标系名字
                ('firstPoint', PointCSNode), #第一个点的数据相关信息
                ('secondPoint', PointCSNode), #第二个点的数据相关信息
                ('coordinatePose', c_double * ROB_AXIS_NUM),
```

```
#线坐标系位姿数据: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
('coordinateJointPos', c_double * ROB_AXIS_NUM)]
#线坐标系关节数据: 关节 1~6 (单位: °)
```

2.50 面坐标系

```
class PlaneCSNode(Structure):
    _fields_ = [('isValid', c_int),          #是否有效
                ('id', c_int),              #id 号
                ('name', c_char * NAMELENTH), #面坐标系名字
                ('firstPoint', PointCSNode), #第一个点的数据相关信息
                ('secondPoint', PointCSNode), #第二个点的数据相关信息
                ('thirdPoint', PointCSNode),  #第三个点的数据相关信息
                ('coordinatePose', c_double * ROB_AXIS_NUM),
    #面坐标系位姿数据: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
    ('coordinateJointPos', c_double * ROB_AXIS_NUM)]
#面坐标系关节数据: 关节 1~6 (单位: °)
```

2.51 机器人限值等级

```
class SafetyLimitsValuesType(c_int):
    Userdefined = 0          #自定义
    LimitLevel_1 = 1         #等级 1 (限制最多)
    LimitLevel_2 = 2         #等级 2 (限制偏多)
    LimitLevel_3 = 3         #等级 3 (限制偏少)
    LimitLevel_4 = 4         #等级 4 (限制最少)
```

2.52 碰撞后处理模式

```
class SafetyCollisionHandleMode(c_int):
    Collision_EnterReboundMode = 0    #进入反弹模式
    Collision_ProgramPause = 1        #暂停
    Collision_ProgramStop = 2         #停止
```

2.53 关节限值模式

```
class LimitsBoundaryPlaneMode(c_int):
    Disabled = 0          #禁用
    Normal = 1            #正常
    Reduced = 2           #缩减
    Both = 3              #二者都是
    Trigger = 4           #触发器缩减模式
```

2.54 安全限值模式

```
class SafetyLimitsMode(c_int):
    invaild = 0          #禁用
    mode_Normal = 1      #正常
    mode_Reduced = 2     #缩减
```

2.55 安全限值

```
class SafetyLimitsValues(Structure):
    _fields_ = [('maxTcpSpeed', c_double),      #tcp 最高速度, 单位: mm/s
                ('maxForce', c_double),         #tcp 最大力, 单位: N
                ('maxElbowSpeed', c_double),     #肘部最高速度, 单位: mm/s
                ('maxElbowForce', c_double),     #肘部最大力, 单位: N
                ('maxStoppingDistance', c_double), #最大停止距离, 单位: mm
                ('maxStoppingTime', c_double),   #最大停止时间, 单位: ms
                ('maxPower', c_double),          #最高功率, 单位: W
                ('maxMomentum', c_double),       #最大动量, 单位: kg m/s
                ('maxTcpAcc', c_double),         #tcp 最高加速度, 预留
                ('maxTcpJerk', c_double)]        #tcp 最高加加速度, 预留
```

2.56 关节限值

```
class SafetyLimitsJointAngle(Structure):
    _fields_ = [('maxJointSpeed', c_double),     #最大角速度, 单位: °/s
                ('minJointPosition', c_double),  #最小位置, 单位: °
                ('maxJointPosition', c_double)]  #最大位置, 单位: °
```

2.57 安全平面

```
class SafetyLimitsBoundaryPlane(Structure):
    _fields_ = [('name', c_char * NAMELENTH),    #安全平面名字
                ('id', c_int),                  #id 号
                ('displacement', c_double),      #z 方向平移, 单位: mm
                ('mode', LimitsBoundaryPlaneMode), #限制模式
                ('planeNormal', c_double),       #预留值 设为 0
                ('distanceToOrigin', c_double),  #预留值 设为 0
                ('elbowRestricted', c_int),      #是否限制第三关节
                ('sourceGeomFeatureType', CoordinateType), #关联的坐标系类型
                ('sourceGeomFeatureId', c_int)]   //关联的坐标系 ID
```

2.58 安全 IO 输入信号

```
class SafetyIOInput(c_int):
    input_invaill = 0          #未分配
    emergencyStop = 1         #紧急停机
    reducedMode = 2           #缩减模式
    safeguardStop = 3         #安全停止
    safeguardReset = 6        #安全重置
```

2.59 安全 IO 输出信号

```
class SafetyIOOutput(c_int):
    output_invaill = 0        #未分配
    systemEmergencyStop = 1   #紧急停止输出
    robotMoving = 2           #机器人运动
    robotNotStopping = 3      #机器人未停止
```

2.60 控制柜网络配置

```
class IPConfig(Structure):
    _fields_ = [('ipAddr', c_int * 4),      #IP 地址
                ('subnetMask', c_int * 4)]  #子网掩码
```

2.61 MODBUS 地址映射功能

```
class ModbusMasterFunctionNo(c_int):
    ReadBool = 0          #ModbusMaster 函数功能码
                           #读位寄存器
    WriteBool = 1         #写位寄存器
    ReadInt16 = 2         #读 16 位整数寄存器
    WriteInt16 = 3        #写 16 位整数寄存器
    ReadInt32 = 4         #读 32 位整数寄存器
    WriteInt32 = 5        #写 32 位整数寄存器
    ReadFloat = 6         #读浮点寄存器
    WriteFloat = 7        #写浮点寄存器
    ReadWriteBool = 8     #预留
    ReadWriteInt16 = 9    #预留
    ReadWriteInt32 = 10   #预留
    ReadWriteFloat = 11   #预留
```

2.62 MODBUS 地址映射

```
class ModbusAddrMap(Structure):
    _fields_ = [('functionNo', ModbusMasterFunctionNo), #modbusMaster 函数功能码
                ('slaveStartAddr', c_int),              #从站寄存器首地址
                ('masterOffsetNo', c_int),              #主站寄存器个数
                ('masterStartAddr', c_int),             #主站寄存器索引
                ('masterRegType', CommuVarType)]         #寄存器类型
```

2.63 MODBUS 连接状态

```
class ModbusMasterOperate(c_int):
    Start = 0 #开始连接
    Stop = 1  #断开连接
```

2.64 MODBUS 参数配置

```
class ModbusMasterConfig(Structure):
    _fields_ = [('id', c_int), #从站 id 号
                ('slaveNo', c_int), #从站站号
                ('slaveIP', c_int * 4), #modbus-tcp 时，从站 ip
                ('scanTime', c_int), #扫描周期
                ('modbusMasterOperate', ModbusMasterOperate), #modbus 连接状态
                ('autoConnect', c_int)] #自动连接
```

2.65 通讯变量选择

```
class CommuVarType(c_int):
    CommuVarType_Bool = 0 #bool 类型
    CommuVarType_Int16 = 1 #16bit 整数类型
    CommuVarType_Int32 = 2 #32bit 整数类型
    CommuVarType_Float = 3 #32bit 浮点类型
```

2.66 脚本程序状态

```
class Lua_ScriptStatus(c_int):
    lua_Script_NoneOP = 0 #脚本为空
    lua_Script_stop = 1 #脚本程序停止
    lua_Script_pause = 2 #脚本程序暂停
    lua_Script_run = 3 #脚本程序运行
    lua_Script_load = 4 #脚本程序正在加载
```


2.67 串口类型

```
class SerialType(c_int):
    rs485_tool_CommSettings = 0          #工具 485
    rs485_1_CommSettings = 1             #485-1
    rs485_2_CommSettings = 2             #485-2
    rs232_CommSettings = 3               #rs232
    ethernet_CommSettings = 4            #以太网
```

2.68 轨迹点数据

```
class PathPoint(Structure):
    _fields_ = [('pose', c_double * ROB_AXIS_NUM),
        #坐标点向量: x,y,z (单位: mm); Rx,Ry,Rz (单位: °)
        ('jointpos', c_double * ROB_AXIS_NUM)]      #关节角度: 关节 1~6 (单位: °)
```

2.69 轨迹参数

```
class PathPara(Structure):
    _fields_ = [('index', c_int),          #轨迹索引 上传时: -1~49, 下载时: 0~49
        ('moveType', c_int),              #运动类型, 0-透传, 1-样条
        ('speed', c_double),              #速度 moveType 为 3 时有效
        ('acc', c_double),                #加速度 moveType 为 3 时有效
        ('blendRadius', c_double)]        #过渡半径 moveType 为 3 时有效
```

2.70 轨迹数据

```
class PathData(Structure):
    _fields_ = [('pathPoints', PathPoint * PATHMAXPOINTS),
        #轨迹数据
        ('pathPointsNum', c_int),          #轨迹数量
        ('moveTime', c_int)]              #运动时间 moveType 为 1 时有效 2~10000, 单位: ms
```

2.71 下载轨迹数据

```
class PathDownloadData(Structure):
    _fields_ = [('pathData', PathData),    #轨迹数据
        ('pathPara', PathPara)]           #轨迹参数
```

2.72 轨迹记录参数

```
class RecordPathPara(Structure):
```

```
_fields_ = [('sampleTime', c_int), #采样时间 recordControl 为 1 时有效，单位：ms
('recordControl', c_int)] #记录控制，0-停止，1-启动，2-暂停
```

2.73 轨迹记录状态

```
class PathRecordStatus(Structure):
    _fields_ = [('recordStatus', c_int), #0-记录完成，1-记录中，2-暂停记录
('waypointNumber', c_int)] #已记录点数
```

2.74 轨迹运行状态

```
class PathRunMsg(Structure):
    _fields_ = [('pathrunstatus', c_int), #0 或 10001-轨迹控制停止 1-轨迹运行中
('pointIndex', c_int)] #当前运行轨迹索引
```

2.75 控制器模式

```
class ControlModes(c_int):
    CONTROL_MODE_POSITION = 0 #控制器启动后的各种 RobotModes
    CONTROL_MODE_TEACH = 1
    #ROBOT_MODE_RUNNING 下，按下示教按钮或者执行 teach_mode()，在此模式下
    手动使机器人来回移动
    CONTROL_MODE_FORCE = 2 #力控命令内
    CONTROL_MODE_TORQUE = 3 #自由驱动模式
```

2.76 力控设置

```
class ForceSetting(Structure):
    _fields_ = [('forceBaseType', c_int), #力控类型： 0-基于 TCP 力传感器,1-基于动力学,2-基于底座力传感器
('forceCtrlType', c_int), #力控用法： 0-运动力控,1-力控拖动
('flexibleAxis', c_int*6)] #力控轴
```

2.77 力控传感器 tcp

```
class ForceTCPMsg(Structure):
    _fields_ = [('forceTcpName', c_char * 20), #力控传感器 tcp 的名称
('forceTcpId', c_int), #力控传感器 tcp 的 ID
('forceTcpOffset', c_double * ROB_AXIS_NUM)] #力控传感器 tcp 偏移量
```

2.78 力控传感器负载

```
class ForcePayLoad(Structure):
    _fields_ = [('forcePayloadName', c_char * 20),      #力控传感器负载名称
                ('forcePayloadId', c_int),             #力控传感器负载的 ID
                ('forceToolPayload', c_double),        #力控传感器负载重量: kg
                ('forceCenterOfGravity', c_double*3)]  #负载中心 (CX, CY, CZ)
```

2.79 力控传感器信息

```
class SensorMessage(Structure):
    _fields_ = [('venderNo', c_int),                  #力传感器厂家号: 0-CGX,1-蓝点,2-鑫精诚
                ('productNo', c_int),                #力传感器产品号
                ('sequenceNo', c_int)]               #力传感器通讯拓扑序号
```

2.80 通讯配置

```
class CommuConfig(Structure):
    _fields_ = [('autoConnect', c_int),               #自动连接 不处理-0 连接-1 关闭-2
                ('mannaulOperate', c_int),            #手动连接 不处理-0 连接-1 关闭-2
                ('paraBuf', c_int)]                  #预留参数
```

2.81 力控传感器工具设置

```
class ForceToolSetting(Structure):
    _fields_ = [('activeForceTCP_name', c_char * 20),  #激活的力控 tcp 名称
                ('activeForcePayload_name', c_char * 20), #激活的力控负载名称
                ('availableForceTCP', ForceTCPMsg * 20), #可以使用的力控 TCP
                ('availableForceTCPLen', c_int),        #可以使用的力控 TCP 数量
                ('availableForcePayLoad', ForcePayLoad*20), #可以使用的力控负载
                ('availableForcePayLoadLen', c_int)]    #可以使用的力控负载数量
```

2.82 力控配置

```
class ForceConfig(Structure):
    _fields_ = [('sensorMessage', SensorMessage),     #传感器信息
                ('forceCtrlTcpImpl', ForceToolSetting), #力控工具设置
                ('commuConfig', CommuConfig)]         #力控通讯配置
```

2.83 力控控制参数

```
class ForceCtlPara(Structure):
    _fields_ = [('taskFrame', c_double * 6),          #力控坐标系
                ('wrench', c_double * 6),           #力给定
                ('limits', c_double * 6),           #力控保护限制（范围：大于0）
                ('limitsLen', c_int),                #力控保护限制长度 0 或 6
                ('mass', c_double * 6),             #力控虚拟质量（范围：0.5~10）
                ('massLen', c_int),                  #力控虚拟质量长度 0、1 或 6
                ('damping', c_double * 6),          #力控虚拟阻尼（范围：0.1~10）
                ('dampingLen', c_int),               #力控虚拟阻尼长度 0、1 或 6
                ('stiffness', c_double * 6),         #力控虚拟刚度（范围：0~10）
                ('stiffnessLen', c_int)]             #力控虚拟刚度长度 0、1 或 6
```

2.84 力控数据

```
class ForceData(Structure):
    _fields_ = [('ftRaw', c_double * 6),             #力传感器原始值
                ('ftCtrl', c_double * 6),           #力传感器控制值
                ('controlMode', ControlModes)]       #控制器模式
```

2.85 机器人状态数据

```
class RobotStateData(Structure):
    _fields_ = [('tcpActualPose', c_double * ROB_AXIS_NUM), #实际 tcp 位置
                ('jointActualPos', c_double * ROB_AXIS_NUM), #实际关节位置
                ('robotMode', RobotModes),                  #机械臂当前状态
                ('robotMoveStatus', c_bool),                 #机械臂运动状态
                ('robotSpeedPercent', c_uint),               #机械臂速度百分比
                ('configurableDigitalOutput', c_int * CR6_CO_NUM)] #控制器可配置数字输出
```

2.86 机械臂和扩展轴数据

```
class RobotExjData(Structure):
    _fields_ = [('actualExjointPos', c_double * EXTERNAL_MAX_NUM), #扩展轴实际位置
                ('actualJointPos', c_double * ROB_AXIS_NUM),       #实际关节角度
                ('actualTcpVector', c_double * ROB_AXIS_NUM)]      #实际 TCP 位姿
```

2.87 扩展轴关节移动参数

```
class MoveExjPara(Structure):
```

```

_fields_ = [('pose', c_double * ROB_EXJ_AXIS_NUM),
            #前六个代表关节姿态 (x,y,z (单位: mm) ;Rx,Ry,Rz (单位: °) ), 后六个代表扩展轴位置
            ('jointPos', c_double * ROB_EXJ_AXIS_NUM),
            #前六个代表关节 1~6 关节角度, 后六个代表扩展轴位置
            ('speed', c_double * ROB_EXJ_AXIS_NUM),
            #前六个代表机械臂速度, 当坐标系类型为关节坐标系时, 分别代表一到六关节的速度 ° /s, 其他时候代表 tcp 速度 mm/s, 第七个代表扩展轴直线速度 mm/s, 第八个代表扩展轴旋转速度 ° /s
            ('acc', c_double * ROB_EXJ_AXIS_NUM),
            #前六个代表关节加速度, 后六个无效
            #acc 可以是 tcp 加速度或者是关节加速度,当 coordinateType 设置为 jointCoordinate 时, 该速度为关节加速度, 其它情况是 tcp 加速度
            #作为 tcp 加速度时, 单位为: mm/s²; 作为关节加速度时, 单位为: ° /s²
            ('tcpOffset', c_double * ROB_EXJ_AXIS_NUM),
            #tcp 偏移量:x,y,z (单位: mm) ;Rx,Ry,Rz (单位: °)
            ('tcpId', c_int),                                     #tcp 的 ID 号
            ('coordinateType', CoordinateType),                 #坐标系类型
            ('coordinatePose', c_double * ROB_AXIS_NUM),
            #参考坐标系位姿:x,y,z (单位: mm) ;Rx,Ry,Rz (单位: °)
            ('coordinateType', c_double),                         #过渡半径
            ('motiontriggerMode', MotiontriggerMode)]            #运动轨迹触发方式
    
```

2.88 扩展轴点动运动参数

```

class MoveJogExjPara(Structure):
    _fields_ = [('jointPos', c_double * EXTERNAL_MAX_NUM),      #扩展轴位置
                ('speed', c_double * EXTERNAL_MAX_NUM),        #扩展轴速度
                ('acc', c_double * EXTERNAL_MAX_NUM)]           #扩展轴加速度
    
```

2.89 扩展轴运动参数

```

class ExjMovePara(Structure):
    _fields_ = [('exjCount', c_int),                             #扩展轴数量
                ('index', c_int * EXTERNAL_MAX_NUM),            #扩展轴索引
                ('speed', c_double * EXTERNAL_MAX_NUM),         #扩展轴速度
                ('position', c_double * EXTERNAL_MAX_NUM),       #扩展轴位置
                ('acctime', c_double),                           #扩展轴加速时间
                ('motiontriggerMode', MotiontriggerMode)]        #运动轨迹触发方式
    
```

2.90 扩展轴类型

```
class ExternalJointType(c_int):
    line_externalJoint = 0          #直线
    rotation_externalJoint = 1      #旋转
```

2.91 扩展轴配置

```
class ExjConfig(Structure):
    _fields_ = [('name', c_char * NAMELENTH),      #扩展轴名称
                ('type', ExternalJointType),      #扩展轴类型
                ('ratio', c_double),              #传动比
                ('minlimit', c_double),            #最小值
                ('maxlimit', c_double),            #最大值
                ('zeroPosition', c_longlong),      #零位(pulse)
                ('exjToRef', c_double * ROB_AXIS_NUM), #扩展轴相对于参考系的关系
                ('masterExjName', c_char * NAMELENTH), #主轴名称
                ('masterSlaveRate', c_double)]      #主从比
```

2.92 常用宏定义

```
ROB_AXIS_NUM = 6
CR6_DI_NUM = 8
CR6_DO_NUM = 8
CR6_CI_NUM = 8
CR6_CO_NUM = 8
CR6_AI_NUM = 3
CR6_AO_NUM = 1
CR6_TOOL_DI_NUM = 2
CR6_TOOL_DO_NUM = 2
CR6_TOOL_AI_NUM = 2
CR6_COM_NUM = 4
MOVECPOINT_NUM = 2
ModbusMasterConfigNumber = 20
ModbusAddrMapNumber = 100
CR6_BOOL_REG_NUM = 1000
CR6_INT16_REG_NUM = 1000
CR6_INT32_REG_NUM = 1000
CR6_FLOAT_REG_NUM = 1000
ENCODER_TICK_NUM = 2
COR_MAX_NUM = 50
DH_PARA_MAX_NUM = 100
DY_PARA_MAX_NUM = 100
```

```
CR6_ISTALL_VAR_NUM = 100
CR6_ISTALL_VAR_NAME_LENGTH = 100
CR6_INSTALL_VAR_MAX_LENGTH = 100
CR6_INSTALL_VAE_TABLE_MAX_LENGTH 10000
Script_MAX_TableElement = 1000
Script_MAX_DataLen = 100
Global_Var_Max_Num = 100 #读全局变量最大个数
Global_Var_Max_Num = 1 #写读全局变量最大个数
COR_MAX_NUM = 50 #协程最大个数
TCP_MAX_NUM = 20
NAMELENTN = 32
BOOL = bool
TRUE = 1
FALSE = 0
PATHMAXPOINTS = 10000
EXTERNAL_MAX_NUM = 10
ROB_EXJ_AXIS_NUM = 12
ROB_EXJOG_AXIS_NUM = 16
```

三、接口函数

3.1 基础

3.1.1 基础接口

3.1.1.1 机器人创建连接

函数

```
cr_create_robot(robotHandle, ipAddr, port, passwd)
```

简介

创建机器人连接。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	POINTER(c_int)	输出	机械臂实例句柄
ipAddr	c_char_p	输入	控制柜 IP 地址
port	c_int	输入	控制柜端口号为 2323，虚拟为 2325
passwd	c_char_p	输入	登录密码

返回值

返回结果为长度为 2 的一维数组，第一个表示连接状态，第二个是分配的句柄值。

示例

```
robotHandle = 1
ipAddr="192.168.6.6"
port=2323
passwd="123"
re1=cgxiapi.cr_create_robot(robotHandle,ipAddr,port,passwd)    #机器人连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.1.2 机器人断开连接

函数

cr_destroy_robot(robotHandle)

简介

释放机器人，断开连接。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```


3.1.1.3 机器人关机

函数

`cr_shutdown(robotHandle)`

简介

机械臂以及控制柜关机。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 `CRresult`，若为 `success` 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
    robotHandle = re1[1].value
    result=cgxiapi.cr_shutdown(robotHandle) #机器人关机
    print("返回结果: ",result.value)
if result.value != 0: #判断机械臂关机是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.1.4 机械臂上电

函数

`cr_poweron(robotHandle)`

简介

机械臂上电。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_poweron(robotHandle) #机械臂上电
print("机械臂上电结果: ",result.value)
if result.value != 0: #判断机械臂上电是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.1.5 机械臂断电

函数

cr_poweroff(robotHandle)

简介

机械臂断电。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_poweroff(robotHandle) #机械臂断电
print("机械臂断电结果: ",result.value)
```

```

if result.value != 0:    #判断机械臂断电是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.1.6 机械臂使能

函数

cr_enable(robotHandle)

简介

机械臂使能。机械臂需要先上电，再使能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
    robotHandle = re1[1].value
    result=cgxiapi.cr_enable(robotHandle)    #机械臂使能
    print("机械臂使能结果: ",result.value)
if result.value != 0:    #判断机械臂使能是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.1.7 机械臂关使能

函数

cr_disable(robotHandle)

简介

机械臂关使能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_disable(robotHandle)  #机械臂关使能
print("机械臂关使能结果: ",result.value)
if result.value != 0:  #判断机械臂关使能是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.1.8 机械臂故障复位

函数

cgxiapi.cr_FaultReset(robotHandle)

简介

机械臂故障复位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1

```

```

re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_FaultReset(robotHandle) #机械臂故障复位
print("机械臂故障复位结果: ",result.value)
if result.value != 0: #判断机械臂故障复位是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.1.9 获取控制柜状态

函数

cr_get_controlMode(robotHandle)

简介

获取控制柜状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CResult 类型的值，第二个是控制柜状态，0-自动，1-手动。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_controlMode(robotHandle) #获取控制柜状态
print("获取控制柜状态结果: ",result[0].value,"控制柜状态",result[1].value)
if result[0].value != 0: #判断获取控制柜状态是否成功
    exit()

```

```
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.1.10 写机械臂速度百分比

函数

cr_set_robotSpeedPercent(robotHandle, speedPercent)

简介

写入机械臂速度百分比（1 至 100）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
speedPercent	c_uint	输入	速度百分比

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
speedPercent = 50
result=cgxiapi.cr_set_robotSpeedPercent(robotHandle, speedPercent) #写机械臂速度百分比
print("写入速度百分比结果: ",result.value)
if result.value != 0: #判断写机械臂速度百分比是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.1.11 读机械臂速度百分比

函数

cr_get_robotSpeedPercent(robotHandle)

简介

读机械臂速度百分比（1 至 100）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个表示接口调用结果，第二个值为读取到的速度百分比。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_robotSpeedPercent(robotHandle) #读机械臂速度百分比
print("读速度百分比结果: ",result[0].value,"速度百分比为",result[1].value)
if result[0].value != 0: #判断读机械臂速度百分比是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.1.12 获取 SDK 版本号

函数

```
cr_get_sdk_version()
```

简介

获取 SDK 版本号。

参数

无。

返回值

返回结果是长度为 2 的一维数组，第一个表示接口调用结果，第二个值为 SDK 版本号。

示例

```
result = cgxiapi.cr_get_sdk_version()    #读 SDK 版本号
print("读 SDK 版本号结果: ", result[0].value)
if result[0].value != 0:    #判断读 SDK 版本号是否成功
    exit()
print("SDK 版本号: ",result[1].value)
```

3.1.1.13 获取机械臂状态数据

函数

```
cr_get_robotStateData(robotHandle,robotStateData);
```

简介

获取机械臂状态数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
robotStateData	POINTER(RobotStateData)	输出	机械臂状态数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
robotStateData = basestruct.RobotStateData()    #获取机械臂状态数据
result = cgxiapi.cr_get_robotStateData(robotHandle, robotStateData)
print("获取机械臂状态数据结果: ",result.value)
if result.value != 0:    #判断获取机械臂状态数据是否成功
    exit()
print("tcpActualPose:", list(robotStateData.tcpActualPose))
print("jointActualPos:", list(robotStateData.jointActualPos))
print("robotMode:", robotStateData.robotMode.value)
print("robotMoveStatus:", robotStateData.robotMoveStatus)
print("robotSpeedPercent:", robotStateData.robotSpeedPercent)
print("configurableDigitalOutput:", list(robotStateData.configurableDigitalOutput))
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
```



```
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2 程序控制

3.1.2.1 下载脚本程序

函数

```
cr_downloadProgram(robotHandle, programfile)
```

简介

下载脚本程序。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
programfile	c_char_p	输入	需要下载的脚本程序

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
programfile = ("function mainFuncProgram()\n"
"--start robotConfig\n"
"TCP_1 = { 10,0,0,0,0,0 } \n"
"Payload_1= { 5,{0,0,0} } \n"
"set_tcp_payload(Payload_1[1], Payload_1[2])\n"
"--end robotConfig\n"
"function RobotProgram()\n"
"while (true)\n"
"do\n"
"--output: CDO_0 = true\n"
"set_standard_digital_out(0, true)\n"
"wait(1)  --sync\n"
"end\n"
"end\n")
```

```

"RobotProgram_result=task_create(RobotProgram)\n"
"function PauseFuncProgram()\n"
"while (true)\n"
"do\n"
"wait(100) --sync\n"
"end\n"
"end\n"
"PauseFuncProgram_result=task_create(PauseFuncProgram)\n"
"end\n"
"mainFuncProgram_result=task_create(mainFuncProgram)"
re2=cgxiapi.cr_downloadProgram(robotHandle, programfile) #下载脚本程序
print("加载程序结果: ",re2.value)
if re2.value != 0: #判断下载脚本程序程序是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.2.2 下载脚本程序（加密）

函数

```
cr_downloadProgram_encryption(robotHandle, programfile)
```

简介

下载脚本程序（加密）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
programfile	c_char_p	输入	需要下载的脚本程序

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
programfile = ("function mainFuncProgram()\n"

```

```

"--start robotConfig\n"
"TCP_1 = { 10,0,0,0,0,0 }\n"
"Payload_1= { 5,{0,0,0} }\n"
"set_tcp_payload(Payload_1[1], Payload_1[2])\n"
"--end robotConfig\n"
"function RobotProgram()\n"
"while (true)\n"
"do\n"
"--output: CDO_0 = true\n"
"set_standard_digital_out(0, true)\n"
"wait(1) --sync\n"
"end\n"
"end\n"
"RobotProgram_result=task_create(RobotProgram)\n"
"function PauseFuncProgram()\n"
"while (true)\n"
"do\n"
"wait(100) --sync\n"
"end\n"
"end\n"
"PauseFuncProgram_result=task_create(PauseFuncProgram)\n"
"end\n"
"mainFuncProgram_result=task_create(mainFuncProgram)")
re2=cgxiapi.cr_downloadProgram_encryption(robotHandle, programfile) #下载脚本程序（加密）
print("加载程序结果：",re2.value)
if re2.value != 0: #判断下载脚本程序（加密）是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.2.3 上传脚本程序

函数

cr_uploadProgram(robotHandle)

简介

上传脚本程序。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CResult 类型的值，第二个是脚本程序。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
re2=cgxiapi.cr_uploadProgram(robotHandle) #上传脚本程序
print("上传程序结果: ",re2[0].value)
if re2[0].value != 0: #判断上传脚本程序是否成功
    exit()
scriptId = re2[1].value
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.4 运行程序

函数

cr_play(robotHandle)

简介

运行程序。若当前无已加载的程序，请先通过 cr_downloadProgram 加载一个程序。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
```

```

robotHandle = re1[1].value
programfile = ("function mainFuncProgram()\n"
"--start robotConfig\n"
"TCP_1 = { 10,0,0,0,0,0 } \n"
"Payload_1= { 5,{0,0,0} } \n"
"set_tcp_payload(Payload_1[1], Payload_1[2])\n"
"--end robotConfig\n"
"function RobotProgram()\n"
"while (true)\n"
"do\n"
"--output: CDO_0 = true\n"
"set_standard_digital_out(0, true)\n"
"wait(1) --sync\n"
"end\n"
"end\n"
"RobotProgram_result=task_create(RobotProgram)\n"
"function PauseFuncProgram()\n"
"while (true)\n"
"do\n"
"wait(100) --sync\n"
"end\n"
"end\n"
"PauseFuncProgram_result=task_create(PauseFuncProgram)\n"
"end\n"
"mainFuncProgram_result=task_create(mainFuncProgram)")
re2=cgxiapi.cr_downloadProgram(robotHandle, programfile) #加载程序
print("加载程序结果: ",re2.value)
if re2.value != 0:    #判断加载程序是否成功
    exit()
re2=cgxiapi.cr_play(robotHandle) #运行程序
print("运行程序结果: ",re2.value)
if re2.value != 0:    #判断运行程序是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.2.5 停止程序运行

函数

```
cr_stop(robotHandle)
```

简介

停止程序运行。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
re2=cgxiapi.cr_stop(robotHandle) #停止程序运行
if re2.value != 0: #判断停止程序运行是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.6 暂停程序运行

函数

cr_pause(robotHandle)

简介

暂停程序运行。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
```

```
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
re2=cgxiapi.cr_pause(robotHandle) #暂停程序运行
if re2.value != 0: #判断暂停程序运行是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.7 读取脚本当前运行行号

函数

```
cr_script_current_line_get(robotHandle)
```

简介

读取脚本当前运行行号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果为长度为 3 的一维数组，第一个值为 CRresult 类型值，第二个值是脚本当前行号，第三个值是脚本行号数组长度。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_script_current_line_get(robotHandle) #读取脚本当前运行行号
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取脚本当前运行行号是否成功
    exit()
print("脚本当前行号: ",result[1][0])
print("脚本行号数组长度: ",result[2].value)
```

```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.8 读脚本运行状态

函数

```
cr_get_lua_scriptstatus(robotHandle)
```

简介

读脚本运行状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。同时返回读取结果。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
scriptstatus = basestruct.Lua_ScriptStatus
result=cgxiapi.cr_get_lua_scriptstatus(robotHandle)  #读脚本运行状态
print("返回结果：",result[0].value)
if result[0].value != 0:  #判断读脚本运行状态是否成功
    exit()
print("读取结果：",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.9 上传工程文件

函数

```
cr_uploadProject(robotHandle, FilePath, filename)
```


简介

上传 crp 文件和脚本程序。如果先使用 `cr_downloadProgram` 下载脚本程序后再上传工程文件，则只有脚本程序有效，crp 文件为空。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
FilePath	c_char_p	输入	文件保存路径
filename	c_char_p	输入	.crp 和脚本程序的文件名

返回值

数据类型为 `CRresult`，若为 `success` 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
FilePath = b"D:/v1.6c/teachpendant_x64/program/"
filename = b"test"
result=cgxiapi.cr_uploadProject(robotHandle, FilePath, filename) #工程文件上传
print("返回结果: ",result.value)
if result.value != 0: #判断工程文件上传是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.10 下载工程文件

函数

```
cr_downloadProject(robotHandle, crpFilepathname, crscriptFilepathname)
```

简介

下载 crp 文件和脚本程序，下载成功后可从示教器上看到目前处于工程暂停状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
crpFilepathname	c_char_p	输入	.crp 文件路径名
crscriptFilepathname	c_char_p	输入	脚本程序路径名

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
crpFilepathname = b"D:/v1.6c/teachpendant_x64/program/test.crp"
crscriptFilepathname = b"D:/v1.6c/teachpendant_x64/program/test.crscript"
result=cgxiapi.cr_downloadProject(robotHandle, crpFilepathname, crscriptFilepathname)
#工程文件下载
print("返回结果: ",result.value)
if result.value != 0: #判断工程文件下载是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.11 是否存在弹窗

函数

```
cr_script_popup_exist(robotHandle)
```

简介

是否存在弹窗。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。同时返回是否存在弹窗，0 为不存在，1 为存在。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_script_popup_exist(robotHandle) #是否存在弹窗
print("返回结果:",result[0].value,"是否存在弹窗:",result[1].value)
if result[0].value != 0: #判断是否存在弹窗是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.1.2.12 读取弹窗信息

函数

```
cr_script_popup_msg_get(robotHandle,popupMsg)
```

简介

读取弹窗信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
popupMsg	POINTER(VariableMsg)	输入	弹窗信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
```

```

    print("连接失败")
    exit()
robotHandle = re1[1].value
popupMsg = basestruct.PopUpMsg()
result = cgxiapi.cr_script_popup_msg_get(robotHandle,popupMsg) #读取弹窗信息
print("返回结果: ",result.value)
if result.value != 0: #判断读取弹窗信息是否成功
    exit()
print("读取结果:",popupMsg.popupType)
str_popupMsg = str(popupMsg.var_data , 'utf-8')
print("弹窗内容",str_popupMsg)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.2.13 设置输入弹窗信息

函数

```
cr_script_popup_msg_set(robotHandle, invarMsg)
```

简介

设置输入弹窗信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
invarMsg	POINTER(VariableMsg)	输入	弹窗信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
invarMsg = basestruct.VariableMsg()
invarMsg.variableName = b'var_1'
invarMsg.variableType = 1

```

```

invarMsg.variableID = 3
invarMsg.boolValue = True
result = cgxiapi.cr_script_popup_msg_set(robotHandle,invarMsg) #设置输入弹窗信息
print("返回结果:",result.value)
if result.value != 0: #判断设置输入弹窗信息是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.1.2.14 关闭弹窗

函数

```
cr_script_popup_close(robotHandle)
```

简介

关闭弹窗，只能关闭输入弹窗，不能关闭输出弹窗。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_script_popup_close(robotHandle) #关闭弹窗
print("返回结果:",result.value)
if result.value != 0: #判断关闭弹窗是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2 运动模块

3.2.1 运动接口

3.2.1.1 轴空间运动 moveJ

函数

`cr_moveJ(robotHandle, pointControlPara)`

简介

轴空间运动，推荐使用 `cr_move_joint` 接口。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数

返回值

数据类型为 `CRresult`，若为 `success` 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
pointControlPara.jointpos = (15,-3,90,0,90,10) #目标点示例数据
pointControlPara.pose = (0,0,0,0,0,0) #moveJ 中无影响
pointControlPara.tcpOffset = (0, 0, 0, 0, 0, 0)
pointControlPara.tcpID = -1
pointControlPara.speed = (speed, speed, speed, speed,speed,speed)
pointControlPara.acc = (acc, acc, acc, acc, acc, acc)
pointControlPara.coordinatePose = (0, 0, 0, 0, 0, 0)
pointControlPara.jerk = (0, 0, 0, 0, 0, 0)
pointControlPara.coordinateType = basestruct.CoordinateType.jointCoordinate
pointControlPara.pointTransType = basestruct.PointTransType.pointTransStop
pointControlPara.pointTransRadius = 0
```

```

pointControlPara.poseTranType = basestruct.PoseTranType.poseTranMoveToTargetPose
pointControlPara.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_moveJ(robotHandle, pointControlPara) #轴空间运动
print("返回结果:",result.value)
if result.value != 0: #判断轴空间运动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.1.2 轴空间运动 movejoint

函数

```
cr_move_joint(robotHandle, pointControlPara, isBlock)
```

简介

轴空间运动。移动相关速度和加速度范围见[附录 A 速度和加速度范围](#)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数
isBlock	c_int	输入	是否阻塞 1-阻塞 0-非阻塞

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
pointControlParaSimple.jointpos = (15,-3,90,0,90,10) #目标点示例数据
pointControlParaSimple.pose = (0,0,0,0,0,0) #moveJ 中无影响
pointControlParaSimple.tcpOffset = (0,0,0,0,0,0)

```

```

pointControlParaSimple.tcpID = -1
pointControlParaSimple.speed = (speed,speed,speed,speed,speed,speed)
pointControlParaSimple.acc = (acc,acc,acc,acc,acc,acc)
pointControlParaSimple.coordinatePose = (0,0,0,0,0,0)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = \
    basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle,pointControlParaSimple,
pointControlPara) #PointControlParaSimple 数据转换为 PointControlPara
if result.value != 0: #判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    exit()
isBlock = 0
result = cgxiapi.cr_move_joint(robotHandle, pointControlPara, isBlock) #轴空间运动
print("返回结果:",result.value)
if result.value != 0: #判断轴空间运动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.1.3 直线运动 moveL

函数

```
cr_moveL(robotHandle,pointControlPara)
```

简介

直线运动，推荐使用 `cr_move_line` 接口。移动相关速度和加速度范围见[附录 A 速度和加速度范围](#)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数

返回值

数据类型为 `CRresult`，若为 `success` 或 `0`，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)

```



```

if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlPara = basestruct.PointControlPara()
speed = 100
acc = 250
jointpose = (60, 60, -60, 60, 60, 60)    #起始点示例数据
pointControlPara.jointpos = jointpose
pointControlPara.tcpID = -1
pointControlPara.speed = (speed, speed, speed, speed, speed, speed)
pointControlPara.acc = (acc, acc, acc, acc, acc, acc)
pointControlPara.coordinatePose = (0, 0, 0, 0, 0, 0)
pointControlPara.jerk = (0, 0, 0, 0, 0, 0)
pointControlPara.coordinateType = basestruct.CoordinateType.baseCoordinate
pointControlPara.pointTransType = basestruct.PointTransType.pointTransStop
pointControlPara.pointTransRadius = 0
pointControlPara.poseTranType = 0
pointControlPara.motiontriggerMode = 1
pointControlPara.pose = (0, -438, 887, -57.34, 0, -149)    #目标点示例数据
result = cgxiapi.cr_moveL(robotHandle, pointControlPara)    #直线运动
print("返回结果:", result.value)
if result.value != 0:    #判断直线运动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.1.4 直线运动 moveline

函数

```
cr_move_line(robotHandle, pointControlPara, isBlock)
```

简介

直线运动。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数
isBlock	c_int	输入	是否阻塞 1-阻塞 0-非阻塞

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
pointControlParaSimple.jointpos = (60, 60, -60, 60, 60, 60)
pointControlParaSimple.pose = ( 0, -438, 887, -57.34, 0, -149)
pointControlParaSimple.tcpOffset = (0,0,0,0,0,0)
pointControlParaSimple.tcpID = -1
pointControlParaSimple.speed = (speed,speed,speed,speed,speed,speed)
pointControlParaSimple.acc = (acc,acc,acc,acc,acc,acc)
pointControlParaSimple.coordinatePose = (0,0,0,0,0,0)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.baseCoordinate
pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = \
    basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle,pointControlParaSimple,
pointControlPara) #PointControlParaSimple 数据转换为 PointControlPara
if result.value != 0: #判断 PointControlParaSimple 数据转换为 PointControlPara 是否成功
    exit()
isBlock = 0
result = cgxiapi.cr_move_line(robotHandle, pointControlPara, isBlock) #直线运动
print("返回结果:",result.value)
if result.value != 0: #判断直线运动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.5 点动运动

函数

`cr_moveJog(robotHandle, pointControlPara)`

简介

点动运动。连续点动控制变化时位置一次最多变 100mm，角度最多变 20°。移动相关速度和加速度范围见附录 A 速度和加速度范围。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlPara	PointControlPara	输入	运动相关参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlPara = basestruct.PointControlPara()
pointControlPara.jointpos = (0, 0, 0, 0, 0, 0) #示例数据
pointControlPara.pose = (0, 5, 0, 0, 0, 0) #沿 y 轴运动 5mm
pointControlPara.tcpOffset = (0, 0, 0, 0, 0, 0)
pointControlPara.tcpID = -1
pointControlPara.speed = (30, 30, 30, 30, 30, 30)
pointControlPara.acc = (30, 30, 30, 30, 30, 30)
pointControlPara.coordinatePose = (114.3, -390.498, 215.347, -180, 0, 0) #点坐标系
pointControlPara.jerk = (0, 0, 0, 0, 0, 0)
pointControlPara.coordinateType = basestruct.CoordinateType.PointCoordinate
#基于点坐标系运动
pointControlPara.pointTransType = basestruct.PointTransType.pointTransStop
pointControlPara.pointTransRadius = 0
pointControlPara.poseTranType = basestruct.PoseTranType.poseTranMoveToTargetPose
pointControlPara.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result=cgxiapi.cr_moveJog(robotHandle, pointControlPara) #点动运动
if result.value != 0: #判断点动运动是否成功
    exit()
```

```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.6 圆弧运动

函数

```
cr_move_circle(robotHandle, pointControlParaList, isBlock)
```

简介

圆弧运动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlParaList	PointControlParaList	输入	圆弧运动相关参数
isBlock	c_int	输入	1-阻塞 0-非阻塞

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。移动相关速度和加速度范围见[附录 A 速度和加速度范围](#)。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
pointControlParaSimple.jointpos = (60, 60, -60, 60, 60, 60)
pointControlParaSimple.pose = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.speed = (30, 30, 30, 30, 30, 30)
pointControlParaSimple.acc = (30, 30, 30, 30, 30, 30)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = \
    basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
```

```

result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple
, pointControlPara) #PointControlParaSimple 转换为 PointControlPara
if result.value != 0: #判断 PointControlParaSimple 转换为 PointControlPara 是否成功
    exit()
cgxiapi.cr_move_joint(robotHandle, pointControlPara, 1) # 移动到初始点
jointpos1 = (60, 55, -65, 60, 60, 60) #中间点关节角
pose1 = [0, 0, 0, 0, 0, 0] #中间点位姿
cgxiapi.cr_kineForward(robotHandle, jointpos1, pose1) #正解求中间点位姿
jointpos2 = (60, 50, -70, 60, 60, 60) #终点关节角
pose2 = [0, 0, 0, 0, 0, 0] #终点位姿
result = cgxiapi.cr_kineForward(robotHandle, jointpos2, pose2) #正解求终点位姿
pointControlParaList = basestruct.PointControlParaList()
index = 0
while index < basestruct.ROB_AXIS_NUM:
    pointControlParaList.pointcontrolpara[0].pose[index] = pose1[index]
    pointControlParaList.pointcontrolpara[0].jointpos[index] = jointpos1[index]
    pointControlParaList.pointcontrolpara[0].tcpOffset[index] = 0
    pointControlParaList.pointcontrolpara[0].coordinatePose[index] = 0
    pointControlParaList.pointcontrolpara[0].speed[index] = 30
    pointControlParaList.pointcontrolpara[0].acc[index] = 30
    pointControlParaList.pointcontrolpara[0].jerk[index] = 60
    pointControlParaList.pointcontrolpara[1].pose[index] = pose2[index]
    pointControlParaList.pointcontrolpara[1].jointpos[index] = jointpos2[index]
    pointControlParaList.pointcontrolpara[1].tcpOffset[index] = 0
    pointControlParaList.pointcontrolpara[1].coordinatePose[index] = 0
    pointControlParaList.pointcontrolpara[1].speed[index] = 30
    pointControlParaList.pointcontrolpara[1].acc[index] = 30
    pointControlParaList.pointcontrolpara[1].jerk[index] = 60
    index += 1
pointControlParaList.pointcontrolpara[0].tcpID = -1
pointControlParaList.pointcontrolpara[0].coordinateType = \
    basestruct.CoordinateType.baseCoordinate
pointControlParaList.pointcontrolpara[0].pointTransType = \
    basestruct.PointTransType.pointTransStop
pointControlParaList.pointcontrolpara[0].pointTransRadius = 0
pointControlParaList.pointcontrolpara[0].poseTranType = \
    basestruct.PoseTranType.poseTranMoveToTargetPose
pointControlParaList.pointcontrolpara[0].motiontriggerMode = \
    basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
pointControlParaList.pointcontrolpara[1].tcpID = -1
pointControlParaList.pointcontrolpara[1].coordinateType = \
    basestruct.CoordinateType.baseCoordinate
pointControlParaList.pointcontrolpara[1].pointTransType = \
    basestruct.PointTransType.pointTransStop
pointControlParaList.pointcontrolpara[1].pointTransRadius = 0

```

```

pointControlParaList.pointcontrolpara[1].poseTranType = \
    basestruct.PoseTranType.poseTranMoveToTargetPose
pointControlParaList.pointcontrolpara[1].motiontriggerMode = \
    basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
pointControlParaList.fixedrot = 0
pointControlParaList.centralangle = 0
result = cgxiapi.cr_move_circle(robotHandle, pointControlParaList, 1) #圆弧运动
if result.value != 0: #判断圆弧运动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.1.7 运动控制

函数

cr_moveControl(robotHandle, moveType)

简介

机械臂运动控制：减速停止，立即停止，运动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
moveType	MoveType	输入	运动控制参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
moveType = 1 #运动模式设置为立即停止
result=cgxiapi.cr_moveControl(robotHandle, moveType) #运动控制
print("运动控制结果: ",result.value)
if result.value != 0: #判断运动控制是否成功
    exit()

```

```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.8 设置阻塞移动中参数阈值

函数

```
cr_move_block_threshold_set(robotHandle, poseDelt, jointDelt)
```

简介

设置阻塞移动中参数阈值。在阻塞移动中，单签位姿/关节角度与目标点的位姿/关节角度的相差值小于位姿/关节阈值时，则被判定为已移动到目标点。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
poseDelt	c_double	输入	位姿阈值 默认 0.02
jointDelt	c_double	输入	关节阈值 默认 0.1

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
poseDelt = 0.1
jointDelt = 0.1
#设置阻塞移动中参数阈值
result = cgxiapi.cr_move_block_threshold_set(robotHandle, poseDelt, jointDelt)
print("设置阻塞移动中参数阈值结果: ", result.value)
if result.value != 0:  #判断设置阻塞移动中参数阈值是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.9 PointControlParaSimple 数据转换

函数

`cr_move_pointControlPara_transfer(robotHandle,pointControlParaSimple,pointControlPara)`

简介

PointControlParaSimple 转换为 PointControlPara。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointControlParaSimple	PointControlParaSimple	输入	简化运动参数
pointControlPara	PointControlPara	输入	运动参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
pointControlParaSimple.jointpos = (15,-3,90,0,90,10)
pointControlParaSimple.pose = (0,0,0,0,0,0)
pointControlParaSimple.tcpOffset = (0,0,0,0,0,0)
pointControlParaSimple.tcpID = -1
pointControlParaSimple.speed = (speed,speed,speed,speed,speed,speed)
pointControlParaSimple.acc = (acc,acc,acc,acc,acc,acc)
pointControlParaSimple.coordinatePose = (0,0,0,0,0,0)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = \
    basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle,pointControlParaSimple,
pointControlPara) #PointControlParaSimple 转换为 PointControlPara
if result.value != 0: #判断 PointControlParaSimple 转换为 PointControlPara 是否成功
    exit()
```



```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.10 设置软件自由驱动启动

函数

cr_set_softFreeDriveEnabled(robotHandle, softFreeDriveEnabled)

简介

设置软件自由驱动启动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
softFreeDriveEnabled	c_int	输入	1 为进入拖拽模式，0 为退出拖拽模式

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
softFreeDriveEnabled = 0  #退出拖拽模式
result = cgxiapi.cr_set_softFreeDriveEnabled(robotHandle, softFreeDriveEnabled)
#设置软件自由驱动启动
if result.value != 0:  #判断设置软件自由驱动启动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.11 读机械臂运动状态

函数

`cr_get_robotMoveStatus(robotHandle)`

简介

读机械臂运动状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值是运动状态，0 为停止，1 为运动。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_robotMoveStatus(robotHandle) #读机械臂运动状态
print("读机械臂运动结果: ",result[0].value,"目前是否运动: ",result[1].value)
if result[0].value != 0: #判断读机械臂运动状态是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.1.12 读取移动缓存区中移动指令的数量

函数

`cr_move_cache_num_get(robotHandle)`

简介

读取当前移动缓存区中移动指令的数量，当 PointControlPara 的 MotiontriggerMode 参数为 MovetriggerbyMoveCache 时，对应的移动指令会进入缓存区。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值是移动指令的数量。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_move_cache_num_get(robotHandle)
#读取移动缓存区中移动指令的数量
if result[0].value != 0: #判断读取移动缓存区中移动指令的数量是否成功
    exit()
print("读取移动缓存区中移动指令结果: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.2 运动状态信息

3.2.2.1 读实际 TCP 位置

函数

```
cr_get_tcpActualPose(robotHandle, pose)
```

简介

读实际 tcp 位置

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pose	c_double*6	输出	待读取的值：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pose=[0,0,0,0,0,0]
result=cgxiapi.cr_get_tcpActualPose(robotHandle,pose) #读实际 TCP 位置
print("读机械臂实际 tcp 结果: ",result.value)
if result.value != 0: #判断读实际 TCP 位置是否成功
    exit()
print("实际 tcp 位姿为",pose)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.2.2 读目标 TCP 位置

函数

```
cr_get_tcpTargetPose(robotHandle, pose)
```

简介

读目标 tcp 位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pose	c_double*6	输出	待读取的值：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
```

```

if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pose=[1,1,0,0,0,0]
result=cgxiapi.cr_get_tcpTargetPose(robotHandle,pose)    #读目标 TCP 位置
print("读机械臂目标 tcp 结果: ",result.value)
if result.value != 0:    #判断读目标 TCP 位置是否成功
    exit()
print("目标位姿为: ",pose)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.2.3 读实际 TCP 速度

函数

```
cr_get_tcpActualSpeed(robotHandle, speed)
```

简介

读实际 tcp 速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
speed	c_double*6	输出	待读取的值: x, y, z (单位: mm/s); Rx, Ry, Rz (单位: °/s)

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
speed=[0,0,0,0,0,0]
result=cgxiapi.cr_get_tcpActualSpeed(robotHandle,speed)    #读实际 TCP 速度
print("读机械臂实际 tcp 速度结果: ",result.value)

```

```
if result.value != 0:    #判断读实际 TCP 速度是否成功
    exit()
print("实际速度为: ",speed)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.2.4 读目标 TCP 速度

函数

```
cr_get_tcpTargetSpeed(robotHandle, speed)
```

简介

读目标 TCP 速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
speed	double *	输出	待读取的值：x，y，z（单位：mm/s）； Rx，Ry，Rz（单位：°/s）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
speed = [0,0,0,0,0,0]
result = cgxiapi.cr_get_tcpTargetSpeed(robotHandle,speed)#读目标 TCP 速度
if result.value != 0:    #判断读目标 TCP 速度是否成功
    exit()
print("目标 TCP 速度: ",speed)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.2.5 读实际 TCP 加速度

函数

```
cr_get_tcpActualAcceleration(robotHandle, acceleration)
```

简介

读实际 TCP 加速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
acceleration	double *	输出	待读取的值：x, y, z（单位：mm/s ² ）； Rx, Ry, Rz（单位：°/s ² ）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
acceleration = [0,0,0,0,0,0]
result = cgxiapi.cr_get_tcpActualAcceleration(robotHandle,acceleration)#读实际 TCP 加速度
if result.value != 0: #判断读实际 TCP 加速度是否成功
    exit()
print("实际 TCP 加速度: ",acceleration)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.2.6 读当前使用的 TCP 偏移信息

函数

```
cr_get_currentTCPmsg(robotHandle, tcpMsg)
```

简介

读当前使用的 TCP 偏移信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
tcpMsgList	POINTER(TCPMsg)	输出	所有 TCP 偏移信息所在的数组，数组长度为 20 柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
tcpMsgList=basestruct.TCPMsg()
result=cgxiapi.cr_get_currentTCPmsg(robotHandle, tcpMsgList) #读当前使用的 TCP 偏移信息
print("读当前使用的 TCP 偏移信息结果: ",result.value)
if result.value != 0: #判断读当前使用的 TCP 偏移信息是否成功
    exit()
print("当前使用的 TCP 偏移信息为: ",tcpMsgList.tcpName)
i=0
while i<6:
    print(tcpMsgList.tcpOffset[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.2.7 读取当前使用负载信息

函数

```
cr_get_currentPayloadmsg(robotHandle, payloadMsg)
```

简介

读取当前负载信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
payloadMsgList	POINTER(PayLoad)	输出	所有负载信息所在的数组

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
payloadMsgList=basestruct.PayLoad()
result=cgxiapi.cr_get_currentPayloadmsg(robotHandle, payloadMsgList)
#读当前使用的负载信息
print("读取当前使用负载信息结果: ",result.value)
if result.value != 0: #判断读当前使用的负载信息是否成功
    exit()
print("当前使用负载信息为: ",payloadMsgList.payloadName,payloadMsgList.toolPayload)
i=0
while i<3:
    print(payloadMsgList.centerOfGravity[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.3 运动配置

3.2.3.1 读所有 TCP 偏移信息列表

函数

```
cr_get_allTCPmsg(robotHandle, tcpMsgList)
```

简介

读所有 TCP 偏移信息列表。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
tcpMsgList	POINTER(TCPMsg)	输入	所有 TCP 偏移信息所在的数组，数组长度为 20 柄

返回值

返回结果为长度为 2 的一维数组，第一个值为 CRresult 类型值，第二个是数组个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
stru_info = create_string_buffer(sizeof(basestruct.TCPMsg()) * 20)
tcpMsgList=POINTER(basestruct.TCPMsg)(stru_info)
result=cgxiapi.cr_get_allTCPmsg(robotHandle, tcpMsgList) #读所有 TCP 偏移信息列表
print("读所有 TCP 偏移信息列表结果: ",result[0].value,result[1].value)
if result[0].value != 0: #判断读所有 TCP 偏移信息列表是否成功
    exit()
validNumber = result[1].value
j=0
i=0
while j<validNumber:
    print(tcpMsgList[j].tcpName)
    while i<basestruct.ROB_AXIS_NUM:
        print(tcpMsgList[j].tcpOffset[i])
        i += 1
    j += 1
    i = 0
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.3.2 读取所有负载信息

函数

```
cr_get_allPayloadmsg(robotHandle, payloadMsgList)
```

简介

读取所有负载信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
payloadMsgList	POINTER(PayLoad)	输入	所有负载信息所在的数组

返回值

返回结果为长度为 2 的一维数组，第一个值为 CRresult 类型值，第二个是数组个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
stru_info = create_string_buffer(sizeof(basestruct.PayLoad()) * 20)
payloadMsgList=POINTER(basestruct.PayLoad)(stru_info)
result=cgxiapi.cr_get_allPayloadmsg(robotHandle, payloadMsgList) #读取所有负载信息
if result[0].value != 0: #判断读取所有负载信息是否成功
    exit()
print("读所有 TCP 偏移信息列表结果: ",result[0].value,result[1].value)
validNumber = result[1].value
j=0
i=0
while j<validNumber:
    print(payloadMsgList[j].payloadName)
    print(payloadMsgList[j].toolPayload)
    while i<3:
        print(payloadMsgList[j].centerOfGravity[i])
        i += 1
    j += 1
    i = 0
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.3.3 获取指定 TCPoffset 的位姿

函数

```
cr_get_AssignTCP_Pose(robotHandle, dstTCPoffset, dstPose)
```

简介

获取指定 TCPoffset 的位姿。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
dstTCPoffset	c_double*6	输入	需变换后 TCP 偏移量: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
dstPose	c_double*6	输出	变化为 dstTCPoffset 后的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
dstTCPoffset = [10,20,30,60,60,60]
dstPose = [10,0,0,0,0,0]
result=cgxiapi.cr_get_AssignTCP_Pose(robotHandle, dstTCPoffset, dstPose)
#获取指定 TCPoffset 的位姿
print("获取指定 TCPoffset 的位姿结果: ",result.value)
if result.value != 0: #判断获取指定 TCPoffset 的位姿是否成功
    exit()
print("指定 TCPoffset 的位姿为: ",dstPose)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4 关节状态

3.2.4.1 读实际关节位置

函数

`cr_get_jointActualPos(robotHandle, pos)`

简介

读实际关节位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pos	c_double*6	输出	待读取的值，关节 1~6（单位：°）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pos = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointActualPos(robotHandle, pos) #读实际关节位置
print("读实际关节位置结果: ",result.value)
if result.value != 0: #判断读实际关节位置是否成功
    exit()
print("实际关节位置为: ",pos)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.2 读目标关节位置

函数

`cr_get_jointTargetPos(robotHandle, jointpos)`

简介

读目标关节位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pos	c_double*6	输出	待读取的值, 关节 1~6 (单位: °)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pos = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointTargetPos(robotHandle, pos) #读目标关节位置
print("读目标关节位置结果: ",result.value)
if result.value != 0: #判断读目标关节位置是否成功
    exit()
print("目标关节位置为: ",pos)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接, 释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.3 读实际关节速度

函数

```
cr_get_jointActualVelocity(robotHandle, velocity)
```

简介

读实际关节速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
velocity	c_double*6	输出	待读取的值, 关节 1~6 (单位: °/s)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
velocity = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointActualVelocity(robotHandle, velocity) #读实际关节速度
print("读实际关节速度结果: ",result.value)
if result.value != 0: #判断读实际关节速度是否成功
    exit()
print("实际关节速度为: ",velocity)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.4 读目标关节速度

函数

```
cr_get_jointTargetVelocity(robotHandle, velocity)
```

简介

读目标关节速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
velocity	double *	输出	待读取的值，关节 1~6（单位：%s）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
```

```

robotHandle = re1[1].value
velocity = [0,0,0,0,0,0]
result = cgxiapi.cr_get_jointTargetVelocity(robotHandle,velocity)#读目标关节速度
if result.value != 0:    #判断读目标关节速度是否成功
    exit()
print("目标关节速度: ",velocity)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.4.5 读实际关节加速度

函数

```
cr_get_jointActualAcceleration(robotHandle, acceleration)
```

简介

读实际关节加速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
acceleration	double *	输出	待读取的值，关节 1~6（单位：°/s ² ）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
acceleration = [0,0,0,0,0,0]
result = cgxiapi.cr_get_jointActualAcceleration(robotHandle,acceleration)#读实际关节加速度
if result.value != 0:    #判断读实际关节加速度是否成功
    exit()
print("实际关节加速度: ",acceleration)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")

```


`exit()`

3.2.4.6 读目标关节加速度

函数

`cr_get_jointTargetAcceleration(robotHandle, acceleration)`

简介

读目标关节加速度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
acceleration	double *	输出	待读取的值，关节 1~6（单位：°/s ² ）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
acceleration = [0,0,0,0,0,0]
result = cgxiapi.cr_get_jointTargetAcceleration(robotHandle, acceleration)#读目标关节加速度
if result.value != 0: #判断读目标关节加速度是否成功
    exit()
print("目标关节加速度: ",acceleration)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.7 读实际关节电机电流

函数

`cr_get_jointActualCurrent(robotHandle, current)`

简介

读实际关节电机电流（数值和关节转矩等效，三关节符号相反）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
current	c_double*6	输出	待读取的值，关节 1~6（单位：mA）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
current = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointActualCurrent(robotHandle, current) #读实际关节电机电流
print("读实际关节电机电流结果: ",result.value)
if result.value != 0: #判断读实际关节电机电流是否成功
    exit()
print("实际关节电机电流为: ",current)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.8 读目标关节电机电流

函数

```
cr_get_jointTargetCurrent(robotHandle, current)
```

简介

读目标关节电机电流（数值和关节转矩等效，三关节符号相反）。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
current	c_double*6	输出	待读取的值，关节 1~6（单位：mA）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
current = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointTargetCurrent(robotHandle, current) #读目标关节电机电流
print("读目标关节电机电流结果: ",result.value)
if result.value != 0: #判断读目标关节电机电流是否成功
    exit()
print("目标关节电机电流为: ",current)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.9 读实际关节转矩

函数

```
cr_get_jointActualTorque(robotHandle, moment)
```

简介

读实际关节转矩。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
moment	c_double*6	输出	待读取的值，关节 1~6 (单位：额定转矩/1000)

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
```

```

    exit()
robotHandle = re1[1].value
moment = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointActualTorque(robotHandle, moment)  #读实际关节转矩
print("读目标关节扭矩结果: ",result.value)
if result.value != 0:  #判断读实际关节转矩是否成功
    exit()
print("目标关节扭矩为: ",moment)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.4.10 读实际关节采集的母线电压

函数

cr_get_jointActualVoltage(robotHandle, voltage)

简介

读实际关节采集的母线电压。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
voltage	c_double*6	输出	待读取的值，关节 1~6（单位：V）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
voltage = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointActualVoltage(robotHandle, voltage)  #读实际关节采集的母线电压
print("读实际关节采集的母线电压结果: ",result.value)
if result.value != 0:  #判断读实际关节采集的母线电压是否成功
    exit()
print("实际关节采集的母线电压为: ",voltage)

```

```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.11 读关节温度

函数

```
cr_get_jointTemperature(robotHandle, temperature)
```

简介

读关节温度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
temperature	c_double*6	输出	待读取的值，关节 1~6（单位：℃）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
temperature = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointTemperature(robotHandle, temperature)  #读关节温度
print("读各关节的温度结果: ",result.value)
if result.value != 0:  #判断读关节温度是否成功
    exit()
print("各关节的温度为: ",temperature)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.4.12 读关节模式

函数

```
cr_get_jointMode(robotHandle,jointMode)
```

简介

读关节模式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
jointMode	c_double*6	输出	返回关节模式

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
jointMode = [0,0,0,0,0,0]
result=cgxiapi.cr_get_jointMode(robotHandle,jointMode) #读关节模式
print("读各关节模式结果: ",result.value)
if result.value != 0: #判断读关节模式是否成功
    exit()
print("各关节模式为: ",jointMode[0].value,jointMode[1].value,jointMode[2].value,
    jointMode[3].value,jointMode[4].value,jointMode[5].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.5 轨迹复现

3.2.5.1 读取控制柜当前存在的轨迹索引

函数

```
cr_path_all_index_get(robotHandle)
```

简介

读取控制柜当前存在的轨迹索引。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是所有轨迹索引（最大为 51），第三个是轨迹索引数组有效长度。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_path_all_index_get(robotHandle)
#读取控制柜当前存在的轨迹索引
print("读取控制柜当前存在的轨迹索引结果: ", result[0].value)
if result[0].value != 0: #判断读取控制柜当前存在的轨迹索引是否成功
    exit()
print("所有轨迹索引:",result[1][0:result[2].value],"轨迹索引数组有效长度:",result[2].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.5.2 上传轨迹

函数

```
cr_path_upload(robotHandle, index, pathData)
```

简介

上传轨迹。运行前可通过 `cr_path_all_index_get` 先判断 `index` 是否存在。上传轨迹的请求索引号的取值范围为：-1~49。其中 0~49 索引下的轨迹保存在控制器 Flash 中，-1 索引下的轨迹数据是上一次轨迹记录控制功能中记录下的轨迹数据，存放在控制器 RAM 中，下电清空。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	轨迹索引：-1~49
pathData	POINTER(PathData)	输出	轨迹数据

返回值

数据类型为 `CRresult`，若为 `success` 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1 #轨迹索引
pathData = basestruct.PathData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
result = cgxiapi.cr_path_upload(robotHandle, index, pathData)
#上传索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
print("上传轨迹结果: ",result.value)
if result.value != 0: #判断上传轨迹是否成功
    exit()
print(pathData.pathPoints[5].pose[0:])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.5.3 下载轨迹

函数

```
cr_path_download(robotHandle, pathPara)
```


简介

下载轨迹。该功能会覆盖控制柜已存在的轨迹，建议运行前通过 `cr_path_all_index_get` 接口先判断该索引是否存在。轨迹下载接口是多个功能复用的接口，其支持 TCP 轨迹复现、关节轨迹复现和分段 movex 功能。当 `moveType` 为 0 时，调用该接口时机器人会直接运行，提供的轨迹数据需要按照 2ms 规划，运行轨迹之前需将机器人移动到轨迹的第一个点位；当 `moveType` 为 1 时为 TCP 轨迹复现，需要调用 `cr_path_action` 接口来运行轨迹；当 `moveType` 为 4 时为关节轨迹复现，需要调用 `cr_path_action` 接口来运行轨迹；当 `moveType` 为 3 时，movex 运动，需要设置速度、加速度、交融半径，需要调用 `cr_path_action` 接口来运行轨迹。

注：在使用相关功能时，需要保证机械臂已经移动到轨迹起始点对应的位置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pathDownloadData	PathDownloadData	输入	下载轨迹参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例 1：下载轨迹

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathDownloadData = basestruct.PathDownloadData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathDownloadData.pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
filePath = b"D:/trait.crpath"
result = cgxiapi.cr_path_file2pathData(filePath, pathDownloadData.pathData)
#轨迹文件转换为轨迹数据
pathDownloadData.pathPara.index = 1
pathDownloadData.pathPara.moveType = 1
pathDownloadData.pathData.moveTime = 100
result = cgxiapi.cr_path_download(robotHandle,pathDownloadData) #下载轨迹到索引 1
print("下载轨迹结果: ",result.value)
if result.value != 0: #判断下载轨迹是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
```

```
print("断开失败")
exit()
```

示例 2：TCP 轨迹复现

(1) 调用 `cr_path_download` 接口将所有 TCP 位置和关节位置下载到机械臂，运动轨迹类型 `movetype` 写 1，运动时间 `moveTime` 写对应执行周期

(2) 调用 `cr_path_action` 执行对应索引的轨迹，`runContol` 参数写 1

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") # 机械臂连接
print("机械臂建立连接返回值: ", re1[0].value)
if re1[0].value != 0: # 判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathData = basestruct.PathData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
filePath = b"D:/trait.crpah"
result = cgxiapi.cr_path_file2pathData(filePath, pathData)
# 轨迹文件转换为轨迹数据
pathDownloadData = basestruct.PathDownloadData()
pathDownloadData.pathData = pathData
pathDownloadData.pathPara.index = 1
pathDownloadData.pathPara.moveType = 1
pathDownloadData.pathData.moveTime = 100
result = cgxiapi.cr_path_download(robotHandle, pathDownloadData) # 下载轨迹到索引 1
print("下载轨迹结果: ", result.value)
if result.value != 0: # 判断下载轨迹是否成功
    exit()
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
i = 0
while i < 6:
    pointControlParaSimple.jointpos[i] = pathData.pathPoints[0].jointpos[i]
    pointControlParaSimple.pose[i] = pathData.pathPoints[0].pose[i]
    i = i + 1
pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.tcpID = -1
pointControlParaSimple.speed = (speed, speed, speed, speed, speed, speed)
pointControlParaSimple.acc = (acc, acc, acc, acc, acc, acc)
pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
```

```

pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
                                                    pointControlPara)

# pointControlParaSimple 转化为 pointControlPara
if result.value != 0: # 判断 pointControlParaSimple 转化为 pointControlPara 是否成功
    exit()
isBlock = 1
result = cgxiapi.cr_move_joint(robotHandle, pointControlPara, isBlock)
# 移动至第一个点位
if result.value != 0: # 判断移动至第一个点位是否成功
    exit()
index = 1
runControl = 1
result = cgxiapi.cr_path_action(robotHandle, index, runControl)
# 启动轨迹索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
print("轨迹控制结果：", result.value)
if result.value != 0: # 判断轨迹索引为 1 的轨迹是否开启成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) # 机器人断开连接，释放句柄
if re0.value != 0: # 判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

示例 3：关节轨迹复现

- （1）调用 `cr_path_download` 接口将所有关节位置下载到机械臂，运动轨迹类型 `movetype` 写 4，运动时间 `moveTime` 写对应的执行周期
- （2）调用 `cr_path_action` 执行对应索引的轨迹，`runContol` 参数写 1

```

robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") # 机械臂连接
print("机械臂建立连接返回值：", re1[0].value)
if re1[0].value != 0: # 判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathData = basestruct.PathData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
filePath = b"D:/trait.crpah"
result = cgxiapi.cr_path_file2pathData(filePath, pathData)
# 轨迹文件转换为轨迹数据
pathDownloadData = basestruct.PathDownloadData()
pathDownloadData.pathData = pathData

```

```
pathDownloadData.pathPara.index = 1
pathDownloadData.pathPara.moveType = 4
pathDownloadData.pathData.moveTime = 100
result = cgxiapi.cr_path_download(robotHandle, pathDownloadData) #下载轨迹到索引 1
print("下载轨迹结果: ", result.value)
if result.value != 0: # 判断下载轨迹是否成功
    exit()
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
i = 0
while i < 6:
    pointControlParaSimple.jointpos[i] = pathData.pathPoints[0].jointpos[i]
    pointControlParaSimple.pose[i] = pathData.pathPoints[0].pose[i]
    i = i + 1
pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.tcpID = -1
pointControlParaSimple.speed = (speed, speed, speed, speed, speed, speed)
pointControlParaSimple.acc = (acc, acc, acc, acc, acc, acc)
pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple,
                                                    pointControlPara)

# pointControlParaSimple 转化为 pointControlPara
if result.value != 0: # 判断 pointControlParaSimple 转化为 pointControlPara 是否成功
    exit()
isBlock = 1
result = cgxiapi.cr_move_joint(robotHandle, pointControlPara, isBlock)
# 移动至第一个点位
if result.value != 0: # 判断移动至第一个点位是否成功
    exit()
index = 1
runControl = 1
result = cgxiapi.cr_path_action(robotHandle, index, runControl)
# 启动轨迹索引为 1 的轨迹, 使用前需判断索引为 1 的轨迹是否存在
print("轨迹控制结果: ", result.value)
if result.value != 0: # 判断轨迹索引为 1 的轨迹是否开启成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) # 机器人断开连接, 释放句柄
if re0.value != 0: # 判断机器人断开连接是否成功
```

```
print("断开失败")
exit()
```

示例 4：分段 moveX

(1) 调用 `cr_path_download` 接口将部分 TCP 位置和关节位置下载到机械臂，运动轨迹类型 `movetype` 写 3，运动时间 `moveTime` 写 2，速度 `speed` 写 `movex` 的运动速度，加速度 `acc` 写 `movex` 的加速度，交融半径 `blendRadius` 写 `movex` 的交融半径

(2) 调用 `cr_path_action` 执行对应索引的分段 `Movex`，`runControl` 参数写 1

(3) 多次调用 `cr_path_download` 接口写入后续的 TCP 位置和关节位置轨迹，参数和第一步中的参数一致

(4) 调用 `cr_path_action` 关闭分段 `Movex`，`runControl` 参数写 0

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机器人连接
print("机械臂建立连接返回值: ", re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathData = basestruct.PathData()
stru_info = create_string_buffer(ctypes.sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
pathDownloadData = basestruct.PathDownloadData()
joints = (ctypes.c_double * 6)()
pose = (ctypes.c_double * 6)()
startPose = (ctypes.c_double * 6)()
result = cgxiapi.cr_get_jointActualPos(robotHandle, joints)
if result.value != 0:
    exit()
result = cgxiapi.cr_get_tcpActualPose(robotHandle, startPose)
if result.value != 0:
    exit()
for i in range(6):
    pose[i] = startPose[i]
# 下载第一段 Movex 轨迹
for i in range(5):
    pose[0] = startPose[0] + (-1) ** i * 50.0
    pose[1] = startPose[1] + 50.0 * i
    for j in range(6):
        pathData.pathPoints[i].pose[j] = pose[j]
        pathData.pathPoints[i].jointpos[j] = joints[j]
pathData.pathPointsNum = 5
pathData.moveTime = 2
pathDownloadData.pathData = pathData
pathDownloadData.pathPara.index = 1
```

```

pathDownloadData.pathPara.moveType = 3          #分段 Movex 类型
pathDownloadData.pathPara.speed = 50            #Movex 速度 mm/s
pathDownloadData.pathPara.acc = 100             #Movex 加速度 mm/s^2
pathDownloadData.pathPara.blendRadius = 10       #Movex 交融半径 mm
result = cgxiapi.cr_path_download(robotHandle, pathDownloadData)
if result.value != 0:
    exit()
#开启分段 Movex, runControl 写 1
result = cgxiapi.cr_path_action(robotHandle, pathDownloadData.pathPara.index, 1)
if result.value != 0:
    exit()
#等待 5s, 下载第二段轨迹
time.sleep(5)
for i in range(5):
    pose[0] = startPose[0] - 50.0 * i
    pose[1] = startPose[1] + (-1) ** i * 50.0
    for j in range(6):
        pathData.pathPoints[i].pose[j] = pose[j]
        pathData.pathPoints[i].jointpos[j] = joints[j]
pathData.pathPointsNum = 5
pathDownloadData.pathData = pathData
result = cgxiapi.cr_path_download(robotHandle, pathDownloadData)
if result.value != 0:
    exit()
#等待 5s, 下载第三段轨迹
time.sleep(5)
for i in range(5):
    pose[0] = startPose[0] - (-1) ** i * 50.0
    pose[2] = startPose[2] - 50.0 * i
    for j in range(6):
        pathData.pathPoints[i].pose[j] = pose[j]
        pathData.pathPoints[i].jointpos[j] = joints[j]
pathData.pathPointsNum = 5
pathDownloadData.pathData = pathData
result = cgxiapi.cr_path_download(robotHandle, pathDownloadData)
if result.value != 0:
    exit()
#关闭分段 Movex, runControl 写 0
result = cgxiapi.cr_path_action(robotHandle, pathDownloadData.pathPara.index, 0)
if result.value != 0:
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)
if re0.value != 0:          #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.5.4 轨迹控制

函数

`cr_path_action(robotHandle, index, runControl)`

简介

轨迹控制。运行前可通过 `cr_path_all_index_get` 先判断轨迹索引 `pathIndex` 是否存在。启动轨迹之前需将机器人移动到轨迹的第一个点位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	轨迹索引（范围：0~49）
runControl	c_int	输入	0-停止，1-启动，2-暂停

返回值

数据类型为 `CResult`，若为 `success` 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
pathData = basestruct.PathData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
result = cgxiapi.cr_path_upload(robotHandle, index, pathData)
#上传索引为1的轨迹，使用前需判断索引为1的轨迹是否存在
if result.value != 0: #判断上传轨迹是否成功
    exit()
pointControlParaSimple = basestruct.PointControlParaSimple()
pointControlPara = basestruct.PointControlPara()
speed = 10
acc = 250
i = 0
while i<6:
    pointControlParaSimple.jointpos[i] = pathData.pathPoints[0].jointpos[i]
    pointControlParaSimple.pose[i] = pathData.pathPoints[0].pose[i]
    i = i+1
pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
```

```

pointControlParaSimple.tcpID = -1
pointControlParaSimple.speed = (speed, speed, speed, speed, speed, speed)
pointControlParaSimple.acc = (acc, acc, acc, acc, acc, acc)
pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
pointControlParaSimple.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
result = cgxiapi.cr_move_pointControlPara_transfer(robotHandle, pointControlParaSimple, pointControlPara) #pointControlParaSimple 转化为 pointControlPara
if result.value != 0: #判断 pointControlParaSimple 转化为 pointControlPara 是否成功
    exit()
isBlock = 1
result = cgxiapi.cr_move_joint(robotHandle, pointControlPara, isBlock) #移动至第一个点位
if result.value != 0: #判断移动至第一个点位是否成功
    exit()
index = 1
runControl = 1
result = cgxiapi.cr_path_action(robotHandle, index, runControl)
#启动轨迹索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
print("轨迹控制结果：", result.value)
if result.value != 0: #判断轨迹索引为 1 的轨迹是否开启成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.5.5 轨迹数据转换为轨迹文件

函数

cr_path_pathData2file(pathPara, filePath)

简介

轨迹数据转换为轨迹文件。

参数

参数名称	数据类型	输入/输出	描述
pathData	PathData	输入	轨迹参数
filePath	POINTER(c_char)	输入	文件保存路径

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathData = basestruct.PathData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
index = 1
result = cgxiapi.cr_path_upload(robotHandle, index, pathData)
#上传索引为 1 的轨迹，使用前需判断索引为 1 的轨迹是否存在
filePath = b"D:/path.crpath"
result = cgxiapi.cr_path_pathData2file(pathData,filePath) #轨迹数据转换为轨迹文件
print("轨迹数据转换为轨迹文件结果: ",result.value)
if result.value != 0: #判断轨迹数据转换为轨迹文件是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.5.6 轨迹文件转换为轨迹数据

函数

```
cr_path_file2pathData(filePath,pathPara)
```

简介

轨迹文件转换为轨迹数据。

参数

参数名称	数据类型	输入/输出	描述
filePath	POINTER(c_char)	输入	文件保存路径
pathData	POINTER(PathData)	输出	轨迹参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathData = basestruct.PathData()
stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
filePath = b"D:/trait.crpah"
result = cgxiapi.cr_path_file2pathData(filePath,pathData) #轨迹文件转换为轨迹数据
print("轨迹文件转换为轨迹数据结果: ", result.value)
if result.value != 0: #判断轨迹文件转换为轨迹数据是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.2.5.7 设置轨迹记录参数

函数

cr_path_recordPara_set(robotHandle, recordPathPara)

简介

设置轨迹记录参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
recordPathPara	RecordPathPara	输入	轨迹记录参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
recordPathPara = basestruct.RecordPathPara()
recordPathPara.recordControl = 1    #记录控制, 0-停止, 1-启动, 2-暂停
recordPathPara.sampleTime = 2
result = cgxiapi.cr_path_recordPara_set(robotHandle, recordPathPara) #设置轨迹记录参数
if result.value != 0:    #判断设置轨迹记录参数是否成功
    exit()
    print("设置轨迹记录参数结果: ", result.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接, 释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.5.8 获取当前轨迹记录状态

函数

```
cr_path_recordStatus_get(robotHandle, pathRecordStatus)
```

简介

获取当前轨迹记录状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pathRecordStatus	POINTER(PathRecordStatus)	输出	当前轨迹记录状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathRecordStatus = basestruct.PathRecordStatus()

```

```

result = cgxiapi.cr_path_recordStatus_get(robotHandle, pathRecordStatus)
#获取当前轨迹记录状态
if result.value != 0:    #判断获取当前轨迹记录状态是否成功
    exit()
    print("获取当前轨迹记录状态结果：", result.value)
    print("轨迹记录状态：", pathRecordStatus.recordStatus
        , "已记录点数：", pathRecordStatus.waypointNumber)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.5.9 获取当前轨迹运行状态

函数

```
cr_path_currentRunStatus_get(robotHandle, pathRunMsg)
```

简介

获取当前轨迹运行状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pathRunMsg	POINTER(PathRunMsg)	输出	当前轨迹运行状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pathRunMsg = basestruct.PathRunMsg()
result = cgxiapi.cr_path_currentRunStatus_get(robotHandle, pathRunMsg)
#获取当前轨迹运行状态
if result.value != 0:    #判断获取当前轨迹运行状态是否成功
    exit()
    print("获取当前轨迹运行状态结果：", result.value)
    print("当前轨迹运行状态：", pathRunMsg.pathrunstatus, "当前运行轨迹点数：",

```

```

pathRunMsg.pointIndex)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.6 扩展轴运动

3.2.6.1 扩展轴关节移动

函数

```
cr_move_joint_exj(robotHandle, moveExjPara, isBlock)
```

简介

扩展轴关节移动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
moveExjPara	MoveExjPara	输入	扩展轴关节移动相关参数
isBlock	BOOL	输入	TRUE-阻塞 FALSE-非阻塞

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
moveExjPara = basestruct.MoveExjPara()
temp_pose = [110, -613, 1129, -57.34, 0.0, 148.98, 0, 0, 0, 0, 0, 0]  #前六个代表关节姿态
temp_jointpose = [50, 50, -50, 50, 50, 50, 300, 200, 0, 0, 0, 0]  #前六个代表关节 1~6 关节角度，后六个代表扩展轴位置
for i in range(12):
    moveExjPara.pose[i] = temp_pose[i]
    moveExjPara.jointPos[i] = temp_jointpose[i]
    if i < 6:
        moveExjPara.speed[i] = 30

```

```

        moveExjPara.acc[i] = 60
        moveExjPara.tcpOffset[i] = 0
        moveExjPara.coordinatePose[i] = 0
moveExjPara.speed[6] = 100    #扩展轴直线速度
moveExjPara.speed[7] = 200    #扩展轴旋转速度
moveExjPara.tcpId = -1
moveExjPara.coordinateType = basestruct.CoordinateType.jointCoordinate
moveExjPara.pointTransRadius = 0
moveExjPara.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
isBlock = 1
result = cgxiapi.cr_move_joint_exj(robotHandle, moveExjPara, isBlock)    #扩展轴关节移动
print("返回结果：", result.value)
if result.value != 0:    #判断扩展轴关节移动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.6.2 扩展轴直线移动

函数

```
cr_move_line_exj(robotHandle, moveExjPara, isBlock)
```

简介

扩展轴直线移动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
moveExjPara	MoveExjPara	输入	扩展轴直线移动相关参数
isBlock	BOOL	输入	TRUE-阻塞 FALSE-非阻塞

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
moveExjPara = basestruct.MoveExjPara()
temp_pose = [110, -613, 1129, -57.34, 0.0, 148.98, 200, 300, 0, 0, 0, 0]    #前六个代表关节姿态,后六个代表扩展轴位置
temp_jointpose = [50, 50, -50, 50, 50, 50, 0, 0, 0, 0, 0, 0]    #前六个代表关节 1~6 关节角度
for i in range(12):
    moveExjPara.pose[i] = temp_pose[i]
    moveExjPara.jointPos[i] = temp_jointpose[i]
    if i < 6:
        moveExjPara.speed[i] = 30
        moveExjPara.acc[i] = 60
        moveExjPara.tcpOffset[i] = 0
        moveExjPara.coordinatePose[i] = 0
moveExjPara.speed[6] = 100
moveExjPara.speed[7] = 200
moveExjPara.tcpId = -1
moveExjPara.coordinateType = basestruct.CoordinateType.baseCoordinate
moveExjPara.pointTransRadius = 0
moveExjPara.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
isBlock = 1
result = cgxiapi.cr_move_line_exj(robotHandle, moveExjPara, isBlock)    #扩展轴直线移动
print("返回结果: ", result.value)
if result.value != 0:    #判断扩展轴直线移动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接, 释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.6.3 扩展轴点动移动

函数

```
cr_moveJog_exj(robotHandle, moveJogExjPara)
```

简介

扩展轴点动移动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
moveJogExjPara	MoveJogExjPara	输入	扩展轴点动移动相关参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
moveJogExjPara = basestruct.MoveJogExjPara()
for index in range(10):
    moveJogExjPara.jointPos[index] = 0
moveJogExjPara.speed[0] = 50
moveJogExjPara.acc[0] = 100
moveJogExjPara.jointPos[0] = 100
result = cgxiapi.cr_moveJog_exj(robotHandle, moveJogExjPara)  #扩展轴点动移动
print("返回结果: ", result.value)
if result.value != 0:  #判断扩展轴点动移动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.2.6.4 扩展轴运动

函数

```
cr_exj_move(robotHandle, exjMovePara, isBlock)
```

简介

扩展轴运动。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
exjMovePara	ExjMovePara	输入	扩展轴运动相关参数
isBlock	BOOL	输入	TRUE-阻塞 FALSE-非阻塞

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
```



```
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
exjMovePara = basestruct.ExjMovePara()
exjMovePara.exjCount = 1
exjMovePara.acctime = 100.0
exjMovePara.index[0] = 0
exjMovePara.position[0] = 100.0
exjMovePara.speed[0] = 50.0
exjMovePara.motiontriggerMode = basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
isBlock = 1
result = cgxiapi.cr_exj_move(robotHandle, exjMovePara, isBlock) #扩展轴运动
print("返回结果: ", result.value)
if result.value != 0: #判断扩展轴运动是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3 系统属性

3.3.1 硬件信息

3.3.1.1 读取控制柜硬件序列号

函数

cr_get_productInfo(robotHandle)

简介

读取控制柜硬件序列号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是序列号。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_productInfo(robotHandle) #读取控制柜硬件序列号
print("读取控制柜硬件序列号结果: ",result[0].value)
if result[0].value != 0: #判断读取控制柜硬件序列号是否成功
    exit()
print("控制柜硬件序列号为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.3.1.2 读取整臂序列号

函数

cr_get_productSn(robotHandle)

简介

读取整臂序列号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是序列号。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_productSn(robotHandle) #读取整臂序列号

```

```

print("读取整臂序列号结果: ",result[0].value)
if result[0].value != 0:    #判断读取整臂序列号是否成功
    exit()
print("整臂序列号为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.3.1.3 获取版本信息

函数

cr_get_sysVersion(robotHandle, version)

简介

获取版本号信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
version	POINTER(SysVersion)	输出	返回版本号信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
version = basestruct.SysVersion()
result=cgxiapi.cr_get_sysVersion(robotHandle, version)    #获取版本信息
print("获取版本信息结果: ",result.value)
if result.value != 0:    #判断获取版本信息是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.3.2 本体参数

3.3.2.1 读机械臂 DH 参数

函数

cr_get_DhParmeter(robotHandle, dhPara)

简介

读机械臂 DH 参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
dhPara	c_double*100	输出	机械臂 DH 参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
dhparam = [0]*100
result=cgxiapi.cr_get_DhParmeter(robotHandle, dhparam) #读机械臂 DH 参数
print("读取 dh 参数结果: ",result.value)
if result.value != 0: #判断读机械臂 DH 参数是否成功
    exit()
i = 0
while i<36:
    print(dhparam[i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.2.2 读机械臂标准 DH 参数

函数

```
cr_get_stdDhParmeter(robotHandle, dhPara)
```

简介

读机械臂标准 DH 参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
dhPara	c_double*100	输出	机械臂 DH 参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
dhpara = [0]*100
result=cgxiapi.cr_get_stdDhParmeter(robotHandle, dhpara) #读机械臂标准 DH 参数
print("读取 dh 参数结果: ",result.value)
if result.value != 0: #判断读机械臂标准 DH 参数是否成功
    exit()
i = 0
while i<36:
    print(dhpara[i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.2.3 读机械臂动力学参数

函数

```
cr_get_DynamicParmeter(robotHandle, dynamicPara)
```

简介

读机械臂动力学参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
dynamicPara	c_double*100	输出	机械臂动力学参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
dynamicPara = [0] * 100
result=cgxiapi.cr_get_DynamicParmeter(robotHandle, dynamicPara) #读机械臂动力学参数
print("读取动力学参数结果: ",result.value)
if result.value != 0: #判断读机械臂动力学参数是否成功
    exit()
i = 0
while i<72:
    print(dynamicPara[i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.2.4 读机械臂零位补偿值

函数

```
cr_get_zeroCompensationOffset(robotHandle, zeroCompensationOffset)
```

简介

读机械臂零位补偿值

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
zeroCompensationOffset	c_double*6	输出	机械臂零位补偿值

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
zeroCompensationOffset = [0,0,0,0,0,0]
result=cgxiapi.cr_get_zeroCompensationOffset(robotHandle, zeroCompensationOffset)
#读机械臂零位补偿值
print("读取零位补偿值结果: ",result.value)
if result.value != 0: #判断读机械臂零位补偿值是否成功
    exit()
print("零位补偿值为: ",zeroCompensationOffset)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.2.5 读机械臂减速比

函数

```
cr_get_reducerRatio(robotHandle, reducerRatio)
```

简介

读机械臂减速比。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
reducerRatio	c_double*6	输出	机械臂减速比

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
reducerRatio = [0,0,0,0,0,0]
result=cgxiapi.cr_get_reducerRatio(robotHandle, reducerRatio) #读机械臂减速比
print("读取机械臂减速比结果: ",result.value)
if result.value != 0: #判断读机械臂减速比是否成功
    exit()
print("机械臂减速比为: ",reducerRatio)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3 系统信息

3.3.3.1 读机械臂当前状态

函数

cr_get_robotMode(robotHandle)

简介

读机械臂当前状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果为长度为 2 的一维数组，第一个值为 CResult 类型值，第二个是机械臂当前状态。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
```



```

    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_robotMode(robotHandle) #读机械臂当前状态
print("读当前机械臂状态结果: ",result[0].value)
if result[0].value != 0: #判断读机械臂当前状态是否成功
    exit()
print("当前状态为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.3.3.2 获取机械臂日志数量

函数

```
cr_cfg_log_number_get(robotHandle)
```

简介

获取机械臂实时日志数量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是日志个数。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_cfg_log_number_get(robotHandle)
print("获取机械臂日志数量结果: ", result[0].value)
print("日志个数为: ", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.3.3.3 获取机械臂实时日志

函数

cr_get_logMsgs(robotHandle, logMsgs, logMsgNumber)

简介

获取机械臂实时日志。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
logMsgs	POINTER(RobotLogMsg)	输出	机械臂日志信息数组
logMsgNumber	POINTER(c_int)	输出	日志个数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
stru_info = create_string_buffer(sizeof(basestruct.RobotLogMsg) * 10)
logmsgs = POINTER(basestruct.RobotLogMsg)(stru_info)
while 1:
    logMsgNumber = c_int(20)
    result = cgxiapi.cr_get_logMsgs(robotHandle, logmsgs, logMsgNumber) #获取机械臂日志
    print("获取机械臂日志结果: ", result.value)
    if result.value != 0: #判断获取当前日志是否成功
        exit()
    print("日志个数为: ", logMsgNumber.value)
    if logMsgNumber.value > 0:
        msg = logmsgs[0].textMessage
        str_msg = str(msg, 'utf-8')
        print(logmsgs[0].robotMessageCode, str_msg)
        time.sleep(2)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
```

`exit()`

3.3.3.4 获取机械臂历史所有日志

函数

`cr_sys_history_log_get(robotHandle, logmsgs)`

简介

获取机械臂历史所有日志。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
logMsgs	POINTER(RobotLogMsg)	输出	机械臂日志信息数组

返回值

返回结果是长度为 2 的一维数组，第一个是 CResult 类型的值，第二个是日志个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
stru_info = create_string_buffer(sizeof(basestruct.RobotLogMsg) * 3000)
logmsgs = POINTER(basestruct.RobotLogMsg)(stru_info)
result = cgxiapi.cr_sys_history_log_get(robotHandle, logmsgs) #获取机械臂历史所有日志
if result[0].value != 0: #判断获取机械臂历史所有日志是否成功
    exit()
if result[1].value > 0:
    msg = logmsgs[0].textMessage
    str_msg = str(msg, 'utf-8')
    print(logmsgs[0].robotMessageCode, str_msg)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3.5 日志功能是否开启

函数

cr_log_enable(enable)

简介

日志功能是否开启。需要在机器人创建连接之前使用。

参数

参数名称	数据类型	输入/输出	描述
enable	c_int	输入	是否开启 1-开启 0-不开启

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
enable = 1
result = cgxiapi.cr_log_enable(enable) #日志功能开启
print("返回结果:",result.value)
if result.value != 0: #判断日志功能是否开启是否成功
    exit()
```

3.3.3.6 日志大小设置

函数

cr_log_set_size(size, num)

简介

日志大小设置。需要在机器人创建连接之前使用。

参数

参数名称	数据类型	输入/输出	描述
size	c_long	输入	单个日志文字大小 字节
num	c_int	输入	日志文件数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
enable = 1
result = cgxiapi.cr_log_enable(enable) #日志功能开启
print("返回结果:",result.value)
if result.value != 0: #判断日志功能是否开启是否成功
```

```
exit()
size = 1024*1024
num = 10
result = cgxiapi.cr_log_set_size(size, num) #日志大小设置
print("返回结果:",result.value)
if result.value != 0: #判断日志大小设置是否成功
    exit()
```

3.3.3.7 读系统时间

函数

cr_get_SystemDateTime(robotHandle)

简介

读控制柜系统日期时间。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CResult 类型的值，第二个是系统时间。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_get_SystemDateTime(robotHandle) #读系统时间
print("读系统时间结果: ",result[0].value)
if result[0].value != 0: #判断读系统时间是否成功
    exit()
print("系统时间为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3.8 读取机型型号

函数

```
cr_sys_robotModel_get(robotHandle,robotModel,len)
```

简介

读取机型型号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
robotModel	c_char_p	输出	机械臂型号
len	c_int	输入	机械臂型号数组长度

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是机型型号。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
robotModel = b'eee'
len = 10
result = cgxiapi.cr_sys_robotModel_get(robotHandle, robotModel, len) #读机型型号
if result[0].value != 0: #判断读机型型号是否成功
    exit()
print("返回结果: ", result[0].value)
print("机型型号: ", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3.9 读取控制器型号

函数

```
cr_get_controllerType(robotHandle,controllerType)
```

简介

读取控制器型号，返回码说明如下。

1-K20
2-MKE
3-K10
4-DK10
5-MK
6-MK40-H
7-MK300
8-MKCAN

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
controllerType	c_char_p	输出	控制器型号

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是机型型号。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
controllerType = b'eee'
result = cgxiapi.cr_get_controllerType(robotHandle, controllerType) #读控制器型号
if result[0].value != 0: #判断读控制器型号是否成功
    exit()
print("返回结果: ", result[0].value)
print("机型型号: ", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3.10 触发黑匣子数据

函数

```
cr_sys_blackBoxData_trigger(robotHandle)
```

简介

触发黑匣子数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_sys_blackBoxData_trigger(robotHandle) #触发黑匣子数据
print("返回结果:",result.value)
if result.value != 0: #判断触发黑匣子数据是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3.11 获取配置改变标志

函数

```
cr_cfg_change_code_get(robotHandle)
```

简介

获取配置改变标志位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是配置改变标志。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_cfg_change_code_get(robotHandle) #获取配置改变标志位
print("获取配置改变标志结果: ", result[0].value)
print("配置改变标志为: ", result[1].value)
if result[0].value != 0: #判断获取配置改变标志位是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.3.3.12 触发系统更新

函数

```
cr_sys_update_trigger(robotHandle)
```

简介

触发系统更新。错误码说明如下：

- 1-机械臂未断电
- 2-安全控制器 ECAT 未处于 OP 状态
- 3-通知安全控制器自更新失败
- 4-控制器写自更新标识失败
- 5-ECAT 网络故障
- 7-当前控制柜 boot 版本不支持软更新

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
```

```
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_sys_update_trigger(robotHandle)    #触发系统更新
print("触发系统更新结果: ", result[0].value)
print("errcode: ", result[1].value)
if result[0].value != 0:    #判断触发系统更新是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4 变量

3.4.1 数字量

3.4.1.1 读控制柜标准数字输出

函数

```
cr_get_stdDigitalOut(robotHandle,index)
```

简介

读控制柜标准数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-7）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CRresult 类型值，第二个是输出值，0-无效，1-有效。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
```

```

    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_stdDigitalOut(robotHandle,index)#读索引号为 0 的控制柜标准数字输出
print("读控制柜标准数字输出结果: ",result[0].value)
if result[0].value != 0:    #判断读控制柜标准数字输出是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.1.2 写控制柜标准数字输出

函数

cr_set_stdDigitalOut(robotHandle, index, val)

简介

写控制柜标准数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-7）
val	c_int	输入	待设定的值：0-无效，1-有效

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
val = 0
result=cgxiapi.cr_set_stdDigitalOut(robotHandle, index, val)
#写索引号为 1 的控制柜标准数字输出为 0
print("写控制柜标准数字输出结果: ",result.value)

```

```

if result.value != 0:    #判断写控制柜标准数字输出是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.1.3 读控制柜可配置数字输出

函数

cr_get_configDigitalOut(robotHandle, index)

简介

读控制柜可配置数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-7）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CResult 类型值，第二个是控制柜可配置数字输出值，0-无效，1-有效。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_configDigitalOut(robotHandle, index)
#读索引号为 0 的控制柜可配置数字输出
print("读控制柜可配置数字输出结果: ",result[0].value)
if result[0].value != 0:    #判断读控制柜可配置数字输出是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.1.4 读控制柜所有可配置数字输出

函数

```
cr_get_all_configDigitalOut(robotHandle, arrayVal, int arraySize)
```

简介

读控制柜所有可配置数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
arrayVal	c_BOOL	输出	待读取的值所存放的位置
arraySize	int	输入	读取的数量

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
arrayVal = [0, 0, 0, 0, 0, 0, 0, 0]
arraySize = 8
result = cgxiapi.cr_get_all_configDigitalOut(robotHandle, arrayVal, arraySize)
#读控制柜所有可配置数字输出
if result.value != 0: #判断读控制柜所有可配置数字输出是否成功
    exit()
for i in range(0, 8):
    print("读取结果: ", arrayVal[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.1.5 写控制柜可配置数字输出

函数

```
cr_set_configDigitalOut( robotHandle, index, val)
```

简介

写控制柜可配置数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-7）
val	c_int	输入	待设定的值：0-无效，1-有效

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
val = 0
result=cgxiapi.cr_set_configDigitalOut(robotHandle,index,val)
#写索引号为0的控制柜可配置数字输出为0
print("写控制柜可配置数字输出结果: ",result.value)
if result.value != 0: #判断写控制柜可配置数字输出是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.1.6 读工具端数字输出

函数

```
cr_get_toolDigitalOut(robotHandle, index)
```

简介

读工具端数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

index	c_int	输入	索引号（范围：0-1）
-------	-------	----	-------------

返回值

返回结果是长度为 2 的一维数组，第一个表示 CRresult 类型值，第二个是工具端数字输出值，0-无效，1-有效。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_toolDigitalOut(robotHandle, index) #读索引号为 0 的工具端数字输出
print("读工具端数字输出结果: ",result[0].value)
if result[0].value != 0: #判断读工具端数字输出是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.1.7 写工具端数字输出

函数

```
cr_set_toolDigitalOut(robotHandle, index, val)
```

简介

写工具端数字输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-1）
val	c_int	输入	待设定的值：0-无效，1-有效

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
val = 0
result=cgxiapi.cr_set_toolDigitalOut(robotHandle, index, val)
#写索引号为 0 的工具端数字输出为 0
print("写工具端数字输出结果: ",result.value)
if result.value != 0: #判断写工具端数字输出是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.1.8 读控制柜标准数字输入

函数

```
cr_get_stdDigitalIn(robotHandle, index)
```

简介

读控制柜标准数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-7）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CResult 类型值，第二个是控制柜标准数字输入值，0-无效，1-有效。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
```



```

    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_stdDigitalIn(robotHandle,index) #读索引号为 0 的控制柜标准数字输入
print("读控制柜标准数字输入结果: ",result[0].value)
if result[0].value != 0: #判断读控制柜标准数字输入是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.1.9 读控制柜可配置数字输入

函数

```
cr_get_configDigitalIn(robotHandle, index)
```

简介

读控制柜可配置数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-7）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CResult 类型值，第二个是控制柜可配置数字输入值，0-无效，1-有效。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_configDigitalIn(robotHandle, index)
#读索引号为 0 的控制柜可配置数字输入
print("读控制柜可配置数字输入结果: ",result[0].value)

```

```

if result[0].value != 0:    #判断读控制柜可配置数字输入是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.1.10 读控制柜所有可配置数字输入

函数

```
cr_get_all_configDigitalIn(robotHandle, arrayVal,int arraySize)
```

简介

读控制柜所有可配置数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
arrayVal	POINTER(BOOL)	输出	待读取的值所存放的位置
arraySize	int	输入	读取的数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
arrayVal = [0, 0, 0, 0, 0, 0, 0, 0]
arraySize = 8
result = cgxiapi.cr_get_all_configDigitalIn(robotHandle, arrayVal, arraySize)
#读控制柜所有可配置数字输入
if result.value != 0:    #判断读控制柜所有可配置数字输入是否成功
    exit()
for i in range(0, 8):
    print("读取结果: ", arrayVal[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功

```

```
print("断开失败")
exit()
```

3.4.1.11 读工具端数字输入

函数

```
cr_get_toolDigitalIn(robotHandle, index)
```

简介

读工具端数字输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-1）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CRresult 类型值，第二个是工具端数字输入值，0-无效，1-有效。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_toolDigitalIn(robotHandle, index) #读索引号为 0 的工具端数字输入
print("读工具端数字输入结果: ",result[0].value)
if result[0].value != 0: #判断读工具端数字输入是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.2 模拟量

3.4.2.1 读控制柜模拟量输出

函数

`cr_get_stdAnalogOut(robotHandle, index)`

简介

读控制柜模拟量输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CResult 类型值，第二个是控制柜模拟量输出值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_stdAnalogOut(robotHandle,index) #读索引号为 0 的控制柜模拟量输出
print("读控制柜模拟量输出结果: ",result[0].value)
if result[0].value != 0: #判断读控制柜模拟量输出是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.2.2 写控制柜模拟量输出

函数

`cr_set_stdAnalogOut(robotHandle, index, val)`

简介

写控制柜模拟量输出。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0）
val	c_int	输入	待设定的值

返回值

数据类型为 CResult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
val = 4
result=cgxiapi.cr_set_stdAnalogOut(robotHandle, index, val)
#写索引号为 0 的控制柜模拟量输出为 4
print("写控制柜标准数字输出结果: ",result.value)
if result.value != 0: #判断写控制柜模拟量输出是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.2.3 读工具端模拟量输入

函数

```
cr_get_toolAnalogIn(robotHandle, index)
```

简介

读工具端模拟量输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-1）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CResult 类型值，第二个是工具端模拟量输入值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_toolAnalogIn(robotHandle, index) #读索引号为 0 的工具端模拟量输入
print("读工具端模拟量输入结果: ",result[0].value)
if result[0].value != 0: #判断读工具端模拟量输入是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.2.4 读控制柜模拟量输入

函数

```
cr_get_stdAnalogIn(robotHandle, index)
```

简介

读控制柜模拟量输入。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引号（范围：0-2）

返回值

返回结果是长度为 2 的一维数组，第一个表示 CResult 类型值，第二个是控制柜模拟量输入值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result=cgxiapi.cr_get_stdAnalogIn(robotHandle, index) #读索引号为 0 的控制柜模拟量输入
print("读控制柜模拟量输入结果: ",result[0].value)
if result[0].value != 0:    #判断读控制柜模拟量输入是否成功
    exit()
print("值为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.2.5 读机械臂控制柜和工具端上所有输入数字量和模拟量

函数

```
cr_get_allDAInput(robotHandle, controllerDI, controllerCI, controllerAI, toolDI, toolAI)
```

简介

读机械臂控制柜和工具端上所有输入数字量和模拟量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
controllerDI	c_int * 8	输出	控制柜标准数字输入量
controllerCI	c_int * 8	输出	控制柜可配置数字输入量
controllerAI	c_double * 3	输出	控制柜模拟输入量
toolDI	c_int * 2	输出	工具端标准数字输入量
toolAI	c_double * 2	输出	工具端模拟输入量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")

```

```

    exit()
robotHandle = re1[1].value
controllerDI = [0,0,0,0,0,0,0,0]
controllerCI = [0,0,0,0,0,0,0,0]
controllerAI = [0,0,0]
toolDI = [0,0]
toolAI = [0,0]
result=cgxiapi.cr_get_allDAInput(robotHandle, controllerDI, controllerCI, controllerAI, toolDI,
    toolAI) #读机械臂控制柜和工具端上所有输入数字量和模拟量
print("读机械臂控制柜和工具端上所有输入数字量和模拟量结果：",result.value)
if result.value != 0: #判断读机械臂控制柜和工具端上所有输入数字量和模拟量是否成功
    exit()
i = 0
while i<8:
    print(controllerDI[i])
    print(controllerCI[i])
    i += 1
i=0
while i<3:
    print(controllerAI[i])
    i += 1
i = 0
while i < 2:
    print(toolDI[i])
    print(toolAI[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.2.6 读机械臂控制柜和工具端上所有输出数字量和模拟量

函数

```
cr_get_allDAOOutput(robotHandle, controllerDO, controllerCO, controllerAO, toolDO)
```

简介

读机械臂控制柜和工具端上所有输出数字量和模拟量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
controllerDO	c_int * 8	输出	控制柜标准数字输出量
controllerCO	c_int * 8	输出	控制柜可配置数字输出量

controllerAO	c_double * 3	输出	控制柜模拟输出量
toolDO	c_int * 2	输出	工具端标准数字输出量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
controllerDO = [0,0,0,0,0,0,0,0]
controllerCO = [0,0,0,0,0,0,0,0]
controllerAO = [0,0,0]
toolDO= [0,0]
result=cgxiapi.cr_get_allDAOOutput(robotHandle, controllerDO, controllerCO, controllerAO, toolDO) #读机械臂控制柜和工具端上所有输出数字量和模拟量
print("读机械臂控制柜和工具端上所有输出数字量和模拟量结果: ",result.value)
if result.value != 0: #判断读机械臂控制柜和工具端上所有输出数字量和模拟量是否成功
    exit()
i = 0
while i<8:
    print(controllerDO[i])
    print(controllerCO[i])
    i += 1
print(controllerAO[1])
i = 0
while i < 2:
    print(toolDO[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3 通讯变量

3.4.3.1 读 bool 寄存器

函数

`cr_get_boolRegValue(robotHandle, startAddress,offsetLength)`

简介

读 bool 寄存器。单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 100 - 起始地址）

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值为寄存器的值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=2
result=cgxiapi.cr_get_boolRegValue(robotHandle,startAddress,offsetLength)
#读地址为 1~2 的 bool 寄存器
print("读位寄存器结果: ",result[0].value)
if result[0].value != 0: #判断读 bool 寄存器是否成功
    exit()
i = 0
while i < 2:
    print(result[1][i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3.2 写 bool 寄存器

函数

```
cr_set_boolRegValue(robotHandle, startAddress, offsetLength, regData)
```

简介

写 bool 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 1000 - 起始地址）
regData	POINTER(c_int)	输入	寄存器数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=5
regData = [0,0,1,0,0]
result=cgxiapi.cr_set_boolRegValue(robotHandle, startAddress, offsetLength, regData)
#写地址 1~5 的 bool 寄存器
print("写位寄存器结果: ",result.value)
if result.value != 0: #判断写 bool 寄存器是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3.3 读 int16 寄存器

函数

`cr_get_int16RegValue(robotHandle, startAddress,offsetLength)`

简介

读 int16 寄存器。单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 100 - 起始地址）

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值为寄存器的值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=2
result=cgxiapi.cr_get_int16RegValue(robotHandle, startAddress,offsetLength)
#读地址 1~2 的 int16 寄存器
print("读 int16 寄存器结果: ",result[0].value)
if result[0].value != 0: #判断读 int16 寄存器是否成功
    exit()
i = 0
while i < 2:
    print(result[1][i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3.4 写 int16 寄存器

函数

`cr_set_int16RegValue(robotHandle, startAddress, offsetLength, regData)`

简介

写 int16 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 1000 - 起始地址）
regData	POINTER(c_int)	输入	寄存器数据（范围：-32768-32767）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=5
regData = [0,1,0,0,0]
result=cgxiapi.cr_set_int16RegValue(robotHandle, startAddress, offsetLength, regData)
#写地址 1~5 的 int16 寄存器
print("写 int16 寄存器结果: ",result.value)
if result.value != 0: #判断写 int16 寄存器是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3.5 读 int32 寄存器

函数

`cr_get_int32RegValue(robotHandle, startAddress, offsetLength)`

简介

读 int32 寄存器。单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 100 - 起始地址）

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值为寄存器的值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=2
result=cgxiapi.cr_get_int32RegValue(robotHandle, startAddress,offsetLength)
#读地址为 1~2 的 int32 寄存器
print("读 int32 寄存器结果: ",result[0].value)
if result[0].value != 0: #判断读 int32 寄存器是否成功
    exit()
i = 0
while i < 2:
    print(result[1][i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3.6 写 int32 寄存器

函数

```
cr_set_int32RegValue(robotHandle, startAddress, offsetLength, regData)
```

简介

写 int32 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 1000 - 起始地址）
regData	POINTER(c_int)	输入	寄存器数据（范围：-2147483648-2147483647）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=5
regData = [1,0,0,0,0]
result=cgxiapi.cr_set_int32RegValue(robotHandle, startAddress, offsetLength, regData)
#写地址为 1~5 的 int32 寄存器
print("写 int32 寄存器结果: ",result.value)
if result.value != 0: #判断写 int32 寄存器是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.3.7 读 float 寄存器

函数

`cr_get_floatRegValue(robotHandle, startAddress, offsetLength)`

简介

读 float 寄存器。单次最多读取 100 个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 100 - 起始地址）

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值为寄存器的值。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=2
result=cgxiapi.cr_get_floatRegValue(robotHandle, startAddress, offsetLength)
#读地址 1~2 的 float 寄存器
print("读 float 寄存器结果: ",result[0].value)
if result[0].value != 0: #判断读 float 寄存器是否成功
    exit()
i = 0
while i < 2:
    print(result[1][i])
    i = i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```


3.4.3.8 写 float 寄存器

函数

```
cr_set_floatRegValue(robotHandle, startAddress, offsetLength, regData)
```

简介

写 float 寄存器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
startAddress	c_uint	输入	寄存器起始地址（范围：0-999）
offsetLength	c_uint	输入	寄存器偏移地址（范围：小于或等于 1000 - 起始地址）
regData	POINTER(c_float)	输入	寄存器数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
startAddress=1
offsetLength=5
regData = [0,0,0,0,0]
result=cgxiapi.cr_set_floatRegValue(robotHandle, startAddress, offsetLength, regData)
#写地址为 1~5 的 float 寄存器
print("写 float 寄存器结果: ",result.value)
if result.value != 0: #判断写 float 寄存器是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.4 脚本变量

3.4.4.1 读安装变量

函数

```
cr_get_intallVarValue(robotHandle, installVarName, installVar)
```

简介

读安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
installVarName	c_char_p	输入	需读取安装变量名称
installVar	POINTER(VariableMsg)	输出	读取的安装变量值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
installVarName="i_var_1"
installVar=basestruct.VariableMsg()
result=cgxiapi.cr_get_intallVarValue(robotHandle, installVarName, installVar)
#读名为 i_var_1 的安装变量的值
print("读安装变量结果: ",result.value)
if result.value != 0: #判断读安装变量是否成功
    exit()
print("变量类型: ",installVar.variableType.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.4.2 修改安装变量

函数

```
cr_set_installVarValue(robotHandle, installVarName, installVar)
```

简介

修改安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
installVarName	c_char_p	输入	需读取安装变量名称
installVar	POINTER(VariableMsg)	输入	需赋值的安装变量值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
installVarName="i_var_1"
installVar=basestruct.VariableMsg()
installVar.variableType = 3
installVar.numberValue = 1
installVar.variableName = b"i_var_2"
result=cgxiapi.cr_set_installVarValue(robotHandle, installVarName, installVar)
#修改名为 i_var_1 的安装变量的值
print("修改安装变量结果：",result.value)
if result.value != 0: #判断修改安装变量是否成功
    exit()
print("变量类型：",installVar.variableType.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.4.3 读取程序变量数据

函数

```
cr_script_var_get(robotHandle, varName, variableMsg)
```

简介

读取程序变量数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
varName	c_char_p	输入	程序变量名
variableMsg	POINTER(VariableMsg)	输出	程序变量数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
varName = b'var_1'
variableMsg = basestruct.VariableMsg()
result=cgxiapi.cr_script_var_get(robotHandle, varName, variableMsg)
#读取名为 var_1 的程序变量数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取程序变量数据是否成功
    exit()
print("读取结果: ",variableMsg.variableType.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.4.4 修改程序变量数据

函数

```
cr_script_var_set(robotHandle, variableMsg)
```

简介

修改程序变量数据。该接口只在程序运行或暂停时有效。使用该接口时需保证修改的值的类型需要与程序中的变量类型对应。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

variableMsg	POINTER(VariableMsg)	输入	程序变量数据
-------------	--	----	--------

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
variableMsg = basestruct.VariableMsg()
variableMsg.variableType = 3
variableMsg.variableName = b'var_1'
variableMsg.numberValue = 100
result = cgxiapi.cr_script_var_set(robotHandle, variableMsg)
#修改名称为 var_1 程序变量数据
print("返回结果: ", result.value)
if result.value != 0: #判断修改程序变量数据是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.5 其它 IO

3.4.5.1 获取编码器计数数量

函数

```
cr_get_encoderTickCnt(robotHandle, encoderChannel)
```

简介

获取编码器计数数量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
encoderChannel	int	输入	编码器通道
encoderTickCount	long long*	输出	编码器计数数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
encoderChannel = 0
result = cgxiapi.cr_get_encoderTickCnt(robotHandle, encoderChannel)
#获取编码器通道 0 的计数数量
print("返回结果: ", result[0].value)
if result[0].value != 0: #判断获取编码器通道计数数量是否成功
    exit()
print("编码器计数数量: ", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.4.5.2 设置监控界面的工具输出电压

函数

```
cr_set_ToolOutputVoltage(robotHandle, toolPower)
```

简介

设置监控界面的工具输出电压。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
toolPower	ToolPower	输入	工具输出电压

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
toolPower = basestruct.ToolPower.Power_on
result = cgxiapi.cr_set_ToolOutputVoltage(robotHandle, toolPower)
#设置监控界面的工具输出电压
print("返回结果: ", result.value)
if result.value != 0:    #判断设置监控界面的工具输出电压是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.4.5.3 读取监控界面的工具输出电压

函数

cr_get_ToolOutputVoltage(robotHandle, toolPower)

简介

读取监控界面的工具输出电压。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
toolPower	POINTER(ToolPower)	输出	工具输出电压

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
toolPower = basestruct.ToolPower()
result = cgxiapi.cr_get_ToolOutputVoltage(robotHandle, toolPower)
#读取监控界面的工具输出电压

```

```

print("返回结果：", result.value)
if result.value != 0:    #判断读取监控界面的工具输出电压是否成功
    exit()
print("监控界面的工具输出电压：", toolPower.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.5 数学模块

3.5.1 位姿变换

3.5.1.1 位姿转换

函数

cr_poseTrans(poseFrom, poseTrans, poseTo)

简介

位姿转换,位姿整体右乘叠加计算。

参数

参数名称	数据类型	输入/输出	描述
poseFrom	c_double*6	输入	原位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
poseTrans	c_double*6	输入	变换位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）
poseTo	c_double*6	输出	结果位姿：x, y, z（单位：mm）； Rx, Ry, Rz（单位：°）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

poseFrom=[0,0,0,10,10,10]
poseTrans=[1,1,1,20,20,20]
poseTo=[0,0,0,0,0,0]
result=cgxiapi.cr_poseTrans(poseFrom, poseTrans, poseTo)    #位姿转换
print("位姿转换结果：",result.value,poseTo)
if result.value != 0:    #判断位姿转换是否成功
    exit()

```


3.5.1.2 位姿求逆

函数

```
cr_compute_pose_inv(pose, poseLen, poseInv, poseInvLen)
```

简介

位姿求逆。

参数

参数名称	数据类型	输入/输出	描述
pose	c_double*6	输入	轴角位姿数据：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
poseLen	c_int	输入	位姿长度，至少为 6
poseInv	c_double*6	输出	位姿的逆：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
poseInvLen	c_int	输入	位姿长度，至少为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
poseLen = 6
poseInvLen = 6
pose = [-66, -438, 887, -57, 0, -148]
poseInv = [0, 0, 0, 0, 0, 0]
result = cgxiapi.cr_compute_pose_inv(pose, poseLen, poseInv, poseInvLen) #位姿求逆
print("返回结果：", result.value)
if result.value != 0: #判断位姿求逆是否成功
    exit()
print("计算结果：")
i = 0
while i < 6:
    print(poseInv[i])
    i += 1
```

3.5.1.3 轴角转欧拉角

函数

```
cr_AxisAngle2Eule(axisAnglePose, eulePose)
```

简介

位姿变换，轴角转欧拉角。

参数

参数名称	数据类型	输入/输出	描述
axisAnglePose	c_double*6	输入	轴角位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
eulePose	POINTER(c_double)	输出	欧拉角位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
axisAnglePose=[1,1,1,20,20,20]
eulePose=[0,0,0,0,0,0]
result=cgxiapi.cr_AxisAngle2Eule(axisAnglePose, eulePose) #轴角转欧拉角
print("轴角转欧拉角: ",result.value)
if result.value != 0: #判断轴角转欧拉角是否成功
    exit()
print(eulePose)
```

3.5.1.4 欧拉角转轴角

函数

```
cr_Eule2AxisAngle(eulePose, axisAnglePose)
```

简介

位姿变换, 欧拉角转轴角。

参数

参数名称	数据类型	输入/输出	描述
eulePose	c_double*6	输入	欧拉角位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
axisAnglePose	POINTER(c_double)	输出	轴角位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
eulePose=[1,1,1,20,20,20]
axisAnglePose=[0,0,0,0,0,0]
result=cgxiapi.cr_Eule2AxisAngle(eulePose, axisAnglePose) #欧拉角转轴角
print("位姿变换: ",result.value)
if result.value != 0: #判断欧拉角转轴角是否成功
```

```
exit()
print(axisAnglePose)
```

3.5.1.5 TCP 位姿转法兰位姿

函数

```
cr_TcpToFlangePose(tcpPose, toolPose, flangePose)
```

简介

TCP 位姿转法兰位姿。

参数

参数名称	数据类型	输入/输出	描述
tcpPose	c_double *6	输入	tcp 位姿在用户坐标系下的描述: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
toolPose	c_double *6	输入	当前使用的 tcp: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
flangePose	c_double *6	输出	法兰在基座坐标系下的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
tcpPose=[1,1,1,20,20,20]
toolPose=[14,14,14,10,10,10]
flangePose=[0,0,0,0,0,0]
result=cgxiapi.cr_TcpToFlangePose(tcpPose, toolPose, flangePose) #TCP 位姿转法兰位姿
print("位姿变换: ",result.value)
if result.value != 0: #判断 TCP 位姿转法兰位姿是否成功
    exit()
print(flangePose)
```

3.5.1.6 基坐标系下位姿转移到用户坐标系下

函数

```
cr_compute_pose_base_to_user(basePose, basePoseLen, userPose, userPoseLen, poseInUser,
poseInUserLen)
```

简介

基坐标系下位姿转移到用户坐标系下。

参数

参数名称	数据类型	输入/输出	描述
basePose	POINTER(c_double)	输入	轴角位姿数据: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
basePoseLen	c_int	输入	位姿长度, 至少为 6
userPose	POINTER(c_double)	输入	用户坐标系位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
userPoseLen	c_int	输入	位姿长度, 至少为 6
poseInUser	POINTER(c_double)	输出	用户坐标系下的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
poseInUserLen	c_int	输入	位姿长度, 至少为 6

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
basePoseLen = 6
userPoseLen = 6
poseInUserLen = 6
basePose = [-66, -438, 887, -57, 0, -148]
userPose = [1, 2, 3, 10, 10, 10]
poseInUser = [0, 0, 0, 0, 0, 0]
result=cgxiapi.cr_compute_pose_base_to_user(basePose, basePoseLen, userPose, userPoseLen,
poseInUser,poseInUserLen) #目标点旋转变换
print("返回结果: ", result.value)
if result.value != 0: #判断目标点旋转变换是否成功
    exit()
print("读取结果: ")
i = 0
while i < 6:
    print(poseInUser[i])
    i += 1
```

3.5.1.7 目标点旋转变换

函数

```
cr_compute_rotate_around_axis(axisPosition, axisDirection, angle, srcPose, dstPose)
```

简介

目标点旋转变换。

参数

参数名称	数据类型	输入/输出	描述
axisPosition	POINTER(c_double)	输入	旋转轴位置(x,y,z (单位: mm))
axisDirection	POINTER(c_double)	输入	旋转轴方向(x,y,z (单位: mm))
angle	POINTER(c_double)	输入	旋转角度 (°)
srcPose	POINTER(c_double)	输入	原始目标位姿(x,y,z (单位: mm); Rx,Ry,Rz (单位: °))
dstPose	POINTER(c_double)	输出	旋转后目标位姿(x,y,z (单位: mm); Rx,Ry,Rz (单位: °))

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
axisPosition = (c_double * 3)(0, 0, 0)
axisDirection = (c_double * 3)(0, 0, 100)
angle = c_double(180)
srcPose = (c_double * 6)(-87, -487.64, 772, 138, -25, 25)
dstPose = (c_double * 6)(0, 0, 0, 0, 0, 0)
result = cgxiapi.cr_compute_rotate_around_axis(axisPosition, axisDirection, angle, srcPose, dstPose) #基坐标系下位姿转移到用户坐标系下
print("返回结果: ", result.value)
if result.value != 0: #判断基坐标系下位姿转移到用户坐标系下是否成功
    exit()
print("读取结果: ")
i = 0
while i < 6:
    print(dstPose[i])
    i += 1
```

3.5.2 运动学

3.5.2.1 正解

函数

```
cr_kineForward(robotHandle, srcJointPose, tarPose)
```

简介

运动学正解。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
srcJointPose	c_double*6	输入	关节空间位置：关节 1~6（单位：°）
tarPose	c_double*6	输出	笛卡尔空间位姿计算结果：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ", re1.value)
robotHandle = re1[1].value
tarPose = [0, 0, 0, 0, 0, 0]
srcJointPose = [60, 60, -60, 60, 60, 60]
result = cgxiapi.cr_kineForward(robotHandle, srcJointPose, tarPose) #正解
if result.value != 0: #判断正解是否成功
    exit()
print("正解结果:", tarPose[0], tarPose[1], tarPose[2], tarPose[3],
      tarPose[4], tarPose[5])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.5.2.2 逆解

函数

```
cr_kineInverse(robotHandle, srcPose, refJointPose, tarJointPose)
```

简介

运动学逆解。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
srcPose	c_double*6	输入	笛卡尔空间位姿值：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
refJointPose	c_double*6	输入	逆解计算用的参考关节空间位置：关

			节 1~6 (单位: °)
tarJointPose	c_double*6	输出	计算成功时关节空间位置计算结果: 关节 1~6 (单位: °)

返回值

数据类型为 CResult, 若为 success 或 0, 则表示成功其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
tarJointPose = [0, 0, 0, 0, 0, 0]
srcPose = [-66.01,-438.24,887.68,-57.34,0,-148.98]
refJointPose = [0, 0, 0, 0, 0, 0]
result = cgxiapi.cr_kineInverse(robotHandle, srcPose, refJointPose, tarJointPose) #逆解
if result.value != 0: #判断逆解是否成功
    exit()
print("逆解结果: ", tarJointPose[0], tarJointPose[1], tarJointPose[2], tarJointPose[3], tarJointPose[4],tarJointPose[5])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接, 释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6 配置

3.6.1 安装

3.6.1.1 设置安装角度

函数

```
cr_cfg_install_angle_set(robotHandle, installAngle)
```

简介

设置安装角度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
installAngle	InstallAngle	输入	安装角度设置 (基座旋转角度范围: 0-360 基座倾斜角度范围: 0-180)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
installAngle = basestruct.InstallAngle()
installAngle.tiltAngle = 90
installAngle.baseAngle = 90
result=cgxiapi.cr_cfg_install_angle_set(robotHandle, installAngle) #设置安装角度
if result.value != 0: #判断设置安装角度是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.1.2 读取安装角度

函数

```
cr_cfg_install_angle_get(robotHandle, installAngle)
```

简介

读取安装角度。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
installAngle	POINTER(InstallAngle)	输出	安装角度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
installAngle = basestruct.InstallAngle()
result=cgxiapi.cr_cfg_install_angle_get(robotHandle, installAngle) #读取安装角度
print("返回结果: ",result.value)
if result.value != 0: #判断读取安装角度是否成功
    exit()
print("安装角度为: ",installAngle.tiltAngle,installAngle.baseAngle)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.2 TCP

3.6.2.1 读取 TCP 个数

函数

```
cr_cfg_tcp_count(robotHandle)
```

简介

读取 TCP 个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取 tcp 个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_tcp_count(robotHandle)  #读取 TCP 个数
print("返回结果: ",result[0].value)
if result[0].value != 0:    #判断读取 TCP 个数是否成功
    exit()
print("tcp 个数为: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.2.2 读取 TCP 数据

函数

cr_cfg_tcp_get(robotHandle, index, tcpMsg)

简介

读取 TCP 数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	tcp 索引（范围：0-19）
tcpMsg	POINTER(TCPMsg)	输出	tcp 数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
tcpMsg = basestruct.TCPMsg()
result=cgxiapi.cr_cfg_tcp_get(robotHandle, index, tcpMsg)  #读取索引号为 1 的 TCP 数据

```

```

print("返回结果: ",result.value)
if result.value != 0:    #判断读取 TCP 数据是否成功
    exit()
    print("tcp 信息:",tcpMsg.tcpName)
for i in range(0,6):
    print(tcpMsg.tcpOffset[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.2.3 读取 TCP 数据(通过名称)

函数

```
cr_cfg_tcp_accord_name_get(robotHandle, tcpName, tcpMsg)
```

简介

读取 TCP 数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
tcpName	c_char_p	输入	tcp 名称
tcpMsg	POINTER(TCPMsg)	输出	tcp 数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
tcpName = b"TCP_1"
tcpMsg = basestruct.TCPMsg()
result=cgxiapi.cr_cfg_tcp_accord_name_get(robotHandle, tcpName, tcpMsg)    #读取 TCP 数据
print("返回结果: ",result.value)
if result.value != 0:    #判断读取 TCP 数据是否成功
    exit()
for i in range(0,6):

```

```

    print(tcpMsg.tcpOffset[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.2.4 增加 TCP

函数

```
cr_cfg_tcp_add(robotHandle, tcpMsg)
```

简介

增加 TCP。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
tcpMsg	TCPMsg	输入	tcp 数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
tcpMsg = basestruct.TCPMsg()
tcpMsg.tcpName = b's'
tcpMsg.tcpOffset[0] = 2
result=cgxiapi.cr_cfg_tcp_add(robotHandle, tcpMsg)  #增加 TCP
print("返回结果: ",result.value)
if result.value != 0:  #判断增加 TCP 是否成功
    exit()
print("tcp 信息:",tcpMsg.tcpName,tcpMsg.tcpOffset[0])
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.2.5 删除 TCP

函数

```
cr_cfg_tcp_delete(robotHandle, index)
```

简介

删除 TCP。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	tcp 索引（范围：0-19）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 2
result=cgxiapi.cr_cfg_tcp_delete(robotHandle, index) #删除索引号为2的TCP
if result.value != 0: #判断删除TCP是否成功
    exit()
print("返回结果: ",result.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.2.6 编辑 TCP 数据

函数

```
cr_cfg_tcp_set(robotHandle, index, tcpMsg)
```

简介

编辑 TCP 数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	tcp 索引（范围：0-19）
tcpMsg	TCPMsg	输入	tcp 数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
tcpMsg = basestruct.TCPMsg()
tcpMsg.tcpName = b'a'
tcpMsg.tcpOffset[0] = 6
index = 2
result=cgxiapi.cr_cfg_tcp_set(robotHandle, index, tcpMsg) #编辑索引号为2的TCP数据
print("返回结果：",result.value)
if result.value != 0: #判断编辑TCP数据是否成功
    exit()
print("tcp 信息:",tcpMsg.tcpName,tcpMsg.tcpOffset[0])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.2.7 读取当前激活的 TCP 索引

函数

```
cr_cfg_tcp_active_get(robotHandle)
```

简介

读取当前激活的 TCP 索引。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回激活的 TCP 索引。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_tcp_active_get(robotHandle) #读取当前激活的 TCP 索引
print("返回结果: ",result[0].value,"激活的 TCP 索引: ",result[1].value)
if result[0].value != 0: #判断读取当前激活的 TCP 索引是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.2.8 激活 TCP

函数

```
cr_cfg_tcp_active_set(robotHandle, index)
```

简介

激活 TCP。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	tcp 索引（范围：0-19）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
```

```

    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
result=cgxiapi.cr_cfg_tcp_active_set(robotHandle, index) #激活索引号为 1 的 TCP
print("返回结果：",result.value)
if result.value != 0: #判断激活 TCP 是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.2.9 计算 TCP 位置

函数

```
cr_compute_tcp_position(poses, posesLen, position, positionLen)
```

简介

计算 TCP 位置。

参数

参数名称	数据类型	输入/输出	描述
poses	POINTER(c_double *6)	输入	多组位姿数据，每组位姿数据都是轴角：x, y, z（单位：mm）；Rx, Ry, Rz（单位：°）
posesLen	c_int	输入	位姿数量，至少为 4
positionLen	c_int	输入	xyz 的长度，至少为 3
position	POINTER(c_double)	输出	tcp 的 xyz 值：x, y, z（单位：mm）

返回值

返回结果为长度为 2 的一维数组，第一个表示接口调用结果，第二个值为位置误差。

示例

```

poses = [[4.31, -66.66, 934.53, 20.30, -121.50, -73.277],
          [-3.71, -68.75, 926.57, 12.589, -124.54, -77.42],
          [13.98, -65.98, 934.53, 20.30, -121.5, -73.27],
          [4.2558, -66.66, 934.53, 8.77, -107.15, -71.457]]
poseLen = 4
position = [0, 0, 0]
positionLen = 3

```



```

result = cgxiapi.cr_compute_tcp_position(poses, poseLen, position, positionLen)
#计算 TCP 位置
print("返回结果: ", result[0].value)
if result[0].value != 0:    #判断计算 TCP 位置是否成功
    exit()
print("计算结果:")
i = 0
while i < 3:
    print(position[i])
    i += 1
print("误差为: ",result[1].value)

```

3.6.2.10 计算 TCP 角度

函数

```
cr_compute_tcp_orientation(csPose, csLen, pointPose, pointLen, orientation, orientationLen)
```

简介

计算 TCP 角度。

参数

参数名称	数据类型	输入/输出	描述
csPose	POINTER(c_double)	输入	参考坐标系的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
csLen	c_int	输入	参考坐标系位姿的长度, 至少为 6
pointPose	POINTER(c_double)	输入	任意一点的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
pointLen	c_int	输入	点位姿的长度, 至少为 6
orientationLen	c_int	输入	rxxyz 的长度, 至少为 3
orientation	POINTER(c_double)	输出	tcp 的 rxxyz 值: Rx, Ry, Rz (单位: °)

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```

csPose = [0, 0, 0, 0, 0, 0]
pointPose = [-36.10, -4.42, 32.62, 13.54, -91.33, 39.52]
orientation = [0,0,0]
csLen = 6
pointLen = 6
orientationLen = 3
result=cgxiapi.cr_compute_tcp_orientation(csPose, csLen, pointPose, pointLen, orientation, ori
entationLen) #计算 TCP 角度

```

```
print("返回结果：",result.value)
if result.value != 0:    #判断计算 TCP 角度是否成功
    exit()
i = 0
while i<3:
    print("计算结果：", orientation[i])
    i += 1
```

3.6.3 负载

3.6.3.1 读取负载个数

函数

cr_cfg_payload_count(robotHandle)

简介

读取负载个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取负载个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_payload_count(robotHandle) #读取负载个数
print("返回结果：",result[0].value,"负载个数为：",result[1].value)
if result[0].value != 0:    #判断读取负载个数是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.3.2 读取负载数据

函数

```
cr_cfg_payload_get(robotHandle, index, payLoad)
```

简介

读取负载数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	负载索引（范围：0-19）
payLoad	POINTER(PayLoad)	输出	负载信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
payLoad = basestruct.PayLoad()
result=cgxiapi.cr_cfg_payload_get(robotHandle, index, payLoad)#读取索引号为1的负载数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取负载数据是否成功
    exit()
print("负载数据: ",payLoad.toolPayload)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.3.3 读取负载数据(通过名称)

函数

```
cr_cfg_payload_accord_name_get( robotHandle, payloadName, payLoad)
```

简介

读取负载数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
payloadName	c_char_p	输入	负载名称
payLoad	POINTER(PayLoad)	输出	负载信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
payloadName= b"Payload_1"
payLoad = basestruct.Payload()
result=cgxiapi.cr_cfg_payload_accord_name_get(robotHandle, payloadName, payLoad)#读取负载数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取负载数据是否成功
    exit()
print("负载数据: ",payLoad.toolPayload)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.3.4 增加负载

函数

```
cr_cfg_payload_add(robotHandle, payLoad)
```

简介

增加负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
payLoad	PayLoad	输入	负载信息

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
payLoad = basestruct.PayLoad()
payLoad.payloadName = b'aa'
payLoad.centerOfGravity[0] = 10
result=cgxiapi.cr_cfg_payload_add(robotHandle, payLoad) #增加负载
print("返回结果: ",result.value)
if result.value != 0: #判断增加负载是否成功
    exit()
print("负载数据: ",payLoad.toolPayload,payLoad.payloadName)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.3.5 删除负载

函数

```
cr_cfg_payload_delete(robotHandle, index)
```

简介

删除负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	负载索引（范围：0-19）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 3
result=cgxiapi.cr_cfg_payload_delete(robotHandle, index) #删除索引号为3的负载
print("返回结果: ",result.value)
if result.value != 0: #判断删除负载是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.3.6 编辑负载

函数

```
cr_cfg_payload_set(robotHandle, index, payLoad)
```

简介

编辑负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	负载索引（范围：0-19）
payLoad	PayLoad	输入	负载信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
```

```

if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
payLoad = basestruct.PayLoad()
payLoad.payloadName = b'cc'
payLoad.toolPayload = 3
payLoad.centerOfGravity[0] = 10
payLoad.centerOfGravity[1] = 20
index = 3
result=cgxiapi.cr_cfg_payload_set(robotHandle, index, payLoad) #编辑索引号为3的负载
print("返回结果: ",result.value)
if result.value != 0:    #判断编辑负载是否成功
    exit()
print("负载数据: ",payLoad.toolPayload,payLoad.payloadName)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.3.7 读取当前激活的负载索引

函数

```
cr_cfg_payload_active_get(robotHandle)
```

简介

读取当前激活的负载索引。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。同时返回激活的负载索引。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
result=cgxiapi.cr_cfg_payload_active_get(robotHandle) #读取当前激活的负载索引
print("返回结果: ",result[0].value,"激活的负载索引: ",result[1].value)
if result[0].value != 0: #判断读取当前激活的负载索引是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.3.8 激活负载

函数

cr_cfg_payload_active_set(robotHandle, index)

简介

激活负载。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	负载索引（范围：0-19）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 3
result=cgxiapi.cr_cfg_payload_active_set(robotHandle, index) #激活索引号为3的负载
print("返回结果: ",result.value)
if result.value != 0: #判断激活负载是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```


3.6.4 初始位/包装位

3.6.4.1 设置初始位

函数

```
cr_cfg_home_pose_set(robotHandle, homePose, len)
```

简介

设置初始位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
homePose	POINTER(c_double)	输入	初始位姿数据
len	c_int	输入	数组长度,长度为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
homePose = [10,10,10,10,10,10]
result=cgxiapi.cr_cfg_home_pose_set(robotHandle, homePose, 6) #设置初始位
print("返回结果: ",result.value)
if result.value != 0: #判断设置初始位是否成功
    exit()
i=0
while(i<6):
    print(homePose[i])
    i=i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.4.2 读取初始位

函数

```
cr_cfg_home_pose_get(robotHandle, homePose, len)
```

简介

读取初始位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
homePose	POINTER(c_double)	输出	初始位姿数据
len	c_int	输入	数组长度,长度为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
homePose = [10,10,10,10,10,10]
len = 6
result=cgxiapi.cr_cfg_home_pose_get(robotHandle, homePose, len) #读取初始位
print("返回结果: ",result.value)
if result.value != 0: #判断读取初始位是否成功
    exit()
i=0
while(i<6):
    print(homePose[i])
    i=i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.4.3 删除初始位

函数

```
cr_cfg_home_pose_delete(robotHandle)
```

简介

删除初始位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_home_pose_delete(robotHandle) #删除初始位
print("返回结果: ",result.value)
if result.value != 0: #判断删除初始位是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.4.4 读取包装位

函数

```
cr_cfg_pack_pose_get(robotHandle, packPose, len)
```

简介

读取包装位。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

packPose	POINTER(c_double)	输出	包装位数据
len	c_int	输入	数组长度, 长度为 6

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功, 其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
packPose = [0,0,0,0,0,0]
result=cgxiapi.cr_cfg_pack_pose_get(robotHandle, packPose, 6) #读取包装位
print("返回结果: ",result.value)
if result.value != 0: #判断读取包装位是否成功
    exit()
i=0
while(i<6):
    print(packPose[i])
    i=i+1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接, 释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.5 启动

3.6.5.1 设置机器人是否自动上电使能

函数

```
cr_cfg_poweron_auto_set(robotHandle, isAuto)
```

简介

设置机器人是否自动上电使能。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
isAuto	c_int	输入	是否自动上电 0-不自动上电, 1-自动上电

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
isAuto = 1
result=cgxiapi.cr_cfg_poweron_auto_set(robotHandle, isAuto)
#设置机器人自动上电使能
if result.value != 0: #判断设置机器人自动上电使能是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.5.2 读取机器人自动上电状态

函数

```
cr_cfg_poweron_auto_get(robotHandle)
```

简介

读取机器人自动上电状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取结果，0-否，1-是。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
```

```

    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_poweron_auto_get(robotHandle) #读取机器人自动上电状态
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取机器人自动上电状态是否成功
    exit()
print("读取结果: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.6 控制参数

3.6.6.1 设置机器人拖动阻尼

函数

```
cr_cfg_joint_drag_damping_set(robotHandle,jointDargDamping,arrSize)
```

简介

设置机器人拖动阻尼。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
jointDargDamping	POINTER(c_int)	输入	机器人拖动阻尼数组 (关节 1-6)
arrSize	c_int	输入	机器人拖动阻尼数组 大小(一般为 6)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
jointDargDamping = (c_int * 6)(50, 50, 50, 50, 50, 50)
arrSize = 6

```

```

result = cgxiapi.cr_cfg_joint_drag_damping_set(robotHandle, jointDargDamping, arrSize) #设置机器人拖动阻尼
print("返回结果: ",result.value)
if result.value != 0:    #判断设置机器人拖动阻尼是否成功
    exit()
for i in range(0, arrSize):
    print("读取结果: ", jointDargDamping[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.6.2 读取机器人拖动阻尼

函数

```
cr_cfg_joint_drag_damping_get(robotHandle,jointDargDamping,arrSize)
```

简介

读取机器人拖动阻尼。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
jointDargDamping	POINTER(c_int)	输出	机器人拖动阻尼数组 (关节 1-6)
arrSize	c_int	输入	机器人拖动阻尼数组 大小(一般为 6)

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
jointDargDamping = (c_int * 6)(0, 0, 0, 0, 0, 0)
arrSize = 6
result = cgxiapi.cr_cfg_joint_drag_damping_get(robotHandle, jointDargDamping, arrSize) #读取机器人拖动阻尼

```

```

print("返回结果: ",result.value)
if result.value != 0:    #判断读取机器人拖动阻尼是否成功
    exit()
for i in range(0, arrSize):
    print("读取结果: ", jointDargDamping[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.6.3 设置转矩异常门限倍率

函数

```
cr_cfg_torque_threshold_ratio_set(robotHandle, torqueThresholdRatio, arrSize)
```

简介

设置转矩异常门限倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
torqueThresholdRatio	c_double	输入	机器人转矩异常门限倍率数组(关节 1-6)
arrSize	int	输入	机器人转矩异常门限倍率数组大小(一般为 6)

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
torqueThresholdRatio = (c_double * 6)(1, 1, 1, 1, 1, 1)
arrSize = 6
result = cgxiapi.cr_cfg_torque_threshold_ratio_set(robotHandle, torqueThresholdRatio, arrSize)
#设置转矩异常门限倍率
if result.value != 0:    #判断设置转矩异常门限倍率是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄

```



```
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.6.4 读转矩异常门限倍率

函数

```
cr_cfg_torque_threshold_ratio_get(robotHandle, torqueThresholdRatio, int arrSize)
```

简介

读转矩异常门限倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
torqueThresholdRatio	c_double	输出	机器人转矩异常门限倍率数组(关节 1-6)
arrSize	int	输入	机器人转矩异常门限倍率数组大小(一般为 6)

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
torqueThresholdRatio = (c_double * 6)(0, 0, 0, 0, 0, 0)
arrSize = 6
result = cgxiapi.cr_cfg_torque_threshold_ratio_get(robotHandle, torqueThresholdRatio, arrSize)
#读取转矩异常门限倍率
if result.value != 0:    #判断读取转矩异常门限倍率是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.6.5 设置扩展轴步长及速度倍率

函数

```
cr_cfg_exj_move_speed_ratio_set(robotHandle, exjMoveSpeedRatio)
```

简介

设置扩展轴步长及速度倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
exjMoveSpeedRatio	double	输入	扩展轴步长及速度倍率

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
exjMoveSpeedRatio = 1
result = cgxiapi.cr_cfg_exj_move_speed_ratio_set(robotHandle, exjMoveSpeedRatio)
#设置扩展轴步长及速度倍率
if result.value != 0: #判断设置扩展轴步长及速度倍率是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.6.6 读扩展轴步长及速度倍率

函数

```
cr_cfg_exj_move_speed_ratio_get(robotHandle, exjMoveSpeedRatio)
```

简介

读扩展轴步长及速度倍率。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
exjMoveSpeedRatio	c_double	输出	扩展轴步长及速度倍率

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
exjMoveSpeedRatio = (c_double)(0)
result = cgxiapi.cr_cfg_exj_move_speed_ratio_get(robotHandle, exjMoveSpeedRatio)
#读取扩展轴步长及速度倍率
if result.value != 0: #判断读取扩展轴步长及速度倍率是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.7 机器人限值

3.6.7.1 设置限制模式

函数

```
cr_cfg_safety_limit_type_set(robotHandle, safetyLimitsValuesType)
```

简介

设置限制模式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
safetyLimitsValuesType	SafetyLimitsValuesType	输入	机器人限制等级

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyLimitsValuesType = basestruct.SafetyLimitsValuesType.LimitLevel_2
result=cgxiapi.cr_cfg_safety_limit_type_set(robotHandle, safetyLimitsValuesType)
#设置限制模式为 2（限制偏多）
print("返回结果: ",result.value)
if result.value != 0: #判断设置限制模式是否成功
    exit()
print("读取结果: ",safetyLimitsValuesType)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.7.2 读取限制模式

函数

```
cr_cfg_safety_limit_type_get(robotHandle, safetyLimitsValuesType)
```

简介

读取限制模式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
safetyLimitsValuesType	POINTER(SafetyLimitsValuesType)	输出	机器人限制等级

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyLimitsValueType = basestruct.SafetyLimitsValueType()
result=cgxiapi.cr_cfg_safety_limit_type_get(robotHandle, safetyLimitsValueType)
#读取限制模式
print("返回结果: ",result.value)
if result.value != 0:    #判断读取限制模式是否成功
    exit()
print("读取结果: ",safetyLimitsValueType.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.7.3 设置自定义模式下参数

函数

```
cr_cfg_safety_limit_para_set(robotHandle, mode, safetyLimitsValues)
```

简介

设置自定义模式下参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	安全限值模式
safetyLimitsValues	SafetyLimitsValues	输入	安全限值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
mode = basestruct.SafetyLimitsMode.mode_Reduced

```

```

safetyLimitsValues = basestruct.SafetyLimitsValues()
safetyLimitsValues.maxPower = 300
safetyLimitsValues.maxMomentum = 25
safetyLimitsValues.maxStoppingTime = 400
safetyLimitsValues.maxStoppingDistance = 500
safetyLimitsValues.maxTcpSpeed = 1500
safetyLimitsValues.maxForce = 150
safetyLimitsValues.maxElbowSpeed = 1500
safetyLimitsValues.maxElbowForce = 100
result=cgxiapi.cr_cfg_safety_limit_para_set(robotHandle, mode, safetyLimitsValues)
#设置自定义模式下参数
print("返回结果: ",result.value)
if result.value != 0:    #判断设置自定义模式下参数是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.7.4 读取自定义模式下参数

函数

```
cr_cfg_safety_limit_para_get(robotHandle, mode, safetyLimitsValues)
```

简介

读取自定义模式下参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	安全限值模式
safetyLimitsValues	POINTER(SafetyLimitsValues)	输出	安全限值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
mode = basestruct.SafetyLimitsMode.mode_Reduced
safetyLimitsValues = basestruct.SafetyLimitsValues()
result=cgxiapi.cr_cfg_safety_limit_para_get(robotHandle, mode, safetyLimitsValues)
#读取自定义模式下参数
print("返回结果: ",result.value)
if result.value != 0:    #判断读取自定义模式下参数是否成功
    exit()
print("读取结果: ",safetyLimitsValues.maxTcpSpeed)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.7.5 设置碰撞后处理方式

函数

```
cr_cfg_safety_collihandle_type_set(robotHandle, safetyCollisionHandleMode);
```

简介

设置碰撞后处理方式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
safetyCollisionHandleMode	SafetyCollisionHandleMode	输入	碰撞后处理方式

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #真实臂连接
print("机械臂建立连接返回值: ", re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyCollisionHandleMode=basestruct.SafetyCollisionHandleMode.Collision_EnterReboundMode
result = cgxiapi.cr_cfg_safety_collihandle_type_set(robotHandle,safetyCollisionHandleMode)
#将碰撞后处理方式设置为进入反弹模式
if result.value != 0:    #判断设置碰撞后处理方式是否成功

```

```
exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.7.6 读取碰撞后处理方式

函数

```
cr_cfg_safety_collihandle_type_get(robotHandle,safetyCollisionHandleMode);
```

简介

读取碰撞后处理方式。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	RobotHandle	输入	机械臂实例句柄
safetyCollisionHandleMode	POINTER(SafetyCollisionHandleMode)	输出	碰撞后处理方式

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #真实臂连接
print("机械臂建立连接返回值: ", re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyCollisionHandleMode = basestruct.SafetyCollisionHandleMode()
result = cgxiapi.cr_cfg_safety_collihandle_type_get(robotHandle,safetyCollisionHandleMode)
#读取碰撞后处理方式
if result.value != 0:  #判断读取碰撞后处理方式是否成功
    exit()
print("碰撞后处理方式:", safetyCollisionHandleMode.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.8 关节限值

3.6.8.1 设置关节限值

函数

```
cr_cfg_safety_joint_limit_set(robotHandle, mode, jointIndex, safetyLimitJoint)
```

简介

设置关节限值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	关节模式
jointIndex	c_int	输入	关节序号（范围：0-5）
safetyLimitJoint	SafetyLimitsJointAngle	输入	关节限值

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
mode = basestruct.SafetyLimitsMode.mode_Reduced
safetyLimitJoint = basestruct.SafetyLimitsJointAngle()
safetyLimitJoint.maxJointPosition = 200
safetyLimitJoint.minJointPosition = -200
safetyLimitJoint.maxJointSpeed = 150
result=cgxiapi.cr_cfg_safety_joint_limit_set(robotHandle, mode, 1, safetyLimitJoint)
#设置二关节限值
print("返回结果: ",result.value)
if result.value != 0: #判断设置关节限值是否成功
    exit()
print("设置结果: ",safetyLimitJoint.maxJointPosition,safetyLimitJoint.minJointPosition,
safetyLimitJoint.maxJointSpeed)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.8.2 读取关节限值

函数

```
cr_cfg_safety_joint_limit_get(robotHandle, mode, jointIndex, safetyLimitJoint)
```

简介

读取关节限值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
mode	SafetyLimitsMode	输入	关节模式
jointIndex	POINTER(c_int)	输入	关节序号（范围：0-5）
safetyLimitJoint	SafetyLimitsJointAngle	输出	关节限值

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
mode = basestruct.SafetyLimitsMode.mode_Normal
safetyLimitJoint = basestruct.SafetyLimitsJointAngle()
result=cgxiapi.cr_cfg_safety_joint_limit_get(robotHandle, mode, 2, safetyLimitJoint)
#读取三关节限值
print("返回结果: ",result.value)
if result.value != 0: #判断读取关节限值是否成功
    exit()
print("设置结果: ",safetyLimitJoint.maxJointPosition,safetyLimitJoint.minJointPosition,
    safetyLimitJoint.maxJointSpeed)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.9 安全平面

3.6.9.1 读取安全平面个数

函数

```
cr_cfg_safety_plane_count(robotHandle)
```

简介

读取安全平面个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回安全平面数量。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_safety_plane_count(robotHandle) #读取安全平面个数
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取安全平面个数是否成功
    exit()
print("读取结果: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.9.2 读取安全平面信息

函数

```
cr_cfg_safety_plane_get(robotHandle, index, safetyLimitsBoundaryPlane)
```

简介

读取安全平面信息。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	安全平面序号（范围：0-7）
safetyLimitsBoundaryPlane	POINTER(SafetyLimitsBoundaryPlane)	输出	安全平面信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyLimitsBoundaryPlane = basestruct.SafetyLimitsBoundaryPlane()
result=cgxiapi.cr_cfg_safety_plane_get(robotHandle, 1, safetyLimitsBoundaryPlane)
#读取序号为 1 的安全平面信息
print("返回结果：",result.value)
if result.value != 0: #判断读取安全平面信息是否成功
    exit()
print("读取结果：",safetyLimitsBoundaryPlane.id,safetyLimitsBoundaryPlane.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.9.3 增加安全平面

函数

```
cr_cfg_safety_plane_add(robotHandle, safetyLimitsBoundaryPlane)
```

简介

增加安全平面。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

safetyLimitsBoundaryPlane	SafetyLimitsBoundaryPlane	输入	安全平面信息
---------------------------	---------------------------	----	--------

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyLimitsBoundaryPlane = basestruct.SafetyLimitsBoundaryPlane()
safetyLimitsBoundaryPlane.name = b'aa'
safetyLimitsBoundaryPlane.id = 2
result=cgxiapi.cr_cfg_safety_plane_add(robotHandle, safetyLimitsBoundaryPlane)
#增加安全平面
print("返回结果: ",result.value)
if result.value != 0: #判断增加安全平面是否成功
    exit()
print("读取结果: ",safetyLimitsBoundaryPlane.id,safetyLimitsBoundaryPlane.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.9.4 删除安全平面

函数

```
cr_cfg_safety_plane_delete(robotHandle, index)
```

简介

删除安全平面。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	安全平面序号（范围：0-7）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_safety_plane_delete(robotHandle, 2) #删除序号为2的安全平面
print("返回结果: ",result.value)
if result.value != 0: #判断删除安全平面是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.9.5 修改安全平面数据

函数

```
cr_cfg_safety_plane_set(robotHandle, index, safetyLimitsBoundaryPlane)
```

简介

修改安全平面数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	安全平面序号 (范围: 0-7)
safetyLimitsBoundaryPlane	SafetyLimitsBoundaryPlane	输入	安全平面信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
safetyLimitsBoundaryPlane = basestruct.SafetyLimitsBoundaryPlane()
safetyLimitsBoundaryPlane.name = b'aa'
safetyLimitsBoundaryPlane.id = 2
result=cgxiapi.cr_cfg_safety_plane_set(robotHandle, 1, safetyLimitsBoundaryPlane)
#修改序号为 1 的安全平面数据
print("返回结果：",result.value)
if result.value != 0:    #判断修改安全平面数据是否成功
    exit()
print("读取结果：",safetyLimitsBoundaryPlane.id,safetyLimitsBoundaryPlane.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.10 安全 I/O

3.6.10.1 设置可配置输入信号

函数

```
cr_cfg_safety_io_input_set(robotHandle, index, safetyinput)
```

简介

设置可配置输入信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引，输入范围：0-3
safetyinput	SafetyIOInput	输入	可配置输入信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyinput = basestruct.SafetyIOInput.emergencyStop

```

```

result=cgxiapi.cr_cfg_safety_io_input_set(robotHandle, 1, safetyinput)
#设置序号为 1 的可配置输入信号为紧急停止模式
print("返回结果: ",result.value)
if result.value != 0:    #判断设置可配置输入信号是否成功
    exit()
print("读取结果: ",safetyinput)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.10.2 读取可配置输入信号

函数

```
cr_cfg_safety_io_input_get(robotHandle, index, safetyinput)
```

简介

读取可配置输入信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引，输入范围：0-3
safetyinput	POINTER(SafetyIOInput)	输出	可配置输入信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyinput = basestruct.SafetyIOInput()
result=cgxiapi.cr_cfg_safety_io_input_get(robotHandle, 1, safetyinput)
#读取序号为 1 的可配置输入信号
print("返回结果: ",result.value)
if result.value != 0:    #判断读取可配置输入信号是否成功
    exit()
print("读取结果: ",safetyinput.value)

```



```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.10.3 设置可配置输出信号

函数

```
cr_cfg_safety_io_output_set(robotHandle, index, safetyoutput)
```

简介

设置可配置输出信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引，输入范围：0-3
safetyoutput	SafetyIOInput	输入	可配置输出信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyoutput = basestruct.SafetyIOOutput.robotMoving
result=cgxiapi.cr_cfg_safety_io_output_set(robotHandle, 1, safetyoutput)
#设置序号为 1 的可配置输出信号为机器人运动
print("返回结果：",result.value)
if result.value != 0:  #判断设置可配置输出信号是否成功
    exit()
print("读取结果：",safetyoutput)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.10.4 读取可配置输出信号

函数

```
cr_cfg_safety_io_output_get(robotHandle, index, safetyoutput)
```

简介

读取可配置输出信号。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	索引，输入范围：0-3
safetyoutput	POINTER(SafetyIOInput)	输出	可配置输出信号

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
safetyoutput = basestruct.SafetyIOOutput()
result=cgxiapi.cr_cfg_safety_io_output_get(robotHandle, 1, safetyoutput)
#读取序号为 1 的可配置输出信号
print("返回结果: ",result.value)
if result.value != 0: #判断读取可配置输出信号是否成功
    exit()
print("读取结果: ",safetyoutput.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.11 示教器控制

3.6.11.1 设置是否启用示教器

函数

```
cr_cfg_safety_tp_use_set(robotHandle, isUse)
```

简介

设置是否启用示教器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
isUse	c_int	输入	是否启用示教器(True 为启用, False 为不启用)

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
isUse = True
result=cgxiapi.cr_cfg_safety_tp_use_set(robotHandle, isUse) #设置启用示教器
print("返回结果: ",result.value)
if result.value != 0: #判断设置启用示教器是否成功
    exit()
print("读取结果: ",isUse)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.11.2 读取当前是否启用示教器

函数

```
cr_cfg_safety_tp_use_get(robotHandle)
```

简介

读取当前启用示教器。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取结果，返回 0 为禁用，1 为启用。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_safety_tp_use_get(robotHandle) #读取当前是否启用示教器
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取当前是否启用示教器是否成功
    exit()
print("读取结果: ",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.12 IO 变量

3.6.12.1 设置数字输入信号配置

函数

```
cr_cfg_var_di_set(robotHandle, type, index, inputConfiguration)
```

简介

设置数字输入信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	c_int	输入	IO 索引（范围：0-7）
inputConfiguration	IOInputConfiguration	输入	数字量输入配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
inputConfiguration = basestruct.IOInputConfiguration()
type = basestruct.VarDigitalIOType.Digital
inputActions = basestruct.InputAction.InputAction_NONE
inputConfiguration.inputActions = inputActions
inputConfiguration.inputFilteringTime = 1
inputConfiguration.inputName = b'ad'
index = 1
result=cgxiapi.cr_cfg_var_di_set(robotHandle, type, index, inputConfiguration)
#设置索引为 1 的数字输入信号配置
print("返回结果: ",result.value)
if result.value != 0: #判断设置数字输入信号配置是否成功
    exit()
print("设置结果:",inputConfiguration.inputFilteringTime)
```

3.6.12.2 读取数字输入信号配置

函数

```
cr_cfg_var_di_get(robotHandle, type, index, inputConfiguration)
```

简介

读取数字输入信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	c_int	输入	IO 索引（范围：0-7）
inputConfiguration	POINTER(IOInputConfiguration)	输出	数字量输入配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
inputConfiguration = basestruct.IOInputConfiguration()
type = basestruct.VarDigitalIOType.Digital
inputConfiguration.inputActions = basestruct.InputAction.InputAction_NONE
index = 1
result=cgxiapi.cr_cfg_var_di_get(robotHandle, type, index, inputConfiguration)
#读取索引号为 1 的数字输入信号配置
print("返回结果: ",result.value)
if result.value != 0: #判断读取数字输入信号配置是否成功
    exit()
print("读取结果:",inputConfiguration.inputFilteringTime)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.3 设置数字输出信号配置

函数

```
cr_cfg_var_do_set(robotHandle, type, index, outputConfiguration)
```

简介

设置数字输出信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	c_int	输入	IO 索引（范围：0-7）
outputConfiguration	IOOutputConfiguration	输入	数字量输出配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
```

```

re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
outputConfiguration = basestruct.IOOutputConfiguration()
type = basestruct.VarDigitalIOType.Digital
outputConfiguration.outputActions = basestruct.OutputAction.OutputAction_RobotON_LOW
outputConfiguration.outputOptions = basestruct.OutputOptions.OutputOptions_Enable
outputConfiguration.outputRule = basestruct.OutputRule.OutputRule_Disable
outputConfiguration.outputName = b'af'
index = 1
result=cgxiapi.cr_cfg_var_do_set(robotHandle, type, index, outputConfiguration)
#设置索引号为 1 的数字输出信号配置
print("返回结果: ",result.value)
if result.value != 0:  #判断设置数字输出信号配置是否成功
    exit()
print("设置结果:",outputConfiguration.outputName)
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.4 读取数字输出信号配置

函数

```
cr_cfg_var_do_get(robotHandle, type, index, outputConfiguration)
```

简介

读取数字输出信号配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	IO 类型
index	c_int	输入	IO 索引（范围：0-7）
outputConfiguration	POINTER(IOOutputConfiguration)	输出	数字量输出配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
outputConfiguration = basestruct.IOOutputConfiguration()
type = basestruct.VarDigitalIOType.Digital
index = 1
result=cgxiapi.cr_cfg_var_do_get(robotHandle, type, index, outputConfiguration)
#读取索引号为 1 的数字输出信号配置
print("返回结果: ",result.value)
if result.value != 0: #判断读取数字输出信号配置是否成功
    exit()
print("设置结果:",outputConfiguration.outputName)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.12.5 设置工具 IO 类型

函数

```
cr_cfg_var_tool_io_type_set(robotHandle,index,toolIoType)
```

简介

设置工具 IO 类型。调用一次或多次该接口后，都需要先调用 `cr_destroy_robot` 接口释放当前句柄，然后再调用 `cr_create_robot` 接口重新创建新句柄。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	IO 索引（4 路 IO 范围：0~3； 10 路 IO 范围：0~9）
toolIoType	POINTER(ToolIoType)	输入	工具 IO 类型

返回值

数据类型为 `CRresult`，若为 `success` 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
toolIoType= basestruct.ToolIoType.IO_OUTPUT
index = 1
result=cgxiapi.cr_cfg_var_tool_io_type_set(robotHandle, index, toolIoType)
#设置索引号为 1 的工具 IO 类型
print("返回结果: ",result.value)
if result.value != 0: #判断设置工具 IO 类型是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.6 读取工具 IO 类型

函数

```
cr_cfg_var_tool_io_type_get(robotHandle,int index,toolIoType)
```

简介

读取工具 IO 类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	IO 索引（4 路 IO 范围：0~3； 10 路 IO 范围：0~9）
toolIoType	POINTER(ToolIoType)	输出	工具 IO 类型

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接

```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
toolIoType= basestruct.ToolIoType()
index = 1
result=cgxiapi.cr_cfg_var_tool_io_type_get(robotHandle, index, toolIoType)
#读取索引号为 1 的工具 IO 类型
print("返回结果: ",result.value)
if result.value != 0:    #判断读取工具 IO 类型是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.7 设置模拟量输入信号类型

函数

```
cr_cfg_var_ai_set(robotHandle, type, index, analogInputConfiguration)
```

简介

设置模拟量输入信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	模拟量类型
index	c_int	输入	模拟量索引（范围：0-2）
analogInputConfiguration	AnalogInputConfiguration	输入	模拟量输入配置

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
analogInputConfiguration = basestruct.AnalogInputConfiguration()
analogInputType = basestruct.AnalogType.Current
type = basestruct.VarDigitalIOType.Analog
analogInputConfiguration.analogInputType = analogInputType
analogInputConfiguration.analogInputName = b'an'
index = 1
result=cgxiapi.cr_cfg_var_ai_set(robotHandle, type, index, analogInputConfiguration)
#设置索引号为 1 的模拟量输入信号类型
print("返回结果: ",result.value)
if result.value != 0:    #判断设置模拟量输入信号类型是否成功
    exit()
print("设置结果:",analogInputConfiguration.analogInputName)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.8 读取模拟量输入信号类型

函数

```
cr_cfg_var_ai_get(robotHandle, type, index, analogInputConfiguration)
```

简介

读取模拟量输入信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	VarDigitalIOType	输入	模拟量类型
index	c_int	输入	模拟量索引（范围：0-2）
analogInputConfiguration	POINTER(AnalogInputConfiguration)	输出	模拟量输入配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")

```

```

    exit()
robotHandle = re1[1].value
analogInputConfiguration = basestruct.AnalogInputConfiguration()
type = basestruct.VarDigitalIOType.Analog
index = 1
result=cgxiapi.cr_cfg_var_ai_get(robotHandle, type, index, analogInputConfiguration)
#读取索引号为 1 的模拟量输入信号类型
print("返回结果: ",result.value)
if result.value != 0:    #判断读取模拟量输入信号类型是否成功
    exit()
print("设置结果:",analogInputConfiguration.analogInputName)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.9 设置模拟量输出信号类型

函数

```
cr_cfg_var_ao_set(robotHandle, index, analogOutputConfiguration)
```

简介

设置模拟量输出信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	模拟量索引（范围：0）
analogOutputConfiguration	AnalogOutputConfiguration	输入	模拟量输出配置

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
analogOutputConfiguration = basestruct.AnalogOutputConfiguration()

```

```

analogOutputConfiguration.analogOutputType = basestruct.AnalogType.Current
analogOutputConfiguration.analogOutputName = b'afo'
index = 0
result=cgxiapi.cr_cfg_var_ao_set(robotHandle, index, analogOutputConfiguration)
#设置索引号为 0 的模拟量输出信号类型
print("返回结果: ",result.value)
if result.value != 0:    #判断设置模拟量输出信号类型是否成功
    exit()
print("设置结果:",analogOutputConfiguration.analogOutputName)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.10 读取模拟量输出信号类型

函数

```
cr_cfg_var_ao_get(robotHandle, index, analogOutputConfiguration)
```

简介

读取模拟量输出信号类型。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	模拟量索引（范围：0）
analogOutputConfiguration	POINTER(AnalogOutputConfiguration)	输出	模拟量输出配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
analogOutputConfiguration = basestruct.AnalogOutputConfiguration()
index = 0
result=cgxiapi.cr_cfg_var_ao_get(robotHandle, index, analogOutputConfiguration)

```

```

#读取索引号为 0 的模拟量输出信号类型
print("返回结果: ",result.value)
if result.value != 0:    #判断读取模拟量输出信号类型是否成功
    exit()
print("读取结果:",analogOutputConfiguration.analogOutputName)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.12.11 设置工具电压输出值

函数

```
cr_cfg_var_to_set(robotHandle, toolPower)
```

简介

设置工具电压输出值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
toolPower	ToolPower	输入	电压值

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
toolPower = basestruct.ToolPower.Power_on
result=cgxiapi.cr_cfg_var_to_set(robotHandle, toolPower)
#设置工具电压输出值
print("返回结果: ",result.value)
if result.value != 0:    #判断设置工具电压输出值是否成功
    exit()
print("设置结果:",toolPower)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功

```

```
print("断开失败")
exit()
```

3.6.12.12 读取工具电压输出值

函数

```
cr_cfg_var_to_get(robotHandle, toolPower)
```

简介

读取工具电压输出值。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
toolPower	POINTER(ToolPower)	输出	电压值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
toolPower = basestruct.ToolPower()
result=cgxiapi.cr_cfg_var_to_get(robotHandle, toolPower)
#读取工具电压输出值
print("返回结果：",result.value)
if result.value != 0: #判断读取工具电压输出值是否成功
    exit()
print("读取结果:",toolPower.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.12.13 设置编码器配置

函数

```
cr_cfg_encoder_config_set(robotHandle, index, encoderConfig)
```

简介

设置编码器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	编码器索引
encoderConfig	EncoderConfiguration	输入	编码器状态

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
encoderConfig = basestruct.EncoderConfiguration()
encoderConfig.eqepType = 0
encoderConfig.eqepCountMode = 0
encoderConfig.strobeInputMode = 0
encoderConfig.eqepEnabled = 1
result=cgxiapi.cr_cfg_encoder_config_set(robotHandle, index, encoderConfig)#设置编码器配置
print("返回结果: ",result.value)
if result.value != 0: #判断设置编码器配置是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.12.14 读取编码器配置

函数

```
cr_cfg_encoder_config_get(robotHandle, index, encoderConfig)
```

简介

读取编码器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	编码器索引
encoderConfig	EncoderConfiguration	输出	编码器状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
encoderConfig = basestruct.EncoderConfiguration()
result=cgxiapi.cr_cfg_encoder_config_get(robotHandle, index, encoderConfig)#读取编码器配置
print("返回结果: ",result.value)
if result.value != 0: #判断读取编码器配置是否成功
    exit()
print("编码器类型: ",encoderConfig.eqepType)
print("编码器模式: ",encoderConfig.eqepCountMode)
print("编码器探针: ",encoderConfig.strobeInputMode)
print("编码器控制: ",encoderConfig.eqepEnabled)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13 通讯变量

3.6.13.1 设置位寄存器配置

函数

```
cr_cfg_var_bit_reg_set(robotHandle, index, bitRegisterConfiguration)
```

简介

设置位寄存器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
bitRegisterConfiguration	BitRegisterConfiguration	输入	寄存器配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
bitRegisterConfiguration= basestruct.BitRegisterConfiguration()
GeneralPurposeBOOLeanRegisterInputActions = basestruct.InputAction.InputAction_NONE
GeneralPurposeBOOLeanRegisterOutputActions = \
    basestruct.OutputAction.OutputAction_RobotON_HIGH
GeneralPurposeBOOLeanRegisterOutputOutputRule = \
    basestruct.OutputRule.OutputRule_Disable
bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterInputActions = \
    GeneralPurposeBOOLeanRegisterInputActions
bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterOutputActions = \
    GeneralPurposeBOOLeanRegisterOutputActions
bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterOutputRule = \
    GeneralPurposeBOOLeanRegisterOutputOutputRule
bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterName = b'aa'
index = 2
result=cgxiapi.cr_cfg_var_bit_reg_set(robotHandle, index, bitRegisterConfiguration)
#设置索引值为 2 的位寄存器配置
print("返回结果: ",result.value)
if result.value != 0: #判断设置位寄存器配置是否成功
    exit()
print("设置结果:",bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterName,
      bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterOutputActions.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.2 读取位寄存器配置

函数

```
cr_cfg_var_bit_reg_get(robotHandle, index, bitRegisterConfiguration)
```

简介

读取位寄存器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
bitRegisterConfiguration	POINTER(BitRegisterConfiguration)	输出	寄存器配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
bitRegisterConfiguration= basestruct.BitRegisterConfiguration()
index = 2
result=cgxiapi.cr_cfg_var_bit_reg_get(robotHandle, index, bitRegisterConfiguration)
#读取索引值为2的位寄存器配置
print("返回结果：",result.value)
if result.value != 0: #判断读取位寄存器配置是否成功
    exit()
print("读取结果:",bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterName,
      bitRegisterConfiguration.GeneralPurposeBOOLeanRegisterOutputActions.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.3 读取所有位寄存器配置

函数

```
cr_cfg_var_all_bit_reg_get(robotHandle, bitRegisterConfigurationArray, arraySize)
```

简介

读取所有位寄存器配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
bitRegisterConfigurationArray	POINTER(BitRegisterConfiguration)	输出	存放寄存器配置的地址
arraySize	c_int	输入	读取的数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
bitRegisterConfiguration= (basestruct.BitRegisterConfiguration * 1000)()
arraySize = 1000
result=cgxiapi.cr_cfg_var_all_bit_reg_get(robotHandle, bitRegisterConfiguration, arraySize)
#读取所有位寄存器配置
print("返回结果: ",result.value)
if result.value != 0: #判断读取所有位寄存器配置是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.4 设置 16 位整数寄存器变量名

函数

```
cr_cfg_var_int16_reg_name_set(robotHandle, index, name, len)
```

简介

设置 16 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
name	c_char_p	输入	寄存器名字
len	c_int	输入	名字长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
name = b'ad'
index = 1
len = 6
result=cgxiapi.cr_cfg_var_int16_reg_name_set(robotHandle, index, name, len)
#设置索引值为 1 的 16 位整数寄存器变量名
print("返回结果: ",result.value)
if result.value != 0: #判断设置 16 位整数寄存器变量名是否成功
    exit()
print("设置结果:",name)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.5 读取 16 位整数寄存器变量名

函数

```
cr_cfg_var_int16_reg_name_get(robotHandle, index, name, len)
```

简介

读取 16 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
name	c_char_p	输出	寄存器名字
len	c_int	输入	名字长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回变量名。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
name = b'ee'
index = 4
len = 20
result = cgxiapi.cr_cfg_var_int16_reg_name_get(robotHandle, index, name, len)
#读取索引值为4的16位整数寄存器变量名
print("返回结果：", result[0].value)
if result[0].value != 0: #判断读取16位整数寄存器变量名是否成功
    exit()
print("读取结果:", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.6 读取所有 16 位整数寄存器变量名

函数

```
cr_cfg_var_all_int16_reg_name_get(robotHandle, registerConfigurationArray, arrSize)
```

简介

读取所有 16 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
------	------	-------	----

robotHandle	RobotHandle	输入	机械臂实例句柄
registerConfigurationArray	POINTER(RegisterConfiguration)	输出	存放寄存器名字的地址
arrSize	int	输入	读取的名字数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
RegisterConfigurationArray = (basestruct.RegisterConfiguration * 1000)()
arrSize = 1000
result = cgxiapi.cr_cfg_var_all_int16_reg_name_get(robotHandle, registerConfigurationArray,
arrSize)
#读取所有 16 位整数寄存器变量名
if result.value != 0: #判断读取所有 16 位整数寄存器变量名是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.7 设置 32 位整数寄存器变量名

函数

```
cr_cfg_var_int32_reg_name_set(robotHandle, index, name, len)
```

简介

设置 32 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
name	c_char_p	输入	寄存器名字
len	c_int	输入	名字长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
name = b'de'
index = 1
len = 6
result=cgxiapi.cr_cfg_var_int32_reg_name_set(robotHandle, index, name, len)
#设置索引值为 1 的 32 位整数寄存器变量名
print("返回结果: ",result.value)
if result.value != 0: #判断设置 32 位整数寄存器变量名是否成功
    exit()
print("设置结果:",name)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.8 读取 32 位整数寄存器变量名

函数

```
cr_cfg_var_int32_reg_name_get(robotHandle, index, name, len)
```

简介

读取 32 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
name	c_char_p	输出	寄存器名字
len	c_int	输入	名字长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回变量名称。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
name = b'eee'
index = 4
len = 20
result = cgxiapi.cr_cfg_var_int32_reg_name_get(robotHandle, index, name, len)
#读取索引值为4的32位整数寄存器变量名
print("返回结果: ", result[0].value)
if result[0].value != 0: #判断读取32位整数寄存器变量名是否成功
    exit()
print("读取结果:", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.9 读取所有 32 位整数寄存器变量名

函数

```
cr_cfg_var_all_int32_reg_name_get(robotHandle, registerConfigurationArray, arrSize)
```

简介

读取所有 32 位整数寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
registerConfigurationArray	POINTER(RegisterConfiguration)	输出	存放寄存器名字的地址
arrSize	c_int	输入	读取的名字数量

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
```

```

re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
RegisterConfigurationArray = (basestruct.RegisterConfiguration * 1000)()
arrSize = 1000
result = cgxiapi.cr_cfg_var_all_int32_reg_name_get(robotHandle, registerConfigurationArray,
arrSize)
#读取所有 32 位整数寄存器变量名
if result.value != 0: #判断读取所有 32 位整数寄存器变量名是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.13.10 设置浮点寄存器变量名

函数

```
cr_cfg_var_float_reg_name_set(robotHandle, index, name, len)
```

简介

设置浮点寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
name	c_char_p	输入	寄存器名字
len	c_int	输入	名字长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
name = b'ff'
index = 1
len = 6
result=cgxiapi.cr_cfg_var_float_reg_name_set(robotHandle, index, name, len)
#设置索引值为1的浮点寄存器变量名
print("返回结果：",result.value)
if result.value != 0:    #判断设置浮点寄存器变量名是否成功
    exit()
print("设置结果:",name)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.13.11 读取浮点寄存器变量名

函数

```
cr_cfg_var_float_reg_name_get(robotHandle, index, name, len)
```

简介

读取浮点寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	寄存器索引值（范围：0-999）
name	c_char_p	输出	寄存器名字
len	c_int	输入	名字长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回变量名称。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
name = b'pppp'
index = 4

```

```
len = 20
result = cgxiapi.cr_cfg_var_float_reg_name_get(robotHandle, index, name, len)
#读取索引值为 4 的浮点寄存器变量名
print("返回结果: ", result[0].value)
if result[0].value != 0:    #判断读取浮点寄存器变量名是否成功
    exit()
print("读取结果:", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.13.12 读取所有浮点寄存器变量名

函数

```
cr_cfg_var_all_float_reg_name_get(robotHandle,registerConfigurationArray,arrSize)
```

简介

读取所有浮点寄存器变量名。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
registerConfigurationArray	POINTER(RegisterConfiguration)	输出	存放寄存器名字的地址
arrSize	c_int	输入	读取的名字数量

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
RegisterConfigurationArray = (basestruct.RegisterConfiguration * 1000)()
arrSize = 1000
result = cgxiapi.cr_cfg_var_all_float_reg_name_get(robotHandle, registerConfigurationArray,
arrSize)
#读取所有浮点寄存器变量名
```

```

if result.value != 0:    #判断读取所有浮点寄存器变量名是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.14 安装变量

3.6.14.1 读取安装变量个数

函数

```
cr_cfg_var_install_count(robotHandle)
```

简介

读取安装变量个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取结果。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_var_install_count(robotHandle)    #读取安装变量个数
print("返回结果：",result[0].value)
if result[0].value != 0:    #判断读取安装变量个数是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.14.2 读取安装变量数据

函数

```
cr_cfg_var_install_get(robotHandle, index, variableMsg)
```

简介

读取安装变量数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	安装变量索引（范围：0-99）
variableMsg	POINTER(VariableMsg)	输出	安装变量数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
variableMsg = basestruct.VariableMsg()
result=cgxiapi.cr_cfg_var_install_get(robotHandle, index, variableMsg)
#读取索引号为 1 的安装变量数据#
print("返回结果：",result.value)
if result.value != 0: #判断读取安装变量数据是否成功
    exit()
print("读取结果:",variableMsg.variableName)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.14.3 增加安装变量

函数

```
cr_cfg_var_install_add(robotHandle, variableMsg)
```

简介

增加安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
variableMsg	POINTER(VariableMsg)	输入	安装变量数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
variableMsg = basestruct.VariableMsg()
variableMsg.variableName = b'qw'
variableMsg.variableType = 3 #变量类型为 number
variableMsg.numberValue = 6
variableMsg.variableID = 3
result=cgxiapi.cr_cfg_var_install_add(robotHandle, variableMsg) #增加安装变量
print("返回结果: ",result.value)
if result.value != 0: #判断增加安装变量是否成功
    exit()
print("增加结果:",variableMsg.variableName,variableMsg.variableID)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.14.4 删除安装变量

函数

```
cr_cfg_var_install_delete(robotHandle, index)
```

简介

删除安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	安装变量索引（范围：0-99）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
result=cgxiapi.cr_cfg_var_install_delete(robotHandle, index) #删除索引号为1的安装变量
print("返回结果: ",result.value)
if result.value != 0: #判断删除安装变量是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.14.5 修改安装变量

函数

```
cr_cfg_var_install_set(robotHandle, index, variableMsg)
```

简介

修改安装变量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	安装变量索引（范围：0-99）
variableMsg	POINTER(VariableMsg)	输入	安装变量数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
variableMsg = basestruct.VariableMsg()
variableMsg.variableName = b'er'
variableMsg.variableType = 3
variableMsg.numberValue = 9
index = 1
result=cgxiapi.cr_cfg_var_install_set(robotHandle, index, variableMsg)
#修改索引号为 1 的安装变量
print("返回结果: ",result.value)
if result.value != 0: #判断修改安装变量是否成功
    exit()
print("设置结果:",variableMsg.variableName)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15 坐标系

3.6.15.1 读取点坐标系个数

函数

```
cr_cfg_cs_point_count(robotHandle)
```

简介

读取点坐标系个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取个数。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_cs_point_count(robotHandle) #读取点坐标系个数
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取点坐标系个数是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.2 读取点坐标系数据

函数

```
cr_cfg_cs_point_get(robotHandle, index, pointCSNode)
```

简介

读取点坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	点坐标系索引（范围：0-9）
pointCSNode	POINTER(PointCSNode)	输出	点坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()

```

```

robotHandle = re1[1].value
pointCSNode = basestruct.PointCSNode()
index = 1
result=cgxiapi.cr_cfg_cs_point_get(robotHandle, index, pointCSNode)
#读取索引号为 1 的点坐标系数据
print("返回结果: ",result.value)
if result.value != 0:    #判断读取点坐标系数据是否成功
    exit()
print("读取结果:",pointCSNode.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.3 读取点坐标系数据(通过名称)

函数

```
cr_cfg_cs_point_accord_name_get(robotHandle, csName, pointCSNode)
```

简介

读取点坐标系数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
csName	c_char_p	输入	点坐标系名称
pointCSNode	POINTER(PointCSNode)	输出	点坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointCSNode = basestruct.PointCSNode()
csName= b"point_1"
result=cgxiapi.cr_cfg_cs_point_accord_name_get(robotHandle, csName, pointCSNode)

```

```
#读取点坐标数据
print("返回结果: ",result.value)
if result.value != 0:    #判断读取点坐标数据是否成功
    exit()
print("读取结果:",pointCSNode.isValid)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.4 增加点坐标系

函数

```
cr_cfg_cs_point_add(robotHandle, pointCSNode)
```

简介

增加点坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
pointCSNode	PointCSNode	输入	点坐标数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointCSNode = basestruct.PointCSNode()
pointCSNode.name = b'c0'
pointCSNode.id = 1
pointCSNode.point.toolAxisAngle = (30, 30, 30, 30, 30, 30)
pointCSNode.point.toolPosition = (0, 0, 0, 0, 0, 0)
cgxiapi.cr_kineForward(robotHandle, pointCSNode.point.toolAxisAngle,
pointCSNode.point.toolPosition)
pointCSNode.isValid = 1
result = cgxiapi.cr_cfg_cs_point_add(robotHandle, pointCSNode)    #增加点坐标系
```

```
print("返回结果：", result.value)
if result.value != 0:    #判断增加点坐标系是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.5 删除点坐标系

函数

```
cr_cfg_cs_point_delete(robotHandle, index)
```

简介

删除点坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	点坐标系索引（范围：0-9）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
result=cgxiapi.cr_cfg_cs_point_delete(robotHandle, index)    #删除索引号为 1 的点坐标系
print("返回结果：",result.value)
if result.value != 0:    #判断删除点坐标系是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.6 修改点坐标系数据

函数

```
cr_cfg_cs_point_set(robotHandle, index, pointCSNode)
```

简介

修改点坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	点坐标系索引（范围：0-9）
pointCSNode	PointCSNode	输入	点坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
pointCSNode = basestruct.PointCSNode()
pointCSNode.name = b'c2'
pointCSNode.id = 1
pointCSNode.point.toolAxisAngle = (30,30,30,30,30,30)
pointCSNode.point.toolPosition = (0,0,0,0,0,0)
cgxiapi.cr_kineForward(robotHandle, pointCSNode.point.toolAxisAngle,
pointCSNode.point.toolPosition)
pointCSNode.isValid = 1
index = 1
result = cgxiapi.cr_cfg_cs_point_set(robotHandle, index, pointCSNode)
#修改索引号为 1 的点坐标系数据
print("返回结果: ",result.value)
if result.value != 0: #判断修改点坐标系数据是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.7 读取线坐标系个数

函数

```
cr_cfg_cs_line_count(robotHandle)
```

简介

读取线坐标系个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_cs_line_count(robotHandle) #读取线坐标系个数
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取线坐标系个数是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.8 读取线坐标系数据

函数

```
cr_cfg_cs_line_get(robotHandle, index, lineCSNode)
```

简介

读取线坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	线坐标系索引（范围：0-9）
lineCSNode	POINTER(LineCSNode)	输出	线坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
lineCSNode = basestruct.LineCSNode()
index = 1
result=cgxiapi.cr_cfg_cs_line_get(robotHandle, index, lineCSNode)
#读取索引号为 1 的线坐标系数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取线坐标系数据是否成功
    exit()
print("读取结果:",lineCSNode.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.9 读取线坐标系数据(通过名称)

函数

```
cr_cfg_cs_line_accord_name_get(robotHandle, csName, lineCSNode)
```

简介

读取线坐标系数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
csName	c_char_p	输入	线坐标系名称

lineCSNode	POINTER(LineCSNode)	输出	线坐标系数据
------------	---------------------	----	--------

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
lineCSNode = basestruct.LineCSNode()
csName= b"line_1"
result=cgxiapi.cr_cfg_cs_line_accord_name_get(robotHandle, csName, lineCSNode)
#读取线坐标系数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取线坐标系数据是否成功
    exit()
print("读取结果:",lineCSNode.isValid)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.10 增加线坐标系

函数

```
cr_cfg_cs_line_add(robotHandle, lineCSNode)
```

简介

增加线坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
lineCSNode	LineCSNode	输入	线坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
lineCSNode = basestruct.LineCSNode()
lineCSNode.name = b'line'
lineCSNode.id = 4
lineCSNode.firstPoint.name = b'p1'
lineCSNode.firstPoint.id = 1
lineCSNode.secondPoint.name = b'p2'
lineCSNode.secondPoint.id = 2
lineCSNode.firstPoint.point.toolAxisAngle = (0,0,90,0,-90,0) #第一个点的关节角度
lineCSNode.firstPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,lineCSNode.firstPoint.point.toolAxisAngle,
lineCSNode.firstPoint.point.toolPosition) #正解求第一个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
lineCSNode.firstPoint.isValid = 1
lineCSNode.secondPoint.point.toolAxisAngle = (9.29,3.39,86.49,0.08,-89.91,9.29)
#第二个点的关节角度
lineCSNode.secondPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,lineCSNode.secondPoint.point.toolAxisAngle,
lineCSNode.secondPoint.point.toolPosition) #正解求第二个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
lineCSNode.secondPoint.isValid = 1
lineCSNode.coordinateJointPos = (0,0,0,0,0,0)
lineCSNode.coordinatePose = (0,0,0,0,0,0)
result=cgxiapi.cr_compute_cs_line(lineCSNode.firstPoint.point.toolPosition, 6,
lineCSNode.secondPoint.point.toolPosition, 6, lineCSNode.coordinatePose, 6)
#计算线坐标系位姿
if result.value != 0: #判断计算线坐标系是否成功
    exit()
result = cgxiapi.cr_kineInverse(robotHandle, lineCSNode.coordinatePose,
lineCSNode.coordinateJointPos, lineCSNode.coordinateJointPos)
#逆解求线坐标系关节角度
if result.value == 0: #判断逆解是否存在，即坐标系是否有效
    lineCSNode.isValid = 1
else:
```

```
lineCSNode.isValid = 0
result = cgxiapi.cr_cfg_cs_line_add(robotHandle, lineCSNode) #增加线坐标系
print("返回结果: ",result.value)
if result.value != 0: #判断增加线坐标系是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.11 删除线坐标系

函数

cr_cfg_cs_line_delete(robotHandle, index)

简介

删除线坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	线坐标系线索引（范围：0-9）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
result=cgxiapi.cr_cfg_cs_line_delete(robotHandle, index) #删除索引号为 1 的线坐标系
print("返回结果: ",result.value)
if result.value != 0: #判断删除线坐标系是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.12 修改线坐标系数据

函数

`cr_cfg_cs_line_set(robotHandle, index, lineCSNode)`

简介

修改线坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	线坐标系索引（范围：0-9）
lineCSNode	LineCSNode	输入	线坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
lineCSNode = basestruct.LineCSNode()
lineCSNode.name = b'line'
lineCSNode.id = 4
lineCSNode.firstPoint.name = b'lp1'
lineCSNode.firstPoint.id = 1
lineCSNode.secondPoint.name = b'lp2'
lineCSNode.secondPoint.id = 2
lineCSNode.firstPoint.point.toolAxisAngle = (0,0,90,0,-90,0) #第一个点的关节角度
lineCSNode.firstPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,lineCSNode.firstPoint.point.toolAxisAngle,
lineCSNode.firstPoint.point.toolPosition)#正解求第一个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
lineCSNode.firstPoint.isValid = 1
lineCSNode.secondPoint.point.toolAxisAngle = (9.29,3.39,86.49,0.08,-89.91,9.29)
#第二个点的关节角度
lineCSNode.secondPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,lineCSNode.secondPoint.point.toolAxisAngle,
```

```

lineCSNode.secondPoint.point.toolPosition)#正解求第二个点的位姿
if result.value != 0:    #判断正解是否成功
    exit()
lineCSNode.secondPoint.isValid = 1
lineCSNode.coordinateJointPos = (0,0,0,0,0,0)
lineCSNode.coordinatePose = (0,0,0,0,0,0)
result=cgxiapi.cr_compute_cs_line(lineCSNode.firstPoint.point.toolPosition, 6,
lineCSNode.secondPoint.point.toolPosition, 6, lineCSNode.coordinatePose, 6)
#计算线坐标系位姿
if result.value != 0:    #判断计算线坐标系位姿是否成功
    exit()
result = cgxiapi.cr_kineInverse(robotHandle, lineCSNode.coordinatePose,
lineCSNode.coordinateJointPos, lineCSNode.coordinateJointPos)
#逆解求线坐标系关节角度
if result.value == 0: #逆解是否存在，即线坐标系是否有效
    lineCSNode.isValid = 1
else:
    lineCSNode.isValid = 0
index = 1
result = cgxiapi.cr_cfg_cs_line_set(robotHandle, index, lineCSNode)
#修改索引号为 1 的线坐标系数据
print("返回结果: ",result.value)
if result.value != 0:    #判断修改线坐标系数据是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.13 读取面坐标系个数

函数

```
cr_cfg_cs_plane_count(robotHandle)
```

简介

读取面坐标系个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_cs_plane_count(robotHandle) #读取面坐标系个数
print("返回结果: ",result[0].value)
if result[0].value != 0: #判断读取面坐标系个数是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.14 读取面坐标系数据

函数

```
cr_cfg_cs_plane_get(robotHandle, index, planeCSNode)
```

简介

读取面坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	面坐标系索引（范围：0-9）
planeCSNode	POINTER(PlaneCSNode)	输出	面坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
```

```

robotHandle = re1[1].value
planeCSNode = basestruct.PlaneCSNode()
index = 1
result=cgxiapi.cr_cfg_cs_plane_get(robotHandle, index, planeCSNode)
#读取索引号为 1 的面坐标系数数据
print("返回结果: ",result.value)
if result.value != 0:    #判断读取面坐标系数数据是否成功
    exit()
print("读取结果:",planeCSNode.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.15 读取面坐标系数数据(通过名称)

函数

```
cr_cfg_cs_plane_accord_name_get(robotHandle, csName, planeCSNode)
```

简介

读取面坐标系数数据(通过名称)。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
csName	c_char_p	输入	面坐标系名称
planeCSNode	POINTER(PlaneCSNode)	输出	面坐标系数数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
planeCSNode = basestruct.PlaneCSNode()
csName= b"plane_1"
result=cgxiapi.cr_cfg_cs_plane_accord_name_get(robotHandle, csName, planeCSNode)

```

```
#读取面坐标系数据
print("返回结果: ",result.value)
if result.value != 0:    #判断读取面坐标系数据是否成功
    exit()
print("读取结果:",planeCSNode.name)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.16 增加面坐标系

函数

cr_cfg_cs_plane_add(robotHandle, planeCSNode)

简介

增加面坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
planeCSNode	PlaneCSNode	输入	面坐标系数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
planeCSNode = basestruct.PlaneCSNode()
planeCSNode.firstPoint.name = b'pp1'
planeCSNode.firstPoint.id = 1
planeCSNode.firstPoint.isValid = 1
planeCSNode.secondPoint.name = b'pp2'
planeCSNode.secondPoint.id = 2
planeCSNode.secondPoint.isValid = 1
planeCSNode.thirdPoint.name = b'pp3'
planeCSNode.thirdPoint.id = 3
```



```

planeCSNode.thirdPoint.isValid = 1
planeCSNode.name = b'plane'
planeCSNode.firstPoint.point.toolAxisAngle = (0,0,90,0,-90,0) #第一个点的关节角度
planeCSNode.firstPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,planeCSNode.firstPoint.point.toolAxisAngle,
planeCSNode.firstPoint.point.toolPosition) #正解求第一个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
planeCSNode.secondPoint.point.toolAxisAngle = (0,4.84,84.91,0.25,-89.99,0)
#第二个点的关节角度
planeCSNode.secondPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,planeCSNode.secondPoint.point.toolAxisAngle,
planeCSNode.secondPoint.point.toolPosition) #正解求第二个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
planeCSNode.thirdPoint.point.toolAxisAngle = (-8.1,-1.5,91.48,0.04,-90.07,-8.1)
#第三个点的关节角度
planeCSNode.thirdPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,planeCSNode.thirdPoint.point.toolAxisAngle,
planeCSNode.thirdPoint.point.toolPosition) #正解求第三个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
planeCSNode.coordinateJointPos = (0,0,0,0,0,0)
planeCSNode.coordinatePose = (0,0,0,0,0,0)
result=cgxiapi.cr_compute_cs_plane(planeCSNode.firstPoint.point.toolPosition, 6,
planeCSNode.secondPoint.point.toolPosition, 6, planeCSNode.thirdPoint.point.toolPosition,
6, planeCSNode.coordinatePose, 6) #计算面坐标系
if result.value != 0: #判断计算面坐标系是否成功
    exit()
result = cgxiapi.cr_kineInverse(robotHandle, planeCSNode.coordinatePose,
planeCSNode.coordinateJointPos, planeCSNode.coordinateJointPos)
#逆解求面坐标系关节角度
if result.value == 0: #判断逆解是否成功, 即面坐标系是否有效
    planeCSNode.isValid = 1
else:
    planeCSNode.isValid = 0
result = cgxiapi.cr_cfg_cs_plane_add(robotHandle,planeCSNode) #增加面坐标系
print("返回结果: ",result.value)
if result.value != 0: #判断增加面坐标系是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接, 释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")

```

`exit()`

3.6.15.17 删除面坐标系

函数

`cr_cfg_cs_plane_delete(robotHandle, index)`

简介

删除面坐标系。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	面坐标系索引（范围：0-9）

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
result=cgxiapi.cr_cfg_cs_plane_delete(robotHandle, index) #删除索引号为1的面坐标系
print("返回结果: ",result.value)
if result.value != 0: #判断删除面坐标系是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.18 修改面坐标系数据

函数

`cr_cfg_cs_plane_set(robotHandle, index, planeCSNode)`

简介

修改面坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	面坐标系索引（范围：0-9）
planeCSNode	PlaneCSNode	输入	面坐标系数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
planeCSNode = basestruct.PlaneCSNode()
planeCSNode.firstPoint.name = b'pp1'
planeCSNode.firstPoint.id = 1
planeCSNode.firstPoint.isValid = 1
planeCSNode.secondPoint.name = b'pp2'
planeCSNode.secondPoint.id = 2
planeCSNode.secondPoint.isValid = 1
planeCSNode.thirdPoint.name = b'pp3'
planeCSNode.thirdPoint.id = 3
planeCSNode.thirdPoint.isValid = 1
planeCSNode.name = b'plane1'
planeCSNode.firstPoint.point.toolAxisAngle = (0,0,90,0,-90,0)#第一个点的关节角度
planeCSNode.firstPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,planeCSNode.firstPoint.point.toolAxisAngle,
planeCSNode.firstPoint.point.toolPosition)#正解求第一个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
planeCSNode.secondPoint.point.toolAxisAngle = (0,4.84,84.91,0.25,-89.99,0)
#第二个点的关节角度
planeCSNode.secondPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,planeCSNode.secondPoint.point.toolAxisAngle,
planeCSNode.secondPoint.point.toolPosition)#正解求第二个点的位姿
if result.value != 0: #判断正解是否成功
    exit()
planeCSNode.thirdPoint.point.toolAxisAngle = (-8.1,-1.5,91.48,0.04,-90.07,-8.1)
#第三个点的关节角度
```

```

planeCSNode.thirdPoint.point.toolPosition = (0,0,0,0,0,0)
result=cgxiapi.cr_kineForward(robotHandle,planeCSNode.thirdPoint.point.toolAxisAngle,
planeCSNode.thirdPoint.point.toolPosition)#正解求第一个点的位姿
if result.value != 0:    #判断正解是否成功
    exit()
planeCSNode.coordinateJointPos = (0,0,0,0,0,0)
planeCSNode.coordinatePose = (0,0,0,0,0,0)
result=cgxiapi.cr_compute_cs_plane(planeCSNode.firstPoint.point.toolPosition, 6,
planeCSNode.secondPoint.point.toolPosition, 6, planeCSNode.thirdPoint.point.toolPosition, 6,
planeCSNode.coordinatePose, 6) #计算面坐标系
if result.value != 0:    #判断修改面坐标系数据是否成功
    exit()
result = cgxiapi.cr_kineInverse(robotHandle, planeCSNode.coordinatePose,
planeCSNode.coordinateJointPos, planeCSNode.coordinateJointPos)
#逆解求面坐标系位姿
if result.value == 0:    #判断逆解是否成功，即面坐标系是否有效
    planeCSNode.isValid = 1
else:
    planeCSNode.isValid = 0
index = 1
result = cgxiapi.cr_cfg_cs_plane_set(robotHandle,index,planeCSNode)
#修改索引号为 1 的面坐标系数据
print("返回结果：",result.value)
if result.value != 0:    #判断修改面坐标系数据是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.19 读取基坐标系数据

函数

```
cr_cfg_cs_base_get(robotHandle, csPose, len)
```

简介

读取基坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
csPose	POINTER(c_double)	输出	基坐标系数据
len	c_int	输入	数据长度,长度为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
csPose = [1,1,0,0,1,1]
len = 6
result=cgxiapi.cr_cfg_cs_base_get(robotHandle, csPose, len)
#读取基坐标系数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取基坐标系数据是否成功
    exit()
print("读取结果: ",csPose[0],csPose[1],csPose[2],csPose[3],csPose[4],csPose[5])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.15.20 读取工具坐标系数据

函数

```
cr_cfg_cs_tool_get(robotHandle, csPose, len)
```

简介

读取工具坐标系数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
csPose	POINTER(c_double)	输出	工具坐标系数据
len	c_int	输入	数据长度

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
```

```

re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
csPose = [1,1,0,1,0,1]
result=cgxiapi.cr_cfg_cs_tool_get(robotHandle, csPose, 6) #读取工具坐标系数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取工具坐标系数据是否成功
    exit()
print("读取结果: ",csPose[0],csPose[1],csPose[2],csPose[3],csPose[4],csPose[5])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.21 计算点坐标系

函数

```
cr_compute_cs_point(pointPose, pLen, pointCS, csLen)
```

简介

计算点坐标系。

参数

参数名称	数据类型	输入/输出	描述
pointPose	POINTER(c_double)	输入	计算点坐标系的位姿：x, y, z (单位：mm)；Rx, Ry, Rz (单位：°)
pLen	c_int	输入	位姿的长度，至少为 6
pointCS	POINTER(c_double)	输出	点坐标系的位姿：x, y, z (单 位：mm)；Rx, Ry, Rz (单 位：°)
csLen	c_int	输入	位姿的长度，至少为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```

pointPose = [-36.10, -4.42, 32.62, 13.54, -91.33, 39.52]
pointCS = [0, 0, 0, 0, 0, 0]
csLen = 6

```

```

pLen = 6
result=cgxiapi.cr_compute_cs_point(pointPose, pLen, pointCS, csLen) #计算点坐标系
print("返回结果: ",result.value)
if result.value != 0: #判断计算点坐标系是否成功
    exit()
i = 0
while i<6:
    print("计算结果: ", pointCS[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.22 计算线坐标系

函数

```
cr_compute_cs_line(point1Pose, p1Len, point2Pose, p2Len, lineCS, csLen)
```

简介

计算线坐标系。

参数

参数名称	数据类型	输入/输出	描述
point1Pose	POINTER(c_double)	输入	计算线坐标系的位姿 1: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
p1Len	c_int	输入	位姿的长度, 至少为 6
point2Pose	POINTER(c_double)	输入	计算线坐标系的位姿 2: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
p2Len	c_int	输入	位姿的长度, 至少为 6
lineCS	POINTER(c_double)	输出	线坐标系的位姿: x, y, z (单位: mm); Rx, Ry, Rz (单位: °)
csLen	c_int	输入	线坐标系的长度, 至少为 6

返回值

数据类型为 CRresult, 若为 success 或 0, 则表示成功其它均为失败。

示例

```

point1Pose = [-66, -438, 887, -57, 0, -148]
point2Pose = [-76, -438, 887, -57, 0, -148]

```

```

lineCS = [0, 0, 0, 0, 0, 0]
csLen = 6
p1Len = 6
p2Len = 6
result=cgxiapi.cr_compute_cs_line(point1Pose, p1Len, point2Pose, p2Len, lineCS, csLen)
#计算线坐标系
print("返回结果: ",result.value)
if result.value != 0:    #判断计算线坐标系是否成功
    exit()
i = 0
while i<6:
    print("计算结果: ", lineCS[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.15.23 计算面坐标系

函数

```

cr_compute_cs_plane(point1Pose, p1Len, point2Pose, p2Len, point3Pose, p3Len, planeCS,
csLen)

```

简介

计算面坐标系。

参数

参数名称	数据类型	输入/输出	描述
point1Pose	POINTER(c_double)	输入	计算面坐标系的位姿 1: x, y, z（单位: mm）; Rx, Ry, Rz（单位: °）
p1Len	c_int	输入	位姿的长度，至少为 6
point2Pose	POINTER(c_double)	输入	计算面坐标系的位姿 2: x, y, z（单位: mm）; Rx, Ry, Rz（单位: °）
p2Len	c_int	输入	位姿的长度，至少为 6
point3Pose	POINTER(c_double)	输入	计算面坐标系的位姿 3: x, y, z（单位: mm）; Rx, Ry, Rz（单位: °）
p3Len	c_int	输入	位姿的长度，至少为 6

planeCS	POINTER(c_double)	输出	面坐标系的位姿：x，y，z（单位：mm）；Rx，Ry，Rz（单位：°）
csLen	c_int	输入	面坐标系的长度，至少为 6

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功其它均为失败。

示例

```
point1Pose = [-70, -436.75, 893, 27.15, 22, 150.76]
point2Pose = [-75, -436.75, 893, 27.15, 22, 150.76]
point3Pose = [-75, -440, 893, 27.15, 22, 150.76]
planeCS = [0, 0, 0, 0, 0, 0]
csLen = 6
p1Len = 6
p2Len = 6
p3Len = 6
result=cgxiapi.cr_compute_cs_plane(point1Pose, p1Len, point2Pose, p2Len, point3Pose,
p3Len, planeCS, csLen) #计算面坐标系
print("返回结果：",result.value)
if result.value != 0: #判断计算面坐标系是否成功
    exit()
i = 0
while i<6:
    print("计算结果：", planeCS[i])
    i += 1
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.16 串口通讯

3.6.16.1 设置串口配置

函数

```
cr_cfg_comm_serial_setting_set(robotHandle, serialType, serialCommSettings)
```

简介

设置串口配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
serialType	SerialType	输入	串口类型
serialCommSettings	SerialCommSettings	输入	串口配置数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
serialType = basestruct.SerialType.rs485_1_CommSettings
serialCommSettings = basestruct.SerialCommSettings()
serialCommSettings.baudRate = basestruct.BaudRate.BaudRate115200
serialCommSettings.parity = basestruct.ParityCheck.odd
serialCommSettings.serialCommType = basestruct.SerialCommType.Modbus_ASCII
serialCommSettings.dataBits = 7
serialCommSettings.retryNumber = 2
serialCommSettings.timeout = 1000
serialCommSettings.stopBits = 1
serialCommSettings.modbusSlaveNo = 1
result=cgxiapi.cr_cfg_comm_serial_setting_set(robotHandle, serialType, serialCommSettings)
#设置串口配置
print("返回结果: ",result.value)
if result.value != 0: #判断设置串口配置是否成功
    exit()
print("设置结果:",serialType)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.16.2 读取串口配置

函数

```
cr_cfg_comm_serial_setting_get(robotHandle, serialType, serialCommSettings)
```

简介

读取串口配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
serialType	SerialType	输入	串口类型
serialCommSettings	POINTER(SerialCommSettings)	输出	串口配置数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
serialType = basestruct.SerialType.rs485_1_CommSettings
serialCommSettings = basestruct.SerialCommSettings()
result=cgxiapi.cr_cfg_comm_serial_setting_get(robotHandle, serialType, serialCommSettings)
#读取串口配置
print("返回结果: ",result.value)
if result.value != 0: #判断读取串口配置是否成功
    exit()
print("读取结果:",serialType)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.17 网络

3.6.17.1 设置控制柜网络配置

函数

```
cr_cfg_comm_ethernet_ip_set(robotHandle, ipconfig)
```

简介

设置控制柜网络配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
ipconfig	IPConfig	输入	网络地址配置

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
ipconfig = basestruct.IPConfig()
ipconfig.ipAddr[0] = 192
ipconfig.ipAddr[1] = 168
ipconfig.ipAddr[2] = 6
ipconfig.ipAddr[3] = 7
ipconfig.subnetMask[0] = 255
ipconfig.subnetMask[1] = 255
ipconfig.subnetMask[2] = 255
ipconfig.subnetMask[3] = 0
result=cgxiapi.cr_cfg_comm_ethernet_ip_set(robotHandle, ipconfig)#设置控制柜网络配置
print("返回结果: ",result.value)
if result.value != 0: #判断设置控制柜网络配置是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.17.2 读取控制柜网络配置

函数

```
cr_cfg_comm_ethernet_ip_get(robotHandle, ipconfig)
```

简介

读取控制柜网络配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
ipconfig	POINTER(IPConfig)	输出	网络地址配置

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
ipconfig = basestruct.IPConfig()
result=cgxiapi.cr_cfg_comm_ethernet_ip_get(robotHandle, ipconfig)#读取控制柜网络配置
print("返回结果: ",result.value)
if result.value != 0: #判断读取控制柜网络配置是否成功
    exit()
print("读取结果:",ipconfig.ipAddr[3])
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.17.3 设置控制柜从站配置

函数

```
cr_cfg_comm_ethernet_modbus_slave_num_set(robotHandle, modbusSlaveNo)
```

简介

设置控制柜从站配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
modbusSlaveNo	c_int	输入	从站号

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
modbusSlaveNo = 1
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
modbusSlaveNo = 1
result=cgxiapi.cr_cfg_comm_ethernet_modbus_slave_num_set(robotHandle, modbusSlaveNo)
#设置控制柜从站配置:将控制柜从站站号设为 1
print("返回结果: ",result.value)
if result.value != 0: #判断设置控制柜从站配置是否成功
    exit()
print("设置结果:",modbusSlaveNo)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.17.4 读取控制柜从站配置

函数

```
cr_cfg_comm_ethernet_modbus_slave_num_get(robotHandle)
```

简介

读取控制柜从站配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取结果。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
```

```
exit()
robotHandle = re1[1].value
result=cgxiapi.cr_cfg_comm_ethernet_modbus_slave_num_get(robotHandle)
#读取控制柜从站配置
print("返回结果: ",result[0].value)
if result[0].value != 0:    #判断读取控制柜从站配置是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18 modbus 主站

3.6.18.1 读取从站个数

函数

cr_cfg_comm_modbus_slave_count(robotHandle)

简介

读取从站个数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。同时返回读取从站个数。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs232_CommSettings
result = cgxiapi.cr_cfg_comm_modbus_slave_count(robotHandle,type)
#读取 rs232 从站个数
print("返回结果: ",result[0].value)
```

```

if result[0].value != 0:    #判断读取从站个数是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.18.2 读取从站数据

函数

```
cr_cfg_comm_modbus_slave_get(robotHandle, type, index, modbusConfig)
```

简介

读取从站数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	c_int	输入	从站索引（范围：0-19）
modbusConfig	POINTER(ModbusMasterConfig)	输出	从站配置信息

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
type = basestruct.SerialType.rs485_1_CommSettings
modbusConfig = basestruct.ModbusMasterConfig()
result=cgxiapi.cr_cfg_comm_modbus_slave_get(robotHandle, type, index, modbusConfig)
#读取 rs485_1 从站索引号为 1 的数据
print("返回结果: ",result.value)
if result.value != 0:    #判断读取从站数据是否成功
    exit()
print("读取结果:",modbusConfig.id)

```



```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.3 添加从站

函数

`cr_cfg_comm_modbus_slave_add(robotHandle, type, modbusConfig)`

简介

添加从站。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
modbusConfig	POINTER(ModbusMasterConfig)	输入	从站配置信息

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
modbusConfig = basestruct.ModbusMasterConfig()
modbusConfig.autoConnect = 0
modbusConfig.id = 1
modbusConfig.modbusMasterOperate = basestruct.ModbusMasterOperate.Stop
modbusConfig.scanTime = 10
modbusConfig.slaveNo = 1
result=cgxiapi.cr_cfg_comm_modbus_slave_add(robotHandle, type, modbusConfig)
#添加 rs485_1 的从站
print("返回结果：",result.value)
if result.value != 0:  #判断添加从站是否成功
    exit()
print("增加结果:",modbusConfig.id)
```

```
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.4 删除从站

函数

cr_cfg_comm_modbus_slave_delete(robotHandle, type, index)

简介

删除从站。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	c_int	输入	从站索引（范围：0-19）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 1
type = basestruct.SerialType.rs232_CommSettings
result=cgxiapi.cr_cfg_comm_modbus_slave_delete(robotHandle, type, index)
#删除 rs232 索引号为 1 的从站
print("返回结果：",result.value)
if result.value != 0:  #判断删除从站是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.5 修改从站数据

函数

`cr_cfg_comm_modbus_slave_set(robotHandle, type, index, modbusConfig)`

简介

修改从站数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	c_int	输入	从站索引（范围：0-19）
modbusConfig	ModbusMasterConfig	输入	从站配置信息

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
modbusConfig = basestruct.ModbusMasterConfig()
modbusConfig.autoConnect = 0
modbusConfig.id = 1
modbusConfig.modbusMasterOperate = basestruct.ModbusMasterOperate.Stop
modbusConfig.scanTime = 5
modbusConfig.slaveIP[0] = 192
modbusConfig.slaveNo = 1
index = 1
result=cgxiapi.cr_cfg_comm_modbus_slave_set(robotHandle, type, index, modbusConfig)
#修改 rs485_1 索引号为 1 的从站数据
print("返回结果: ",result.value)
if result.value != 0: #判断修改从站数据是否成功
    exit()
print("设置结果:",modbusConfig.scanTime)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
```

```
print("断开失败")
exit()
```

3.6.18.6 从站连接/断开

函数

```
cr_cfg_comm_modbus_slave_operate(robotHandle, type, index, modbusOperate)
```

简介

从站连接/断开。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	c_int	输入	从站索引（范围：0-19）
modbusOperate	ModbusMasterOperate	输入	从站操作

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
modbusConfig = basestruct.ModbusMasterConfig()
modbusOperate = basestruct.ModbusMasterOperate.Start
index = 1
result=cgxiapi.cr_cfg_comm_modbus_slave_operate(robotHandle, type, index, modbusOperate)
#连接 rs485_1 索引号为 1 的从站
print("返回结果: ",result.value)
if result.value != 0: #判断从站连接/断开是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.7 获取从站连接状态

函数

```
cr_cfg_comm_modbus_slave_connect_get(robotHandle, type, index, status)
```

简介

获取从站连接状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
index	c_int	输入	从站索引（范围：0-19）
status	POINTER(c_int)	输出	从站连接状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
status = c_int(0)
type = basestruct.SerialType.rs485_1_CommSettings
index = 0
result = cgxiapi.cr_cfg_comm_modbus_slave_connect_get(robotHandle, type, index, byref(status))
#获取 rs485_1 索引号为 0 的从站连接状态
print("返回结果：",result.value)
if result.value != 0: #判断获取从站连接状态是否成功
    exit()
print("从站连接状态：", status.value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.8 读取地址映射数量

函数

```
cr_cfg_comm_modbus_addr_map_count(robotHandle, type, serialIndex, count)
```

简介

读取地址映射数量。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	c_int	输入	从站索引（范围：0-19）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值：",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
serialIndex = 1
result = cgxiapi.cr_cfg_comm_modbus_addr_map_count(robotHandle, type, serialIndex)
#读取 rs485_1 索引号为 1 的从站地址映射数量
print("返回结果：",result[0].value)
if result[0].value != 0: #判断读取地址映射数量是否成功
    exit()
print("读取结果:",result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.9 读取地址映射数据

函数

```
cr_cfg_comm_modbus_addr_map_get(robotHandle, type, serialIndex, index, addrMap)
```

简介

读取地址映射数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	c_int	输入	从站索引（范围：0-19）
index	c_int	输入	地址映射索引（范围：0-99）
addrMap	POINTER(ModbusAddrMap)	输出	地址映射数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
addrMap = basestruct.ModbusAddrMap()
index = 1
serialIndex = 1
result=cgxiapi.cr_cfg_comm_modbus_addr_map_get(robotHandle, type, serialIndex, index,
addrMap) #读取 rs485_1 从站索引为 1，地址映射索引号为 1 的地址映射数据
print("返回结果: ",result.value)
if result.value != 0: #判断读取地址映射数据是否成功
    exit()
print("读取结果:",addrMap.slaveStartAddr)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.10 添加地址映射

函数

```
cr_cfg_comm_modbus_addr_map_add(robotHandle, type, serialIndex, addrMap)
```

简介

添加地址映射。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	c_int	输入	从站索引（范围：0-19）
addrMap	ModbusAddrMap	输入	地址映射数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs232_CommSettings
addrMap = basestruct.ModbusAddrMap()
addrMap.functionNo = basestruct.ModbusMasterFunctionNo.ReadInt16
addrMap.masterOffsetNo = 1
addrMap.masterRegType = basestruct.CommuVarType.CommuVarType_Float
addrMap.masterStartAddr = 1
addrMap.slaveStartAddr = 234
index = 1
result=cgxiapi.cr_cfg_comm_modbus_addr_map_add(robotHandle, type, index, addrMap)
#添加 rs232 从站索引号为 1 的地址映射
print("返回结果: ",result.value)
if result.value != 0: #判断添加地址映射是否成功
    exit()
print("添加结果:",addrMap.slaveStartAddr)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```


3.6.18.11 删除地址映射

函数

```
cr_cfg_comm_modbus_addr_map_delete(robotHandle, type, serialIndex, index)
```

简介

删除地址映射。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	c_int	输入	从站索引（范围：0-19）
index	c_int	输入	地址映射索引（范围：0-99）

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
index = 1
serialIndex = 1
result=cgxiapi.cr_cfg_comm_modbus_addr_map_delete(robotHandle, type, serialIndex, index)
#删除 rs485_1 从站索引号为 1，地址映射索引号为 1 的地址映射
print("返回结果: ",result.value)
if result.value != 0: #判断删除地址映射是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.18.12 修改地址映射数据

函数

```
cr_cfg_comm_modbus_addr_map_set(robotHandle, type, serialIndex, index, addrMap)
```

简介

修改地址映射数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
type	SerialType	输入	从站类型
serialIndex	c_int	输入	从站索引（范围 0-19）
index	c_int	输入	地址映射索引（范围：0-99）
addrMap	ModbusAddrMap	输入	地址映射数据

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
type = basestruct.SerialType.rs485_1_CommSettings
addrMap = basestruct.ModbusAddrMap()
addrMap.functionNo = basestruct.ModbusMasterFunctionNo.ReadInt16
addrMap.masterOffsetNo = 1
addrMap.masterRegType = basestruct.CommuVarType.CommuVarType_Float
addrMap.masterStartAddr = 2
addrMap.slaveStartAddr = 234
index = 1
serialIndex = 1
result=cgxiapi.cr_cfg_comm_modbus_addr_map_set(robotHandle, type, serialIndex, index,
addrMap) #修改 rs485_1 从站索引号为 1，地址映射索引号为 1 的地址映射数据
print("返回结果: ",result.value)
if result.value != 0: #判断修改地址映射数据是否成功
    exit()
print("设置结果:",addrMap.slaveStartAddr)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.19 力控

3.6.19.1 设置力控配置

函数

```
cr_force_cfg_set(robotHandle, forceConfig)
```

简介

设置力控配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
forceConfig	ForceConfig	输入	力控配置参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
forceConfig = basestruct.ForceConfig()
forceConfig.forceCtrlTcpImpl.activeForceTCP_name = b'TCP_4'
#激活的力控传感器 TCP 的名称
forceConfig.forceCtrlTcpImpl.activeForcePayload_name = b'ForcePayload7'
#激活的力控传感器负载的名称
forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpName = b'TCP_4'
forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpId = 0
forceConfig.forceCtrlTcpImpl.availableForceTCP[1].forceTcpName = b'TCP_8'
forceConfig.forceCtrlTcpImpl.availableForceTCP[1].forceTcpId = 1
forceTcpOffset = [0, 0, 0, 0, 0, -110]
for i in range(0, 6):
    forceConfig.forceCtrlTcpImpl.availableForceTCP[0].forceTcpOffset[i] = forceTcpOffset[i]
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forcePayloadName= b'ForcePayload7'
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forcePayloadId= 0
forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forceToolPayload = 1.3
forceCenterOfGravity = [1, 2, 3]
#当前待选的力控传感器负载的配置信息
```

```

for i in range(0, 3):
    forceConfig.forceCtrlTcpImpl.availableForcePayLoad[0].forceCenterOfGravity[i]=
    forceCenterOfGravity[i]
    forceConfig.forceCtrlTcpImpl.availableForceTCPLen = 2
    #当前待选的力控传感器 TCP 的配置信息的个数
    forceConfig.forceCtrlTcpImpl.availableForcePayLoadLen = 1
    #当前待选的力控传感器负载的配置信息的个数
    forceConfig.commuConfig.autoConnect = 1
    #自动与传感器通信，传感器开始接收数据
    forceConfig.commuConfig.mannaulOperate = 0
    #自动与传感器通信，传感器开始接收数据
    forceConfig.commuConfig.paraBuf = 0    #预留参数
    forceConfig.sensorMessage.venderNo = 0    #传感器品牌
    forceConfig.sensorMessage.productNo = 1    #传感器产品号，预留参数
    forceConfig.sensorMessage.sequenceNo = 0    #传感器序列号，预留参数
    result = cgxiapi.cr_force_cfg_set(robotHandle, forceConfig) #设置力控配置
    print("返回结果：", result.value)
    if result.value != 0:    #判断设置力控配置是否成功
        exit()
    re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
    if re0.value != 0:    #判断机器人断开连接是否成功
        print("断开失败")
        exit()

```

3.6.19.2 读力控配置

函数

```
cr_force_cfg_get(robotHandle, forceConfig)
```

简介

读力控配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
forceConfig	POINTER(ForceConfig)	输出	力控配置参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接

```

```

print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
forceConfig = basestruct.ForceConfig()
result = cgxiapi.cr_force_cfg_get(robotHandle, forceConfig)  #读力控配置
if result.value != 0:    #判断读力控配置是否成功
    exit()
print(forceConfig.commuConfig.autoConnect)
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.19.3 开启力控并下发相关设置

函数

```
cr_force_open(robotHandle, forceSetting)
```

简介

开启力控并下发相关设置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
forceSetting	ForceSetting	输入	力控设置参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
forceSetting = basestruct.ForceSetting()
forceSetting.forceBaseType = 0  #基于 TCP 力传感器
forceSetting.forceCtrlType = 1  #力控拖动
flexibleAxis = [0, 0, 1, 0, 0, 0]  #z 轴可拖动

```

```

i=0
while i<6:
    forceSetting.flexibleAxis[i] = flexibleAxis[i]
    i += 1
result = cgxiapi.cr_force_open(robotHandle, forceSetting) #开启力控并下发相关设置
print("返回结果: ", result.value)
if result.value != 0: #判断开启力控并下发相关设置是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接, 释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.19.4 关闭力控

函数

cr_force_close(robotHandle)

简介

关闭力控。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_force_close(robotHandle) #关闭力控
print("返回结果: ", result.value)
if result.value != 0: #判断关闭力控是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接, 释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")

```

exit()

3.6.19.5 设置力控控制参数

函数

cr_force_para_set(robotHandle, forceCtlPara)

简介

设置力控控制参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
forceCtlPara	ForceCtlPara	输入	力控控制参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
forceCtlPara = basestruct.ForceCtlPara()
taskFrame = [0, 0, 0, 0, 0, 0] #力控坐标系为基坐标系
wrench = [0, 0, -10, 0, 0, 0] #z 轴负向施加 10N 的力
limits = [0.1,0.1,0.005,0.002,0.002,0.002]
#x,y 方向速度保护为 0.1m/s, z 方向速度保护为 0.005m/s,Rx,Ry,Rz 偏差保护为 0.002rad
mass = [1, 1, 0.5, 1, 1, 1] #x,y,Rx,Ry,Rz 质量系数均为 1,z 质量系数为 0.5
damping = [1, 1, 0.1, 1, 1, 1] #x,y,Rx,Ry,Rz 阻尼系数均为 1,z 阻尼系数为 0.1
stiffness = [1, 1, 0.1, 1, 1, 1] #x,y,Rx,Ry,Rz 刚度均为 1,z 刚度为 0.1
forceCtlPara.limitsLen = 6 #0、1 或 6
forceCtlPara.massLen = 6 #0、1 或 6
forceCtlPara.dampingLen = 6 #0、1 或 6
forceCtlPara.stiffnessLen = 6 #0、1 或 6
i=0
while i<6:
    forceCtlPara.taskFrame[i] = taskFrame[i]
    forceCtlPara.limits[i] = limits[i]
    forceCtlPara.mass[i] = mass[i]
```

```
        forceCtlPara.damping[i] = damping[i]
        forceCtlPara.stiffness[i] = stiffness[i]
        i+=1
result = cgxiapi.cr_force_para_set(robotHandle,forceCtlPara) #设置力控控制参数
print("返回结果: ", result.value)
if result.value != 0: #判断设置力控控制参数是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.19.6 读力控控制参数

函数

```
cr_force_para_get(robotHandle, forceCtlPara)
```

简介

读力控控制参数。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
forceCtlPara	POINTER(ForceCtlPara)	输出	力控控制参数

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
forceCtlPara = basestruct.ForceCtlPara()
result = cgxiapi.cr_force_para_get(robotHandle, forceCtlPara) #读力控控制参数
print("返回结果: ", result.value)
if result.value != 0: #判断读力控控制参数是否成功
    exit()
i=0
while i < 6:
```



```
print(forceCtlPara.limits[i])
i+=1
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.19.7 读力控数据

函数

```
cr_force_data_get(robotHandle, forceData)
```

简介

读力控数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
forceData	POINTER(ForceData)	输出	力控数据

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")  #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0:  #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
forceData = basestruct.ForceData()
result = cgxiapi.cr_force_data_get(robotHandle, forceData)  #读力控数据
print("返回结果: ", result.value)
if result.value != 0:  #判断读力控数据是否成功
    exit()
i = 0
while i < 6:
    print(forceData.ftCtrl[i])
re0 = cgxiapi.cr_destroy_robot(robotHandle)  #机器人断开连接，释放句柄
if re0.value != 0:  #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.20 扩展轴

3.6.20.1 设置扩展轴使能状态

函数

```
cr_set_exj_enableStatus(robotHandle, index, status)
```

简介

设置扩展轴使能状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	扩展轴索引
status	c_int	输入	扩展轴状态

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
status = 1
result = cgxiapi.cr_set_exj_enableStatus(robotHandle, index, status) #设置扩展轴使能状态
print("返回结果: ", result.value)
if result.value != 0: #判断设置扩展轴使能状态是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.20.2 读取扩展轴使能状态

函数

```
cr_get_exj_enableStatus(robotHandle, index)
```

简介

读取扩展轴使能状态。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
index	c_int	输入	扩展轴索引

返回值

返回结果是长度为 2 的一维数组，第一个是 CRresult 类型的值，第二个是扩展轴状态。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
index = 0
result = cgxiapi.cr_get_exj_enableStatus(robotHandle, index) #读取扩展轴使能状态
print("返回结果: ", result[0].value)
if result[0].value != 0: #判断读取扩展轴使能状态是否成功
    exit()
print("扩展轴状态: ", result[1].value)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.20.3 读取机械臂和扩展轴数据

函数

```
cr_get_robot_exj_data(robotHandle, robotExjData)
```

简介

读取机械臂和扩展轴数据。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
robotExjData	RobotExjData	输入	待读取的值

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
robotExjData = basestruct.RobotExjData()
result = cgxiapi.cr_get_robot_exj_data(robotHandle, robotExjData ) #读取扩展轴使能状态
print("返回结果: ", result.value)
if result.value != 0: #判断读取扩展轴使能状态是否成功
    exit()
print("扩展轴实际位置: ", robotExjData.actualExjointPos)
print("实际关节角度: ", robotExjData.actualJointPos)
print("实际 TCP 位姿: ", robotExjData.actualTcpVector)
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.20.4 获取扩展轴配置

函数

```
cr_cfg_exj_get(robotHandle, exjName, exjConfig)
```

简介

获取扩展轴配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
exjName	c_char_p	输入	扩展轴名称
exjConfig	ExjConfig	输出	扩展轴配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
exjName = b"exj_test03"
exjConfig = basestruct.ExjConfig()
result = cgxiapi.cr_cfg_exj_get(robotHandle, exjName, exjConfig) #获取扩展轴配置
print("返回结果: ", result.value)
if result.value != 0: #判断获取扩展轴配置是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle) #机器人断开连接，释放句柄
if re0.value != 0: #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```

3.6.20.5 设置扩展轴配置

函数

```
cr_cfg_exj_set(robotHandle, exjName, exjConfig)
```

简介

设置扩展轴配置。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
exjName	c_char_p	输入	扩展轴名称
exjConfig	ExjConfig	输入	扩展轴配置

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```
robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123") #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)
if re1[0].value != 0: #判断机器人连接是否成功
    print("连接失败")
```

```

    exit()
robotHandle = re1[1].value
exjConfig = basestruct.ExjConfig()
exjConfig.name = b"exj_test03"
exjConfig.type = 1
exjConfig.ratio = 200    #传动比(pulse/mm)
exjConfig.minlimit = -10000    #最小值
exjConfig.maxlimit = 10000    #最大值
exjConfig.zeroPosition = 0    #零位(pulse)
exjConfig.exjToRef = (c_double * basestruct.ROB_AXIS_NUM)(*[0.0] * basestruct.ROB_AXIS_NUM))
exjConfig.masterExjName = b"exj_test01"    #主轴 ID
exjConfig.masterSlaveRate = 10    #主从比
exjName = b"exj_test04"
result = cgxiapi.cr_cfg_exj_set(robotHandle, exjName, exjConfig)    #设置扩展轴配置
print("返回结果: ", result.value)
if result.value != 0:    #判断设置扩展轴配置是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.20.6 增加扩展轴

函数

```
cr_cfg_exj_add(robotHandle, exjConfig)
```

简介

增加扩展轴。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄
exjConfig	ExjConfig	输入	扩展轴配置

返回值

数据类型为 CResult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)

```

```

if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
exjConfig = basestruct.ExjConfig()
exjConfig.name = b"exj_test03"
exjConfig.type = 1
exjConfig.ratio = 200    #传动比(pulse/mm)
exjConfig.minlimit = -10000    #最小值
exjConfig.maxlimit = 10000    #最大值
exjConfig.zeroPosition = 0    #零位(pulse)
exjConfig.exjToRef = (c_double * basestruct.ROB_AXIS_NUM)(*[0.0] * basestruct.ROB_AXIS_NUM))
exjConfig.masterExjName = b"exj_test01"    #主轴 ID
exjConfig.masterSlaveRate = 10    #主从比
result = cgxiapi.cr_cfg_exj_add(robotHandle, exjConfig)    #增加扩展轴
print("返回结果: ", result.value)
if result.value != 0:    #判断增加扩展轴是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()

```

3.6.20.7 删除扩展轴

函数

```
cr_cfg_exj_delete(robotHandle)
```

简介

删除扩展轴。删除扩展轴只能删除最后一个。

参数

参数名称	数据类型	输入/输出	描述
robotHandle	c_int	输入	机械臂实例句柄

返回值

数据类型为 CRresult，若为 success 或 0，则表示成功，其它均为失败。

示例

```

robotHandle = 1
re1=cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")    #机械臂连接
print("机械臂建立连接返回值: ",re1[0].value)

```

```
if re1[0].value != 0:    #判断机器人连接是否成功
    print("连接失败")
    exit()
robotHandle = re1[1].value
result = cgxiapi.cr_cfg_exj_delete(robotHandle)    #删除扩展轴
print("返回结果: ", result.value)
if result.value != 0:    #判断删除扩展轴是否成功
    exit()
re0 = cgxiapi.cr_destroy_robot(robotHandle)    #机器人断开连接，释放句柄
if re0.value != 0:    #判断机器人断开连接是否成功
    print("断开失败")
    exit()
```


四、综合示例

4.1 机械臂使能

简介

对机械臂进行操作需要使机械臂处于使能状态。本示例的功能为：在机械臂处于未上电状态时，通过 sdk 接口使机械臂上电使能完成。

示例

```
import cgxiapi
from ctypes import *
import time
import basestruct
import inspect
# 机械臂创建连接
robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123") #虚拟臂连接
if re0[0].value == 0:
    robotHandle = re0[1].value
    re1 = cgxiapi.cr_get_robotMode(robotHandle) #读机械臂当前状态
    if re1[0].value==0 and re1[1].value==6 :
        #判断状态,robotMode 为 6 时是本体未上电状态才可以进行上电
        result = cgxiapi.cr_poweron(robotHandle) #机械臂上电
        if result.value == 0:
            while result.value == 0:
                re1 = cgxiapi.cr_get_robotMode(robotHandle) # 读机械臂当前状态
                time.sleep(0.05)
                if re1[1].value == 8:
                    break
            result = cgxiapi.cr_enable(robotHandle)
            if result.value == 0:
                while result.value == 0:
                    re1 = cgxiapi.cr_get_robotMode(robotHandle) # 读机械臂当前状态
                    time.sleep(0.05)
                    if re1[1].value == 103:
                        break
            print("使能完成")
        else:
            print("行数: ",inspect.currentframe().f_lineno," ", "结果: ",re1[0].value)
    else:
        print("行数: ",inspect.currentframe().f_lineno," ", "结果: ",result.value)
```

```

else:
    print("行数: ",inspect.currentframe().f_lineno," ", "结果: ",re1[0].value)
else:
    print("行数: ",inspect.currentframe().f_lineno," ", "结果: ",re0[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()

```

4.2 工程控制

简介

本示例介绍了对工程进行控制的方法，包括下载、上传、运行工程。

注：本示例中下载的工程会将机械臂的 4、5、6 关节各运动 5°，故使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞，以及是否会超出关节角度范围限值。

示例

```

import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

# 机械臂创建连接
robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123") #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    scriptstatus = basestruct.Lua_ScriptStatus
    re1 = cgxiapi.cr_get_lua_scriptstatus(robotHandle) # 读脚本运行状态
    if re1[0].value==0 and re1[1].value==1:
        #当脚本运行状态处于脚本程序停止时才可进行程序下载
        crpFilepathname = b"./program/demo.crp"
        crscriptFilepathname = b"./program/demo.crscrip"
        re2 = cgxiapi.cr_downloadProject(robotHandle, crpFilepathname, crscriptFilepathname)
        #下载程序
        if re2.value==0:
            re3=cgxiapi.cr_play(robotHandle) #运行程序
            while re3.value==0:
                result = cgxiapi.cr_get_robotMode(robotHandle)
                if result[1].value == 103:

```

```

#读机械臂当前状态,程序停止状态下才可以上传程序
        break
    if result[0].value==0:
        FilePath = b"./program/"
        filename = b"test"    #crp 和脚本程序的文件名
        re4 = cgxiapi.cr_uploadProject(robotHandle, FilePath, filename)
#工程文件上传
        if re4.value==0:
            print("行数： ", inspect.currentframe().f_lineno, "已成功上传工程文件")
        else:
            print(" 行数： ", inspect.currentframe().f_lineno, " ", "结果： ",
re4.value)
        else:
            print(" 行数： ", inspect.currentframe().f_lineno, " ", "结果： ",
result[0].value)
        else:
            print("行数： ", inspect.currentframe().f_lineno, " ", "结果： ", re2.value)
        else:
            print("行数： ", inspect.currentframe().f_lineno, " ", "结果： ", re1[0].value)
            print("脚本运行状态： ",re1[1].value)
    else:
        print("行数： ", inspect.currentframe().f_lineno, " ", "结果： ", re0[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()

```

4.3 移动控制

简介

本示例介绍了 moveJ、moveL、cr_move_joint、cr_move_line 四个接口的使用方法，moveJ 和 moveL 是非阻塞移动，cr_move_joint 和 cr_move_line 可以设置是否需要阻塞移动。

注：使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞。

示例

```

import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

```

```

# 机械臂创建连接

```

```

robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123")  #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    pointControlPara = basestruct.PointControlPara()
    pointControlParaSimple = basestruct.PointControlParaSimple()
    speed = 30
    acc = 60
    jointpos = ( 0, 0, 90, 0, -90, 0)
    pointControlPara.jointpos =
(jointpos[0],jointpos[1],jointpos[2],jointpos[3],jointpos[4],jointpos[5])  #目标点示例数据
    pointControlPara.pose = (0, 0, 0, 0, 0, 0)  #在 MoveJ 中无影响
    pointControlPara.tcpOffset = (0, 0, 0, 0, 0, 0)
    pointControlPara.tcpID = -1
    pointControlPara.speed = (speed, speed, speed, speed, speed, speed)
    pointControlPara.acc = (acc, acc, acc, acc, acc, acc)
    pointControlPara.coordinatePose = (0, 0, 0, 0, 0, 0)
    pointControlPara.jerk = (0, 0, 0, 0, 0, 0)
    pointControlPara.coordinateType = basestruct.CoordinateType.jointCoordinate
    pointControlPara.pointTransType = basestruct.PointTransType.pointTransStop
    pointControlPara.pointTransRadius = 0
    pointControlPara.poseTranType = basestruct.PoseTranType.poseTranMoveToTargetPose
    pointControlPara.motiontriggerMode =
basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
    result_movej = cgxiapi.cr_moveJ(robotHandle, pointControlPara) #轴空间运动
    print("返回结果:", result_movej.value)
    if result_movej.value == 0:
        time.sleep(0.5)
        result_moveStatus = cgxiapi.cr_get_robotMoveStatus(robotHandle)
        while result_moveStatus[0].value==0:  #读机械臂运动状态,如果运动已经停止,代表
        上一次运动已经结束,则跳出循环进行下一次运动
            result_moveStatus = cgxiapi.cr_get_robotMoveStatus(robotHandle)
            if result_moveStatus[1].value==0:
                break
        pose = [0, 0, 0, 0, 0, 0]
        result_get = cgxiapi.cr_get_tcpActualPose(robotHandle, pose)  # 读实际 TCP 位置
        pose[2] = pose[2]-150
        if result_get.value == 0:
            pointControlPara.coordinateType = basestruct.CoordinateType.baseCoordinate
            pointControlPara.pose = (pose[0],pose[1],pose[2],pose[3],pose[4],pose[5])
            pointControlPara.jointpos=
(jointpos[0],jointpos[1],jointpos[2],jointpos[3],jointpos[4],jointpos[5])
            result_moveL = cgxiapi.cr_moveL(robotHandle, pointControlPara)

```

```

        if result_move1.value == 0:
            time.sleep(0.05)
            result_moveStatus = cgxiapi.cr_get_robotMoveStatus(robotHandle)
            while result_moveStatus[0].value == 0: # 读机械臂运动状态,如果运动已经停止,代表上一次运动已经结束,则跳出循环进行下一次运动
                result_moveStatus = cgxiapi.cr_get_robotMoveStatus(robotHandle)
                if result_moveStatus[1].value == 0:
                    break
                pose[2] = pose[2] + 150
                pointControlParaSimple.pose = (pose[0], pose[1], pose[2], pose[3], pose[4],
pose[5])
                pointControlParaSimple.jointpos = (jointpos[0], jointpos[1], jointpos[2],
jointpos[3], jointpos[4], jointpos[5])
                pointControlParaSimple.speed = (speed, speed, speed, speed, speed, speed)
                pointControlParaSimple.acc = (acc, acc, acc, acc, acc, acc)
                pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
                pointControlParaSimple.tcpID = -1
                pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
                pointControlParaSimple.coordinateType=
basestruct.CoordinateType.baseCoordinate
                pointControlParaSimple.pointTransType=
basestruct.PointTransType.pointTransStop
                pointControlParaSimple.motiontriggerMode=
basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
                result_trans = cgxiapi.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple,pointControlPara)
#PointControlParaSimple 数据转换为 PointControlPara
                isBlock = 1
                result_moveline = cgxiapi.cr_move_line(robotHandle, pointControlPara,
isBlock) #阻塞直线运动
                if result_moveline.value == 0:
                    jointpos2 = (90, 0, 90, 0, -90, 0)
                    pointControlParaSimple.jointpos=(jointpos2[0],jointpos2[1],
jointpos2[2], jointpos2[3], jointpos2[4], jointpos2[5])
                    pointControlParaSimple.pose = (0, 0, 0, 0, 0, 0)
                    pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
                    pointControlParaSimple.tcpID = -1
                    pointControlParaSimple.speed = (speed, speed, speed, speed, speed,
speed)
                    pointControlParaSimple.acc = (acc, acc, acc, acc, acc, acc)
                    pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
                    pointControlParaSimple.coordinateType=
basestruct.CoordinateType.jointCoordinate
                    pointControlParaSimple.pointTransType=
basestruct.PointTransType.pointTransStop

```

```

        pointControlParaSimple.motiontriggerMode=
basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
        result_trans = cgxiapi.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple,pointControlPara)
#PointControlParaSimple 数据转换为 PointControlPara
        isBlock = 1
        result_movejoint=cgxiapi.cr_move_joint(robotHandle, pointControlPara,
isBlock)  #阻塞式的轴空间运动，由于是阻塞的轴空间运动，则运动会在运动到位后在进行
后续的程序运行，因此不用像之前做判断
        if result_movejoint.value==0:
            result = cgxiapi.cr_get_tcpActualPose(robotHandle, pose)
#读实际 TCP 位置
            pose[2] = pose[2]-150
            pointControlParaSimple.coordinateType=
basestruct.CoordinateType.baseCoordinate
            pointControlParaSimple.pose = (pose[0], pose[1], pose[2], pose[3],
pose[4], pose[5])
            pointControlParaSimple.jointpos = (jointpos2[0], jointpos2[1],
jointpos2[2], jointpos2[3], jointpos2[4], jointpos2[5])
            result_trans=
cgxiapi.cr_move_pointControlPara_transfer(robotHandle,pointControlParaSimple,
pointControlPara)  #PointControlParaSimple 数据转换为 PointControlPara
            result_move1 = cgxiapi.cr_moveL(robotHandle, pointControlPara)
#直线运动
            if result_move1.value == 0:
                time.sleep(0.05)
                result_moveStatus=
cgxiapi.cr_get_robotMoveStatus(robotHandle)
                while result_moveStatus[0].value == 0:
#读机械臂运动状态,如果运动已经停止，代表上一次运动已经结束，则跳出循环进行下一
次运动
                    result_moveStatus=
                    cgxiapi.cr_get_robotMoveStatus(robotHandle)
                    if result_moveStatus[1].value == 0:
                        break
                    pose[2] = pose[2] + 150
                    pointControlParaSimple.pose = (pose[0], pose[1], pose[2],
pose[3], pose[4], pose[5])
                    pointControlParaSimple.jointpos = (jointpos2[0], jointpos2[1],
jointpos2[2], jointpos2[3], jointpos2[4], jointpos2[5])
                    result_trans=
cgxiapi.cr_move_pointControlPara_transfer(robotHandle,pointControlParaSimple,
pointControlPara)  #PointControlParaSimple 数据转换为 PointControlPara
                    isBlock = 0
                    result = cgxiapi.cr_move_line(robotHandle, pointControlPara,

```

```

isBlock)    #非阻塞式直线运动，下发运动后会立刻执行下一条指令
            if result.value == 0:
                print("运动已完成")
            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result.value)

            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_move1.value)

            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_movejoint.value)

            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_moveline.value)

            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_move1.value)

            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_get.value)

            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_movej.value)

        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", re0[0].value)
    result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
    if result_destroy.value!=0:
        print("断开失败")
        exit()

```

4.4 轨迹再现

简介

本示例介绍了记录一段轨迹并把轨迹下载到控制柜中的方法。

注：本示例将轨迹下载到索引 2 中，如果控制器中已存在索引为 2 的轨迹，将新记录的轨迹下载到索引 2 后，原轨迹将会被覆盖。使用本示例前，需注意机械臂运动过程中是否会和其他物体发生碰撞。

示例

```

import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

```

```

# 机械臂创建连接
robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123")  #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    pointControlParaSimple = basestruct.PointControlParaSimple()
    pointControlPara = basestruct.PointControlPara()
    speed = 90
    acc = 200
    pointControlParaSimple.jointpos = (0, 0, 90, 0, -90, 0)
    pointControlParaSimple.pose = (0, 0, 0, 0, 0, 0)
    pointControlParaSimple.tcpOffset = (0, 0, 0, 0, 0, 0)
    pointControlParaSimple.tcpID = -1
    pointControlParaSimple.speed = (speed, speed, speed, speed, speed, speed)
    pointControlParaSimple.acc = (acc, acc, acc, acc, acc, acc)
    pointControlParaSimple.coordinatePose = (0, 0, 0, 0, 0, 0)
    pointControlParaSimple.coordinateType = basestruct.CoordinateType.jointCoordinate
    pointControlParaSimple.pointTransType = basestruct.PointTransType.pointTransStop
    pointControlParaSimple.motiontriggerMode=
basestruct.MotiontriggerMode.MovetriggerbyOnlyRpc
    result_trans1=cgxiapi.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple, pointControlPara)
#PointControlParaSimple 数据转换为 PointControlPara
    isBlock = 1
    re1 = cgxiapi.cr_move_joint(robotHandle, pointControlPara, isBlock)  #轴空间运动
    #轨迹
    recordPathPara = basestruct.RecordPathPara()
    recordPathPara.recordControl = 1  # 记录控制, 0-停止, 1-启动, 2-暂停
    recordPathPara.sampleTime = 2
    result_set = cgxiapi.cr_path_recordPara_set(robotHandle, recordPathPara)
#设置轨迹记录参数, 开启轨迹记录
    time.sleep(0.5)
    pointControlParaSimple.jointpos = (90, 0, 90, 0, -90, 0)
    result_trans2=cgxiapi.cr_move_pointControlPara_transfer(robotHandle,
pointControlParaSimple,pointControlPara)
# PointControlParaSimple 数据转换为 PointControlPara
    isBlock = 1
    if result_set.value==0:
        re2 = cgxiapi.cr_move_joint(robotHandle, pointControlPara, isBlock)  # 轴空间运动
        if re2.value==0:
            recordPathPara.recordControl = 1  # 记录控制, 0-停止, 1-启动, 2-暂停

```



```

        result_set = cgxiapi.cr_path_recordPara_set(robotHandle, recordPathPara)
# 结束轨迹记录
        if result_set.value==0:
            index = -1 # 轨迹索引
            pathData = basestruct.PathData()
            stru_info = create_string_buffer(sizeof(basestruct.PathPoint) * 10000)
            pathData.pathPoints = POINTER(basestruct.PathPoint)(stru_info)
            result_upload = cgxiapi.cr_path_upload(robotHandle, index, pathData)
#上传轨迹
            if result_upload.value==0:
                result_get = cgxiapi.cr_path_all_index_get(robotHandle)
#读取控制柜当前存在的轨迹索引
                i=0
                while i<result_get[2].value:
                    if(result_get[1][i] == 2):
#将下载的轨迹索引设置为 2，故需要判断其是否已经存在
                        print("行数：", inspect.currentframe().f_lineno, "存在轨迹索引为 2 的轨迹")

                        pathDownloadData = basestruct.PathDownloadData()
                        pathDownloadData.pathData = pathData
                        pathDownloadData.pathPara.index = 2
                        pathDownloadData.pathPara.moveType = 1
                        result_download = cgxiapi.cr_path_download(robotHandle,
pathDownloadData) #下载轨迹到索引 2
                        if result_download.value==0:
                            print("行数：", inspect.currentframe().f_lineno, "已成功下载索引为 2 的轨迹")
                        else:
                            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_download.value)
                            break
                        if(i==result_get[2].value-1):
                            pathDownloadData = basestruct.PathDownloadData()
                            pathDownloadData.pathData = pathData
                            pathDownloadData.pathPara.index = 2
                            pathDownloadData.pathPara.moveType = 1
                            result_download = cgxiapi.cr_path_download(robotHandle,
pathDownloadData) # 下载轨迹到索引 2
                            if result_download.value == 0:
                                print("行数：", inspect.currentframe().f_lineno, "已成功下载索引为 2 的轨迹")
                            else:
                                print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_download.value)

```

```

        i += 1
    else:
        print(" 行数：", inspect.currentframe().f_lineno, "    ", "结果：",
result_upload.value)
    else:
        print(" 行数：", inspect.currentframe().f_lineno, "    ", "结果：",
result_set.value)
    else:
        print("行数：", inspect.currentframe().f_lineno, "    ", "结果：", re2.value)
    else:
        print("行数：", inspect.currentframe().f_lineno, "    ", "结果：", result_set.value)
else:
    print("行数：", inspect.currentframe().f_lineno, "    ", "结果：", re0[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()

```

4.5 安装变量操作

简介

本示例介绍了与安装变量相关的接口的使用方法，包括读取安装变量个数、读取安装变量、修改安装变量、增加安装变量。

示例

```

import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

# 机械臂创建连接
robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123") #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    variableMsg = basestruct.VariableMsg()
    variableMsg.variableName = b'var_11'
    variableMsg.variableType = 3 #number
    variableMsg.numberValue = 22
    variableMsg.variableID = 8
    result_count = cgxiapi.cr_cfg_var_install_count(robotHandle)

```

```

if result_count[0].value==0:
    if result_count[1].value>0:
        i=0
        while i<result_count[1].value:
            variable_get = basestruct.VariableMsg()
            result = cgxiapi.cr_cfg_var_install_get(robotHandle, i, variable_get)
            if variable_get.variableName==variableMsg.variableName:
                #判断是否存在我想添加的同名安装变量，如果有就修改，没有就增加
                result_set = cgxiapi.cr_cfg_var_install_set(robotHandle, i, variableMsg)
                if result_set.value==0:
                    print("行数：", inspect.currentframe().f_lineno, " 已成功修改安
装变量")
                else:
                    print("行数：", inspect.currentframe().f_lineno, " ", "结果：",
result_set.value)
                    break
            if i==(result_count[1].value-1):
                result_add = cgxiapi.cr_cfg_var_install_add(robotHandle, variableMsg)
                if result_add.value == 0:
                    print("行数：", inspect.currentframe().f_lineno, " 已成功增加安
装变量")
                else:
                    print("行数：", inspect.currentframe().f_lineno, " ", "结果：",
result_add.value)
            i+=1
        else:
            result_add = cgxiapi.cr_cfg_var_install_add(robotHandle, variableMsg)
            if result_add.value==0:
                print("行数：", inspect.currentframe().f_lineno, " 已成功增加安装变量")
            else:
                print("行数：", inspect.currentframe().f_lineno, " ", "结果：",
result_add.value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_count[0].value)
    else:
        print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_count[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()

```

4.6 点坐标系操作

简介

本示例介绍了与点坐标系操作相关的接口的使用方法，包括读取点坐标系个数、读取点坐标系、修改安装变量、增加安装变量、删除点坐标系。

示例

```
import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

# 机械臂创建连接
robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325, "123") #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    pointCSNode = basestruct.PointCSNode()
    pointCSNode.name = b'point3'
    pointCSNode.point.toolAxisAngle = (30, 30, 30, 30, 30, 30)
    pointCSNode.point.toolPosition = (0, 0, 0, 0, 0, 0)
    result_count = cgxiapi.cr_cfg_cs_point_count(robotHandle) #读取点坐标系个数
    Pcount = result_count[1].value
    if result_count[0].value==0:
        if Pcount>0:
            i=0
            while i<Pcount:
                pointCS = basestruct.PointCSNode()
                result_get = cgxiapi.cr_cfg_cs_point_get(robotHandle, i, pointCS)
                if pointCS.name==pointCSNode.name:
                    result_kineF=cgxiapi.cr_kineForward(robotHandle,
pointCSNode.point.toolAxisAngle, pointCSNode.point.toolPosition) #正解求点的位姿
                    if result_kineF.value==0:
                        pointCSNode.isValid = 1
                        pointCSNode.id = 8
                        result_set = cgxiapi.cr_cfg_cs_point_set(robotHandle, i,
pointCSNode) #设置点坐标系
                        if result_set.value==0:
                            print("行数：", inspect.currentframe().f_lineno, " 修改成功")
                        else:
```

```

        print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ",
", result_set.value)
    else:
        print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ",
result_kineF.value)
        break
    if i==Pcount-1 :
        result_kineF=cgxiapi.cr_kineForward(robotHandle,
pointCSNode.point.toolAxisAngle,pointCSNode.point.toolPosition) #正解求点的位姿
        if result_kineF.value==0:
            pointCSNode.isValid = 1
            pointCSNode.id = 8
            result_add=cgxiapi.cr_cfg_cs_point_add(robotHandle,
pointCSNode) # 增加点坐标系数据
            if result_add.value == 0:
                print("行数: ", inspect.currentframe().f_lineno, "增加成功")
            else:
                print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ",
result_add.value)
                i+=1
        else:
            pointCSNode.isValid = 1
            pointCSNode.id = 1
            result_kineF=cgxiapi.cr_kineForward(robotHandle,
pointCSNode.point.toolAxisAngle,pointCSNode.point.toolPosition) # 正解求点的位姿
            result_add = cgxiapi.cr_cfg_cs_point_add(robotHandle, pointCSNode)
#增加点坐标系数据
            if result_add.value==0:
                print("行数: ", inspect.currentframe().f_lineno, " 增加成功")
            else:
                print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ",
result_add[0].value)
                index = 0
                result_get = cgxiapi.cr_cfg_cs_point_get(robotHandle, index, pointCSNode)
#读取索引号为 0 的点坐标系数据,可能不存在该坐标系
                if result_get.value==0:
                    print(pointCSNode.name)
                    index = 0
                    result_delete = cgxiapi.cr_cfg_cs_point_delete(robotHandle, index)
#删除索引号为 0 的点坐标系
                    if result_delete.value==0:
                        print("行数: ", inspect.currentframe().f_lineno, " 已成功删除索引号为 0
的坐标系")
                    else:

```

```

        print(" 行数 : ", inspect.currentframe().f_lineno, " ", " 结果 : ",
result_delete.value)
    else:
        print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ", result_get.value)
    else:
        print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ", result_count[0].value)
else:
    print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ", re0[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()

```

4.7 输出弹窗

简介

本示例介绍了读取输出弹窗的方法，首先通过加载运行程序来创建一个输出弹窗，再判断弹窗是否存在，若存在则读取输出弹窗信息。

示例

```

import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

# 机械臂创建连接
robotHandle = 1
# re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123") #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    programfile = ("function mainFuncProgram()\n"
        "--start robotConfig\n"
        "TCP_1 = { 0,0,0,0,0,0 }\n"
        "Payload_1= { 0,{0,0,0} }\n"
        "set_tcp_payload(Payload_1[1], Payload_1[2])\n"
        "--end robotConfig\n"
        "function RobotProgram()\n"
        "while (true)\n"
        "do\n"
        "--popup_out:text\n"
        "popup_message(\"hello\", \"message\", false, false, true)\n"

```

```

        "wait(1)  --sync\n"
        "end\n"
        "end\n"
        "RobotProgram_Result = task_create(RobotProgram)\n"
        "function PauseFuncProgram()\n"
        "while (true)\n"
        "do\n"
        "wait(100)  --sync\n"
        "end\n"
        "end\n"
        "PauseFuncProgram_Result = task_create(PauseFuncProgram)\n"
        "end\n"
        "mainFuncProgram_Result = task_create(mainFuncProgram)"
result_download = cgxiapi.cr_downloadProgram(robotHandle, programfile)  #加载程序
if result_download.value==0:
    result_play = cgxiapi.cr_play(robotHandle)  #运行程序
    if result_play.value==0:
        result_existPop = cgxiapi.cr_script_popup_exist(robotHandle)
#是否存在弹窗：1-是，0-否
        if result_existPop[0].value==0:
            if result_existPop[1].value==1:  #有弹窗信息
                popupMsg = basestruct.PopUpMsg()
                result_getPop=cgxiapi.cr_script_popup_msg_get(robotHandle,
popupMsg)
                if result_getPop.value==0:
                    print("行数：", inspect.currentframe().f_lineno, "  读取弹窗信息
成功")
                    print("  行  数  ：  ", inspect.currentframe().f_lineno, "      ",
popupMsg.popupType)
                    str_popupMsg = str(popupMsg.var_data, 'utf-8')
                    print("弹窗内容", str_popupMsg)
                    result = cgxiapi.cr_stop(robotHandle)
                else:
                    print("行数：", inspect.currentframe().f_lineno, "      ", "结果：",
result_getPop.value)
            else:
                print("行数：", inspect.currentframe().f_lineno, "  没有弹窗信息")
        else:
            print("  行  数  ：  ", inspect.currentframe().f_lineno, "      ", "  结  果  ：  ",
result_existPop[0].value)
    else:
        print("行数：", inspect.currentframe().f_lineno, "      ", "结果：", result_play.value)
else:
    print("  行  数  ：  ", inspect.currentframe().f_lineno, "      ", "  结  果  ：  ",

```

```
result_download.value)
else:
    print("行数: ", inspect.currentframe().f_lineno, " ", "结果: ", re0[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()
```

4.8 数学变量

简介

本示例介绍了将当前基坐标系下的 TCP 位姿转换到点坐标系下的过程。

示例

```
import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

# 机械臂创建连接
robotHandle = 1
re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
# re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123") #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    pointCSNode = basestruct.PointCSNode()
    pointCSNode.name = b'point523'
    pointCSNode.point.toolAxisAngle = (30, 30, 30, 30, 30, 30)
    pointCSNode.point.toolPosition = (0, 0, 0, 0, 0, 0)
    result_kineF = cgxiapi.cr_kineForward(robotHandle, pointCSNode.point.toolAxisAngle,
    pointCSNode.point.toolPosition) #正解求点的位姿
    if result_kineF.value==0:
        pointCSNode.isValid = 1
        pointCSNode.id = 111
        result_add = cgxiapi.cr_cfg_cs_point_add(robotHandle, pointCSNode) #增加点坐标系
        if result_add.value==0:
            pose = [0, 0, 0, 0, 0, 0]
            result_getTcp = cgxiapi.cr_get_tcpActualPose(robotHandle, pose)
            #读实际 TCP 位置
            if result_getTcp.value==0:
                basePoseLen = 6
                userPoseLen = 6
```



```

        poseInUserLen = 6
        basePose=[0, 0, 0, 0, 0, 0]
        userPose = [0, 0, 0, 0, 0, 0]
        i=0
        while i<6:
            basePose[i] = pose[i]
            userPose[i] = pointCSNode.point.toolPosition[i]
#将点坐标系设置为用户坐标系
            i+=1
        poseInUser = [0, 0, 0, 0, 0, 0]
        result_compute=cgxiapi.cr_compute_pose_base_to_user(basePose,
basePoseLen, userPose, userPoseLen, poseInUser,poseInUserLen)
#基坐标系下位姿转移到用户坐标系下
        if result_compute.value==0:
            print(poseInUser)
        else:
            print(" 行数： ", inspect.currentframe().f_lineno, " ", " 结果： ",
result_compute.value)
        else:
            print(" 行数： ", inspect.currentframe().f_lineno, " ", " 结果： ",
result_getTcp.value)
        else:
            print("行数： ", inspect.currentframe().f_lineno, " ", "结果： ", result_add.value)
        else:
            print("行数： ", inspect.currentframe().f_lineno, " ", "结果： ", result_kineF.value)
    else:
        print("行数： ", inspect.currentframe().f_lineno, " ", "结果： ", re0[0].value)
result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
if result_destroy.value!=0:
    print("断开失败")
    exit()

```

4.9 TCP 操作

简介

本示例介绍了与 TCP 操作相关的接口的使用方法，包括读取 TCP、修改 TCP、增加 TCP、删除 TCP、激活 TCP、读取当前激活的 TCP 索引。

示例

```

import cgxiapi
from ctypes import *
import time
import basestruct
import inspect

```

```
# 机械臂创建连接
robotHandle = 1
# re0 = cgxiapi.cr_create_robot(robotHandle, "192.168.6.6", 2323, "123")
re0=cgxiapi.cr_create_robot(robotHandle, "127.0.0.1", 2325,"123") #虚拟臂连接

if re0[0].value == 0:
    robotHandle = re0[1].value
    result_activeTcp = cgxiapi.cr_cfg_tcp_active_get(robotHandle)
    #读取当前激活的 TCP 索引
    if result_activeTcp[0].value==0:
        index = 0
        tcpMsg = basestruct.TCPMsg()
        result_getTcp = cgxiapi.cr_cfg_tcp_get(robotHandle, index, tcpMsg)
        #需要先存在索引号为 0 的 TCP
        if result_getTcp.value==0:
            offset = [3,10,9,90,80,70]
            i=0
            while i<6:
                tcpMsg.tcpOffset[i] = offset[i]
                i+=1
            result_setTcp = cgxiapi.cr_cfg_tcp_set(robotHandle, index, tcpMsg)
            #编辑索引号为 0 的 TCP 数据
            if result_setTcp.value==0:
                tcpMsg2 = basestruct.TCPMsg()
                tcpMsg2.tcpName = b'tcp_222'
                offset2 = [5,10,15,10,20,30]
                i = 0
                while i < 6:
                    tcpMsg2.tcpOffset[i] = offset2[i]
                    i += 1
                result_add = cgxiapi.cr_cfg_tcp_add(robotHandle, tcpMsg2) #增加 tcp
                if result_add.value==0:
                    index = 1
                    result_setActive = cgxiapi.cr_cfg_tcp_active_set(robotHandle, index)
                    #读取当前激活的 TCP 索引
                    if result_setActive.value==0:
                        result_activeTcp2 = cgxiapi.cr_cfg_tcp_active_get(robotHandle)
                        # 读取当前激活的 TCP 索引
                        if result_activeTcp2[0].value==0:
                            print("行数: ", inspect.currentframe().f_lineno, " 当前激活
的 TCP 索引为",result_activeTcp2[1].value)
                            index = 0
                            result_delete = cgxiapi.cr_cfg_tcp_delete(robotHandle, index)
```

```
#删除索引号为 0 的 TCP
```

```
        if result_delete.value==0:
            print("行数：", inspect.currentframe().f_lineno, " 已成功删除索引号为 0 的 TCP")
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_delete.value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_activeTcp2[0].value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_setActive.value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_add.value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_setTcp.value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_getTcp.value)
        else:
            print("行数：", inspect.currentframe().f_lineno, " ", "结果：", result_activeTcp[0].value)
    else:
        print("行数：", inspect.currentframe().f_lineno, " ", "结果：", re0[0].value)
    result_destroy=cgxiapi.cr_destroy_robot(robotHandle)
    if result_destroy.value!=0:
        print("断开失败")
    exit()
```

附录 A 速度和加速度范围

各机型对应速度范围

机型	关节速度范围限制($^{\circ}/s$)	TCP 速度范围限制(mm/s)
C3	1-180	0.01-1000
C4/C6/C6-DK	1-180	0.01-2000
G3	1-360	0.01-2000
G3a	1-270	0.01-1500
G4/G6/G7	1-360	0.01-3000
G4a/G6a	1-270	0.01-2000
G9/G10/G12	1-240	0.01-3000
G9a/G12a	1-180	0.01-2000
G18	1-240	0.01-2000
G18a	1-180	0.01-1500
G20	1-270	0.01-4000
G30	1-270	0.01-3000
X6	1-360	0.01-3000
X9/X12/X18	1-240	0.01-3000

各机型对应加速度范围

关节加速度限值统一为 $0.01\sim 2500\text{ }^{\circ}/s^2$;

TCP 加速度限值统一为 $0.01\sim 20000\text{ mm}/s^2$ 。

各机型对应点动速度范围

关节点动速度限值统一为 $0.1\sim15\text{ }^{\circ}/\text{s}$;

关节点动加速度限值统一为 $0.01\sim90\text{ }^{\circ}/\text{s}^2$ 。

TCP 点动速度限值统一为 $0.1\sim120\text{ mm/s}$;

TCP 点动加速度限值统一为 $0.01\sim720\text{ mm/s}^2$ 。